



National University of Computer and Emerging Sciences, Lahore



A.R.T.E.M.I.S: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

Hashir Rauf Roll No. 23L-2572 BS(DS)

Hira Khalid Roll No. 23L-2594 BS(DS)

Syeda Doureesha Batool Roll No. 23L-2651 BS(DS)

Supervisor: Mr. Mubasher Hussain

Final Year Project

November 9, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: A.R.T.E.M.I.S: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: November 9, 2026

Name: Hashir Rauf

Signature: .....

Name: Hira Khalid

Signature: .....

Name: Syeda Doureesha Batool

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

People lose time returning to interrupted work, and the assistants capable of relieving this require broad access to a user's machine, trading away the privacy and control that make such a tool acceptable. This report presents A.R.T.E.M.I.S., a bounded, opt-in desktop assistant that operates only on folders the user has explicitly granted, deletes nothing, and routes every state-changing or outward action through preview and approval with undo available afterwards. A questionnaire of 374 respondents and 13 interviews confirmed the problem and supported the central hypothesis, with 65 percent rating a workspace-only assistant more trustworthy than a full-access one. The report specifies the requirements, architecture and design by which this restriction is enforced structurally rather than promised behaviourally, describes the working prototype that implements it, and reports the first-phase results. The enforcement layer passed all 554 automated tests and held under adversarial use, while an interface campaign of 27 sessions found 71 distinct defects that are carried into the second phase. Of 30 prospective users who were consulted, 26 expressed trust in such an assistant and 10 of them fully, on the condition that it does exactly what it says. The evidence supports the direction of the hypothesis, partly supports adoption, and leaves the measurement of usefulness to the controlled study planned for the second phase.

Executive Summary

This report is the third deliverable of final year project F26-008 and documents the problem, the supporting research, the requirements, the design, the implemented prototype and the first-phase results of A.R.T.E.M.I.S., short for Adaptive Reasoning, Task Execution and Multi-agent Intelligence System, is a desktop assistant built on the idea that a user should not have to choose between a useful assistant and a safe one. It only works inside folders the user has chosen to give it, keeps nothing from outside them, never deletes anything, and shows a preview before it changes a file or sends anything off the computer. The user has to approve that preview. A panel called "What ARTEMIS Knows" lists everything the system currently holds, and any entry can be removed. Our hypothesis is that these limits are a strength and not a cost: an assistant that stays inside a scope the user picked will be trusted more, and so used more, than one that does not.

Foundational research tested the problem and the hypothesis before any design was committed to. A questionnaire drew 374 valid responses and 13 interviews were coded against a five-axis guide. The problem was confirmed, with 44 percent of respondents struggling to recover context after a break and 42 percent losing more than an hour a week to the surrounding friction. The hypothesis was supported, with 65 percent rating a workspace-bounded assistant more trustworthy than a full-access one, and local-first operation emerging as the most common deployment preference. Two findings shaped the design directly. Respondents divided almost evenly on whether an assistant should ask before acting, which made approval granularity a configurable setting rather than a fixed decision. They were also forgiving of a single mistake where recovery was easy, which made visible, reliable undo more important than eliminating every error.

The architecture turns the hypothesis into structure through three enforcement points. A workspace broker is the only component that resolves a path, and it canonicalises before testing containment, which is what makes traversal and symbolic link escapes impossible rather than merely discouraged. A policy engine classifies every proposed action into one of four risk tiers, where reads run automatically, reversible changes are gated by user configuration, outward actions are always gated, and destructive operations are refused because no code path to them exists. A tool dispatcher is the single point at which anything executes, and it refuses any call lacking a matching policy verdict, with every change recorded for reversal before it is made. The review also identified three limitations the project inherits rather than solves, being approval fatigue, the difficulty of detecting unfaithful generated text, and the unreliability of citations, and these are reported as known limitations throughout.

The prototype built in the first phase implements all eight stages of the build sequence, from the

workspace foundation to an optional remote access plane, in about 14,250 lines of Python, and is verified by 554 automated tests that all pass. The enforcement layer added well under a millisecond to a call and held under hostile use: across a campaign of 27 interface sessions nothing was deleted, nothing outside the chosen folder was reachable, and nothing left the computer without approval. The same campaign found 71 distinct defects, one of them critical, because a passing component test suite did not guarantee a working interface. Thirty prospective users were consulted, and 26 of them expressed trust in an assistant that works as described, 10 fully, on the condition that it does exactly what it says. These results support the direction of the central hypothesis and its trust component, partly support adoption, and leave the effect on usefulness to be measured by the controlled study that is planned for the second phase together with the correction of the recorded defects.

The report then proceeds from the problem and its vocabulary to the project vision, the literature and related applications, the requirements, the approach and methodology, the design, the implementation and its test cases, the user manual, the experimental results, and finally the conclusions with the plan for the second phase.

Table of Contents

List of Figures **xii**

List of Tables **xiv**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 2

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 4

2 Project Vision **5**

2.1 Problem Domain Overview 5

2.2 Problem Statement 6

2.3 Problem Elaboration 6

2.4 Goals and Objectives 8

2.5 Project Scope 9

2.6 Sustainable Development Goal (SDG) 10

2.7 Constraints 11

2.8 Business Opportunity 12

2.9 Stakeholders Description/ User Characteristics 12

2.9.1 Stakeholders Summary 12

2.9.2 Key High-Level Goals and Problems of Stakeholders 13

2.10 Conclusion 15

3 Literature Review / Related Work **16**

3.1 Definitions, Acronyms, and Abbreviations 16

3.2 Detailed Literature Review 16

3.2.1 Task Interruption and Resumption 16

3.2.2 Bounded Artificial Intelligence Agents and Trust 17

3.2.3 Agentic Reasoning, Planning and Undo . . . 18
3.2.4 Voice Interaction Parity . . . 19
3.2.5 Code Assistance with Human Review . . . 20
3.2.6 Hallucination in Artificial Intelligence Summarisation . . . 20
3.2.7 Retrieval-Augmented Generation Citation Accuracy . . . 21
3.2.8 Related Development Projects and Applications . . . 22
3.3 Literature Review Summary Table . . . 22
3.4 Conclusion . . . 25

4 Software Requirement Specifications 26
4.1 List of Features . . . 26
4.2 Functional Requirements . . . 27
4.2.1 Workspace Management . . . 27
4.2.2 Action Gating and Recovery . . . 28
4.2.3 Capability Features . . . 29
4.2.4 Disclosure and Model Access . . . 30
4.3 Quality Attributes . . . 30
4.4 Non-Functional Requirements . . . 31
4.5 Assumptions . . . 32
4.6 Use Cases . . . 33
4.7 Hardware and Software Requirements . . . 36
4.7.1 Hardware Requirements . . . 36
4.7.2 Software Requirements . . . 37
4.8 Graphical User Interface . . . 38
4.9 Database Design . . . 43
4.9.1 ER Diagram . . . 43
4.9.2 Data Dictionary . . . 43
4.10 Risk Analysis . . . 46
4.11 Conclusion . . . 47

5 Proposed Approach and Methodology 48
5.1 Overall Approach . . . 48
5.2 The Risk Classification Procedure . . . 49
5.3 The Recovery Procedure . . . 50
5.4 The Prompt Injection Defence . . . 51

5.5 Development Methods . . . 51
5.6 Evaluation Methodology . . . 53
5.7 Conclusion . . . 53

6 High-Level and Low-Level Design 54
6.1 System Overview . . . 54
6.2 Design Considerations . . . 54
6.2.1 Assumptions and Dependencies . . . 54
6.2.2 General Constraints . . . 55
6.2.3 Goals and Guidelines . . . 56
6.2.4 Development Methods . . . 56
6.3 System Architecture . . . 58
6.3.1 Subsystem Architecture: The Gateway . . . 60
6.3.2 Subsystem Architecture: Capability Services . . . 62
6.4 Architectural Strategies . . . 66
6.4.1 Local-First Operation with Optional Egress . . . 66
6.4.2 Separation of Memory from the File Index . . . 67
6.4.3 Disclosure as a Live Query . . . 69
6.4.4 The Remote Plane as a Detachable Component . . . 69
6.4.5 Asynchrony with Bounded Queues . . . 69
6.5 Domain Model/Class Diagram . . . 71
6.6 Policies and Tactics . . . 72
6.6.1 Refusal as the Default Failure Mode . . . 74
6.6.2 Single Enforcement Points . . . 74
6.6.3 Ordering Constraints . . . 74
6.6.4 Testing Strategy . . . 74
6.6.5 Concurrency Handling . . . 75
6.6.6 Source Organisation . . . 75
6.7 Conclusion . . . 75

7 Implementation and Test Cases 76
7.1 Implementation . . . 76
7.1.1 Scope and Status of the Prototype . . . 76
7.1.2 Platform and Interfaces . . . 77
7.1.3 Implementation of the Workspace Broker . . . 78

7.1.4 Implementation of the Policy Engine and Approval Broker 79
7.1.5 Implementation of the Dispatcher and the Undo Journal 79
7.1.6 Implementation of the Gateway 80
7.1.7 Implementation of Indexing, Retrieval and Grounding 80
7.1.8 Implementation of the Capability Services 81
7.1.9 Implementation of Disclosure 81
7.1.10 Implementation of Speech 81
7.1.11 Implementation of the Remote Plane 82
7.1.12 Implementation of the User Interface 82
7.2 Test case Design and description 82
7.2.1 Test Cases 83
7.3 Test Metrics 92
7.3.1 Traceability Matrix 93
7.4 Conclusion 94

8 User Manual **95**
8.1 Conclusion 95

9 Experimental Results and Discussion **96**
9.1 E1: Automated Gate and Unit Tests 96
9.2 E2: Performance Measurements 97
9.3 E3: Hands-on Rounds Through the Interface 98
9.4 E4: Exploratory Interface Campaign 99
9.5 E5: Observations of Prospective Users 102
9.6 E6: Exploratory Persona Simulation 103
9.7 Discussion 104
9.7.1 Findings against the Central Hypothesis 104
9.7.2 Findings against the Research Question 105
9.7.3 Threats to Validity 105
9.7.4 Evaluation Not Yet Done 105
9.8 Conclusion 105

10 Conclusions **107**
10.1 Summary of the Work Done 107
10.2 Findings and Results 107
10.3 Coverage of the Scope and Objectives 107

10.4 Challenges Faced . . . 108
10.5 What Has Been Left Out and Why . . . 109
10.6 Recommendations and Future Work . . . 109
10.7 Plan for FYP-II . . . 109
10.8 Conclusion of the Report . . . 111

A Research Instruments and Response Distributions 116

A.1 Research Method . . . 116
A.2 Respondent Profile . . . 116
A.3 Headline Findings . . . 117
A.4 Segment Analysis . . . 118
A.5 Interview Findings . . . 118
A.6 Additional Response Distributions . . . 119

List of Figures

2.1 **Respondent role and work frequency.** . . . 5
2.2 **Difficulty resuming work after a break.** . . . 7
2.3 **Estimated time lost per week.** . . . 7
2.4 **Reasons for not automating repeated manual tasks.** . . . 8
2.5 **The seventeen UN Sustainable Development Goals.** . . . 11
2.6 **SDGs targeted by this project.** . . . 11
2.7 **Approval-first versus act-then-undo.** . . . 14

4.1 **Feature demand (n = 374).** . . . 27
4.2 **Navigation flow of the control panel.** . . . 39
4.3 **Work screen: ask, tidy and voice.** . . . 39
4.4 **Folders screen: add and list folders.** . . . 40
4.5 **What it knows screen: see and erase.** . . . 40
4.6 **Approval card for a tidy-up plan.** . . . 41
4.7 **A tidy-up, its approval and its undo.** . . . 41
4.8 **Settings screen: data location and cloud models.** . . . 42
4.9 **Remote access controls in Settings.** . . . 42
4.10 **The Work screen in the dark theme.** . . . 43
4.11 **Entity relationship diagram.** . . . 44

6.1 **The layered architecture.** . . . 57
6.2 **The request lifecycle.** . . . 59
6.3 **Internal structure of the gateway.** . . . 60
6.4 **Tier selection and the egress gate.** . . . 61
6.5 **The four risk tiers.** . . . 62
6.6 **Context assembly.** . . . 63
6.7 **Policy evaluation order.** . . . 63
6.8 **A worked classification for a folder tidy.** . . . 64
6.9 **Capability services and their access path.** . . . 64

6.10 Path resolution and confinement. . . . 66
6.11 Five kinds of memory. . . . 68
6.12 The file index compared with system memory. . . . 68
6.13 The remote access plane. . . . 70
6.14 The pairing handshake. . . . 71
6.15 Asynchrony and background progress. . . . 72
6.16 Class diagram. . . . 73

9.1 Results of the interface campaign. . . . 100
9.2 Stated trust among 30 prospective users. . . . 102

A.1 Coded interview responses (13 interviews). . . . 119
A.2 Agreement that files become disorganised. . . . 119
A.3 Agreement on workspace-only trust. . . . 120
A.4 Local versus cloud preference. . . . 120
A.5 Response to a single wrong or unwanted suggestion. . . . 120
A.6 Likelihood of continued use after the novelty. . . . 121
A.7 Most likely reason for stopping after a week. . . . 121
A.8 Recurring themes in free-text responses. . . . 121

List of Tables

2.1 **Stakeholders and their concerns.** . . . 13

3.1 **Summary of the reviewed literature.** . . . 22

4.1 **Use case UC-1: grant a folder as a workspace.** . . . 33
4.2 **Use case UC-2: tidy a folder.** . . . 33
4.3 **Use case UC-3: attempt to reach outside a workspace.** . . . 34
4.4 **Use case UC-4: draft and send electronic mail.** . . . 35
4.5 **Host hardware requirements.** . . . 36
4.6 **Software requirements.** . . . 37
4.7 **Data dictionary of the relational system of record.** . . . 45
4.8 **Project risks.** . . . 46

5.1 **The four risk tiers.** . . . 49
5.2 **The build sequence.** . . . 52

6.1 **Component inventory.** . . . 58
6.2 **Capability services and risk tiers.** . . . 65
6.3 **The degradation contract.** . . . 67

7.1 **Implementation status by stage.** . . . 76
7.2 **Platform, libraries and services.** . . . 77
7.3 **Test case TC-01: workspace boundary.** . . . 83
7.4 **Test case TC-02: policy engine classification.** . . . 84
7.5 **Test case TC-03: tool dispatcher.** . . . 85
7.6 **Test case TC-04: undo of an approved plan.** . . . 86
7.7 **Test case TC-05: approval broker.** . . . 87
7.8 **Test case TC-06: grounding gate.** . . . 88
7.9 **Test case TC-07: outward actions.** . . . 89
7.10 **Test case TC-08: remote device limits.** . . . 90
7.11 **Test case TC-09: speech and the approval gates.** . . . 91

7.12 Test metrics. 92

7.13 Requirements traceability. 93

9.1 The experiments reported in this chapter. 96

9.2 Enforcement and retrieval cost. 97

9.3 Time to scan and to index a folder, by size. 97

9.4 Results of the two hands-on rounds. 98

9.5 Session verdicts by area and stage gate status. 99

9.6 Stated trust, 30 prospective users. 102

9.7 Persona simulation results. 103

9.8 Assessment of the central hypothesis. 104

10.1 Status of the project objectives. 107

10.2 Plan of work for the second phase. 110

A.1 Respondent role and work frequency. 116

A.2 Selected measures by respondent segment. 118

Chapter 1 Introduction

Most of us do not get to work on one thing from start to finish. A student writing a thesis, a developer chasing a bug and a freelancer putting together a client report all jump between many files, and all of them get interrupted. When they come back, the files are still there but the thread is gone. They have to work out again which files mattered, what was already finished and what they meant to do next. On top of that comes a steady stream of small jobs such as sorting folders, pulling together notes scattered over several documents and writing routine emails. None of these jobs moves the real work forward, but they eat time every week.

AI assistants say they can take this load off us, and partly they can. The catch is what they ask for in return. Some tools only work if the user tells them every step, so the user is still doing the coordinating. The more capable tools want broad access to the whole machine, and in return they can do real multi-step work. That deal puts many people off. If an assistant can see everything, the user needs to know what it keeps, what it is about to do and how to undo it, and with most tools they cannot find any of that out.

A.R.T.E.M.I.S., an adaptive reasoning, task execution and multi-agent intelligence system, is a desktop assistant built on the position that these two properties need not be traded against one another. The system operates only within folders a user has explicitly granted, holds nothing outside them, deletes nothing, and passes every action that changes a file or leaves the machine through a preview and approval step that the user controls. A panel called “What ARTEMIS Knows” reports everything the system currently holds, and every entry in it can be deleted. The project’s hypothesis is that this restriction is not a sacrifice but an advantage, and that an assistant confined to a scope the user chose will be trusted, and therefore used, more readily than one that is not.

This chapter explains why this document exists, who it is written for and what the terms and abbreviations in it mean.

1.1 Purpose of this Document

This document sets out the problem, the research behind it, the requirements and the design of A.R.T.E.M.I.S., a desktop AI assistant that helps with everyday work inside folders the user chooses to share with it. It covers what we completed in the first phase, which is the specification, the design, the working prototype and the results so far, and it ends with our plan for the second phase.

The project asks one question: can an assistant that is limited to folders the user has explicitly granted still carry out useful multi-step tasks, while making users trust it more and without losing much useful-

ness? Our hypothesis is that the limit raises trust and adoption at a cost to capability that people will accept.

From this question we drew four objectives. We want to build an assistant that works strictly inside granted workspaces. We want to cut the time and effort people spend getting back into interrupted work and doing routine file jobs. We want the user to stay in control of every important action through preview, approval and undo. And we want to evaluate the result on task completion time, mental workload, usability and trust.

In this phase we used a questionnaire with 374 respondents, 13 interviews that we coded, a structured review of 26 studies grouped into seven themes, and an architecture derived from all three. Two limits apply and we want to state them plainly. First, the document reports only the checks we could make without recruiting participants. The controlled evaluation described in Chapter 5 belongs to the second phase. Second, we assume one user on one desktop computer throughout, so multi-user and enterprise use are outside the scope.

1.2 Intended Audience

The main readers are the evaluation panel and our supervisor. They need to judge whether the problem is real, whether the research backs our solution and whether the design is sound and can be finished in the time left. For them the document focuses on justification, and every major design decision is linked to the finding that led to it.

The second reader is the project team. Chapters 4 and 6 are what we build against, and we wrote the requirements so that each one can be checked when a feature is done, not argued about afterwards.

The third reader is anyone who extends the system later. For them the design chapter lists the rules that must survive any change. The two most important are that only one component turns a path into a real location, and only one component carries out actions.

1.3 Definitions, Acronyms, and Abbreviations

The following terms are used throughout this report.

ARTEMIS: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System, the assistant specified by this report

Workspace: a folder the user has explicitly granted, and the only region of the filesystem the system may access

Grant: the record of a user's decision to opt a folder in as a workspace, carrying its permission level and cloud policy

Workspace Broker: the single component that resolves a logical reference into a real filesystem path, or refuses

Policy Engine: the component that classifies each proposed action into a risk tier and decides whether it runs, is gated, or is refused

Tool Dispatcher: the single component permitted to execute an action

Plan: an ordered set of proposed actions produced by the planner, and the unit at which approval and undo operate

Tool Call: one proposed action within a plan

Verdict: the policy engine's decision about a single tool call

Risk Tier: the classification of an action as read, reversible, outward or destructive, written T0 to T3

Egress: the transfer of any data out of the user's machine

Approval Gate: the point at which a proposed action is held until the user approves or rejects it

Undo Journal: the record of how to reverse each change, written before that change is made

Audit Log: the append-only, hash-chained record of every proposal, refusal and action

Canonicalisation: resolving a path to its true location, following symbolic links and relative segments

AI: Artificial Intelligence

LLM: Large Language Model

RAG: Retrieval-Augmented Generation

STT: Speech to Text

TTS: Text to Speech

VAD: Voice Activity Detection

CLI: Command Line Interface

GUI: Graphical User Interface

PWA: Progressive Web Application

API: Application Programming Interface

UI: User Interface

UX: User Experience

FYP: Final Year Project

MVP: Minimum Viable Product

SDG: Sustainable Development Goal

SRS: Software Requirement Specification

DO: Durable Object, a stateful coordination primitive used in the optional remote access plane

1.4 Conclusion

This chapter described the problem of interrupted knowledge work, the poor deal that current assistants offer, and the purpose, readers and vocabulary of this report. The rest of the report is organised as follows. Chapter 2 gives the project vision: goals, scope, constraints, the sustainable development goal and the stakeholders. Chapter 3 reviews the literature and related applications in seven themes and says where this project fills a gap and where it inherits a limitation. Chapter 4 specifies the requirements, with features, functional and non-functional requirements, use cases and risks. Chapter 5 explains our approach and development method. Chapter 6 presents the design: the architecture, the domain model and the policies we follow in implementation. Chapter 7 describes the prototype and its test cases and metrics. Chapter 8 is kept for the user manual, which we will write in the second phase. Chapter 9 reports the experiments from this phase and what they say about our hypothesis. Chapter 10 gives the conclusions, the scope covered, what was left out and our plan for the second phase.

Chapter 2 Project Vision

Here we say what the project is for. We describe the problem area, state the problem and break it into smaller parts, set the goals, draw the edges of the scope, name the sustainable development goal we target, list our constraints, note the business opportunity and describe the people who would use the system. The evidence comes from our own early research, a questionnaire with 374 respondents and 13 coded interviews, which Chapter 3 covers in more detail.

2.1 Problem Domain Overview

Domain: personal knowledge work, done by one person at a desktop across many files. Users are students, developers, freelancers and employed knowledge workers. Their work is spread over artefacts and broken up in time. One task may touch documents, source files, notes, messages and web pages, and it is started and stopped many times over days or weeks.

The survey backs this up. Out of 374 people, 82 percent work across multiple files every day or a few times a week, and nobody does it less than a few times a month. Figure 2.1 breaks down who answered and how often they do this kind of work.

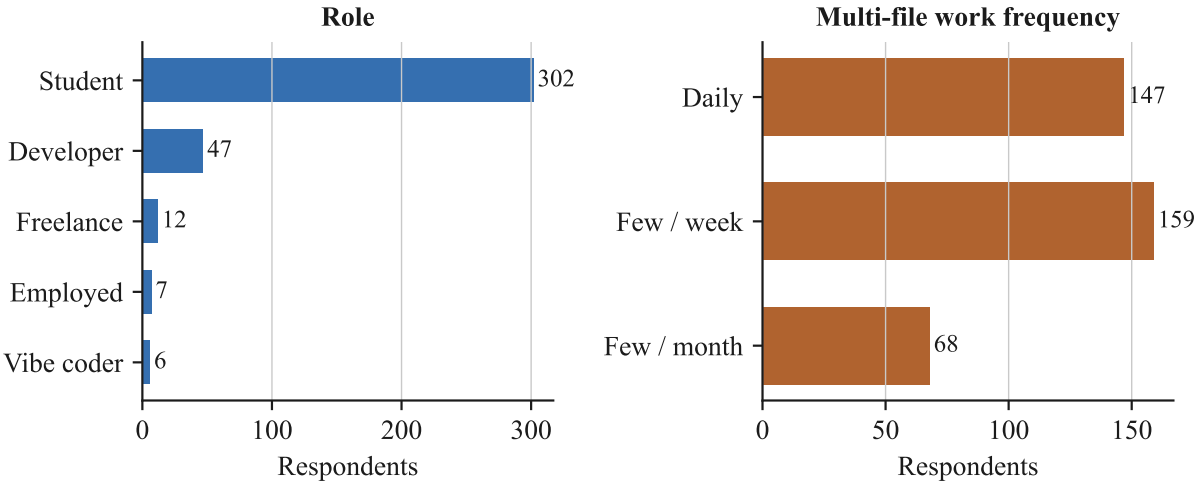


Figure 2.1: Respondent role and work frequency.

The sample is student-heavy, which matches the intended early-adopter profile, and multi-file work is frequent across every group.

Within this domain ARTEMIS is a desktop assistant that operates on folders the user has explicitly granted. It restores context after an interruption by reporting what was recently active and what was intended next. It organises files by proposing a set of moves the user previews and approves. It gathers material from several documents into a synthesis in which each claim carries a citation back to its source.

It observes repeated manual sequences and offers, after the third repetition, to automate them. It reads source code to explain errors and proposes fixes as a difference the user reviews. It searches the web on demand, prepares electronic mail for approval before sending, and drafts documents as new files. Each of these is available by voice as well as by typing.

To see what the boundary means in practice, consider three things the system cannot do. It cannot open a path outside a granted workspace. It has no operation for deleting user data. And it cannot change a file or send anything off the machine until the user has approved it.

2.2 Problem Statement

Users of multi-file knowledge work lose substantial time and effort re-establishing context after interruptions and performing routine file and correspondence tasks, and the artificial intelligence assistants capable of relieving this burden require broad, unbounded access to the user's machine, which creates privacy exposure, risk of unintended modification and loss of user control that many users are unwilling to accept.

2.3 Problem Elaboration

When we broke the problem down, we found five separate parts, and the project handles each one differently.

The first part is losing the thread after an interruption. Coming back after a day or more, the clues for where you were are spread over the filesystem, the open applications and your own memory, and memory is the part that lets you down. In our survey 44 percent said they often or always struggle to remember what they were doing when they return to a project. Only 7 percent never do. Figure 2.2 has the full picture.

The figure shows how often respondents return to a project after a break of a day or more and struggle to recall what they were doing or what to do next.

Second, file disorder builds up. Folders get messy all the time and are cleaned only now and then, so they drift further from any order that would make things easy to find. 59 percent of respondents agreed or strongly agreed that their folders get disorganised faster than they can clean them (mean 3.6 out of 5).

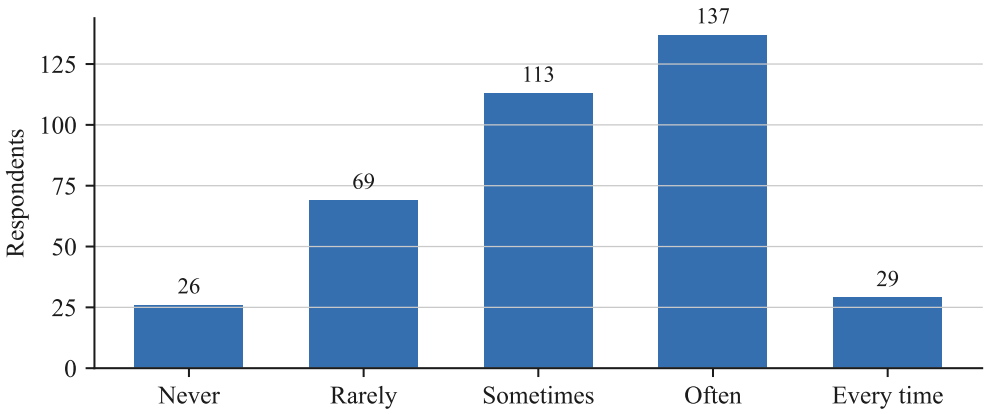


Figure 2.2: Difficulty resuming work after a break.

Third, the small jobs add up. People were asked how much time they lose each week to getting reoriented, tidying, repeating manual tasks and hunting through scattered notes. 42 percent said more than an hour, and 14 percent said more than three. Most answers sat between thirty and sixty minutes (Figure 2.3). It is a steady drain and not a crisis, and that is what the system is meant to ease.

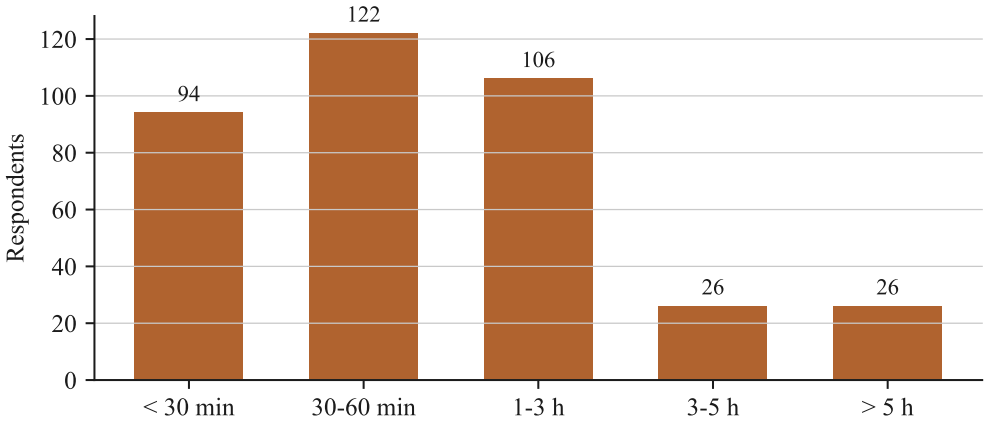


Figure 2.3: Estimated time lost per week.

The estimate covers reorienting after breaks, tidying files, repeating manual tasks and searching scattered notes.

Fourth, why isn't this work already automated? We asked, and the two biggest answers were not trusting automation to get it right (36 percent) and not knowing how to set it up (28 percent), as Figure 2.4 shows. The jobs are real and people know about them. What is missing is a tool they are willing to point at those jobs.

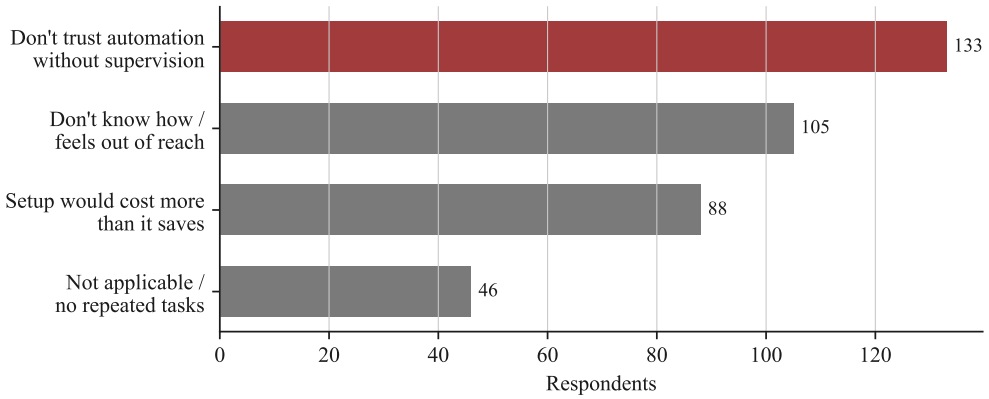


Figure 2.4: Reasons for not automating repeated manual tasks.

Distrust of automation and lack of know-how dominate.

The fifth part is the one that sets this project apart, and it is the bargain existing assistants ask for. A capable assistant usually gets wide access to the machine, and then the user cannot tell what it holds, what it is allowed to do or how to undo it. We tested the alternative in the questionnaire. 65 percent of respondents rated an assistant limited to files they had added as meaningfully more trustworthy than one with full access, with a mean of 3.8 out of 5.

2.4 Goals and Objectives

Our goal is to show that an assistant restricted to what the user opted in can still do useful multi-step automation, build trust in a way we can measure, and leave the user in control.

The objectives that realise this goal are as follows.

- To develop a desktop assistant that operates strictly within folders the user has explicitly granted, such that access outside a grant is structurally impossible rather than merely prohibited
- To reduce the time and effort users spend resuming interrupted work and performing routine file, document and correspondence tasks
- To keep the user in control of every consequential action by routing it through preview, approval and undo
- To guarantee that the system deletes no user data, by providing no code path to a destructive operation
- To make everything the system holds visible and individually deletable through a disclosure panel
- To support voice as a complete interaction mode subject to the same approval gates as typing

- To evaluate the system on task completion time, cognitive workload, usability and trust, and to report inherited limitations honestly

2.5 Project Scope

The scope of this project is to design, build and evaluate a desktop assistant that helps a single user manage daily work within explicitly granted workspaces. The system will allow the user to:

- Grant a folder as a workspace, and revoke it so that everything derived from it is removed
- Resume interrupted work by reviewing what was recently active and what was intended next
- Tidy a folder by previewing and approving a proposed set of moves
- Synthesise notes across several documents with each claim traceable to its source
- Receive an offer to automate a manual sequence after it has been repeated three times
- Ask questions about source code and receive explanations and proposed fixes as differences
- Search the web on demand
- Draft electronic mail and send it only after approving the draft
- Draft documents, always written as new files
- Perform any of the above by voice as well as by typing
- Inspect everything the system holds, and delete any part of it
- Reverse an approved plan as a single unit

The system will be implemented in Python. Multi-step plans will be produced using LangGraph [1], which provides an explicit pause point before any workspace-changing or outward call. Language model access will be local-first through Ollama [2], with an optional and per-workspace cloud fallback. Persistence will use local SQLite [3] together with a per-workspace vector index. The interface will comprise a command line surface and a local graphical control panel.

The project deliverables will be:

- A working desktop assistant implementing the features above
- This specification and design document, and the evaluation report that follows it
- A user manual
- A test suite covering the boundary, policy, approval and undo guarantees

The project scope does not include the following:

- Access to any part of the filesystem outside a granted workspace
- Deletion or unsnapshotted overwriting of user data
- Autonomous action, meaning any file change or outward action taken without approval
- Background browsing, screen recording, keystroke capture or always-on audio recording
- Mobile applications, beyond the optional remote control surface described in Chapter 6
- Multi-user, shared or enterprise deployment
- Hosting user data on any project-operated server

2.6 Sustainable Development Goal (SDG)

The project targets Sustainable Development Goal 9, Industry, Innovation and Infrastructure, and in particular target 9.5, which concerns enhancing scientific research and upgrading technological capabilities [4]. The complete list of goals is shown in Figure 2.5.

The project's contribution to this goal is its local-first design. The system's default configuration performs its work using a model running on the user's own machine, which means the capability it provides does not depend on continuous connectivity to a data centre, on a subscription, or on the user's willingness to transmit personal documents to a third party. This matters for the population the project targets. Students and independent practitioners, particularly in regions where connectivity is intermittent or costly, are precisely the users for whom a cloud-dependent assistant is least accessible, and a local-first assistant extends a current technological capability to them on terms they can meet. Our own data supports the preference: local-first operation was the single most common deployment choice among respondents at 40 percent, against 22 percent who actively preferred cloud.

The project also contributes to Goal 8, Decent Work and Economic Growth, through target 8.2 on productivity through technological upgrading, since the system's purpose is to remove recurring friction from knowledge work and return that time to substantive activity.

The figure shows the goals as numbered coloured tiles, each combining the goal number, its short title and a representative icon. This project targets Goal 9, Industry, Innovation and Infrastructure, and secondarily Goal 8, Decent Work and Economic Growth.

The goals this project addresses are shown explicitly in Figure 2.6: **SDG 9, Industry, Innovation and Infrastructure** (target 9.5) as the primary goal, and **SDG 8, Decent Work and Economic Growth**



Figure 2.5: The seventeen UN Sustainable Development Goals.

(target 8.2) as the secondary goal.



Figure 2.6: SDGs targeted by this project.

2.7 Constraints

The project operates under the following constraints, each of which shapes the design presented in Chapter 6.

- The system must run on a typical student laptop, which bounds the size of the local language model, the resident memory footprint and the acceptable index size
- Core operation must not require internet connectivity, so any capability that depends on the network must degrade to a stated limitation rather than an error
- User data must not leave the machine except under an explicit per-workspace opt-in, which places the egress decision in the model router rather than in any feature
- The system must hold no inbound listening port beyond the loopback interface, so the optional remote plane must dial outward rather than accept connections

- Credentials must reside in the operating system keyring and never on disk in the system's own storage tree
- The project must be completed by a team of three within two academic semesters, which requires that the highest-risk components be built and validated early
- The evaluation must be conducted with participants drawn from a single institution's networks, which bounds the generalisability of its conclusions

2.8 Business Opportunity

The assistant market is presently divided between tools that are capable but demand broad access and tools that are constrained but require step-by-step direction. The opportunity this project addresses lies between them, and our data indicates that the gap is occupied by real demand rather than by a hypothetical preference: a majority of respondents rated a bounded assistant as more trustworthy, and distrust of automation was the single most common reason given for not having automated this work already.

Three conditions make the opportunity actionable. Local language models have become capable enough to run useful inference on consumer hardware, which makes local-first operation practical rather than aspirational. Regulatory and personal sensitivity to data handling continues to rise, which increases the value of an assistant that can state precisely what it holds. The target population is already familiar with assistants, since 77 percent of our respondents have used one, so the product does not have to establish the category.

The corresponding risk is differentiation, and our data identifies it plainly. A recurring theme in free-text responses was that the concept appeared already covered by existing developer assistants. The answer is the bounded workspace and local-first operation, and the finding indicates that this must be stated explicitly in the product rather than left for the user to infer.

2.9 Stakeholders Description/ User Characteristics

The users of this system are individuals performing knowledge work on their own machines, together with the roles involved in evaluating and maintaining the project.

2.9.1 Stakeholders Summary

Table 2.1 identifies each stakeholder, their role with respect to the system, and their principal concern.

Table 2.1: Stakeholders and their concerns.

Stakeholder	Role	Primary concern
Student	Primary user. Works across documents, notes and source files on a personal laptop, frequently interrupted. Represents 81 percent of the research sample.	Recovering context quickly; keeping personal and academic files private; no configuration burden.
Developer	Primary user. Works across source files and documentation, and is the principal user of the code assistant. Represents 13 percent of the sample.	Correctness of proposed changes; never having code modified without review.
Freelancer or knowledge worker	Primary user. Manages client documents and correspondence.	Confidentiality of client material; reliable correspondence drafting with approval.
Project supervisor	Assesses the project's academic soundness and progress.	Traceability of design decisions to evidence; feasibility within the timeline.
Evaluation panel	Assesses the delivered project against its stated objectives.	Whether the hypothesis was tested and whether limitations are stated honestly.
Development team	Designs, implements and evaluates the system.	Preserving the architectural invariants under change; completing the scope in time.
Future maintainer	Extends the system after the project concludes.	Locating the enforcement points; understanding which properties must not be broken.

2.9.2 Key High-Level Goals and Problems of Stakeholders

The primary users share a single high-level goal, which is to spend less time on the mechanics of their work and more on its substance, and they share a single high-level problem, which is that the tools capable of granting this require a degree of access they are not prepared to give. Our research refines this general statement in four respects that bear directly on the design.

Users are divided on how much they wish to be asked. The questionnaire presented a choice between an assistant that seeks approval before every file change or outward action and one that acts on reasonable requests and permits review and undo afterwards. The split was 192 to 182, shown in Figure 2.7. No single interaction model satisfies this population, and approval granularity is therefore specified as a user-configurable setting in Chapter 4 rather than fixed in the design.

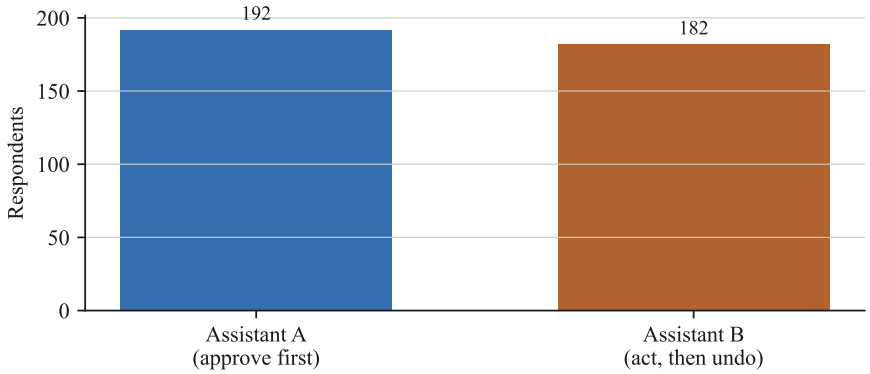


Figure 2.7: Approval-first versus act-then-undo.

The population is close to evenly divided.

Users will tolerate a mistake if they can reverse it. Asked what a single wrong or unwanted suggestion would do to their usage, 65 percent said they would continue using the system, some with increased caution, and only 10 percent would abandon it. This finding establishes that a visible and reliable undo is more valuable than an unattainable perfection, and it is why the undo journal is specified as a critical component rather than a convenience.

Users will stop for reasons of fit rather than friction. The most frequently predicted reasons for abandoning the system after a week were that it did not fit existing habits, at 27 percent, and that it made a mistake the user could not trust it to avoid again, at 24 percent. Approval fatigue was the smallest named risk at 10 percent. This tempers a natural design instinct: reducing the number of prompts addresses the least common concern and would erode the trust that the approval model provides.

Users want the system to learn their structure, which is a request the bounded design must accommodate carefully. The most frequently requested capability not already planned was that the assistant learn the user's folder organisation over time. This is specified as per-workspace memory, visible and deletable in the disclosure panel, so that personalisation remains inside the bounded-scope guarantee rather than quietly widening it.

2.10 Conclusion

This chapter has stated the vision of the project. It described a problem with two halves, the cost of resuming interrupted work and the unwillingness of many users to give an assistant broad access, and it bounded the work by goals, a scope, exclusions and constraints. It identified the sustainable development goal the project supports, the opportunity it addresses and the stakeholders whose characteristics govern the design. The evidence for all of this is the foundational research, whose distributions are given in Appendix A. The chapter that follows reviews the literature against which these claims are tested.

Chapter 3 Literature Review / Related Work

What has other work already shown, and what is still missing? This chapter looks at published research and at existing applications to find out. We reviewed 26 studies in seven themes, each tied to something the system has to do or guarantee. For each theme we summarise the work, weigh its strengths and weaknesses and say how it relates to our system. At the end we are honest about three things: where the project truly fills a gap, where it fills one only partly, and where it inherits a limit we cannot claim to have solved.

3.1 Definitions, Acronyms, and Abbreviations

The following terms are used specifically within this chapter, in addition to those defined in Chapter 1.

Agentic system: a system in which a language model plans a sequence of steps and invokes tools to carry them out, rather than only producing text

Human-in-the-loop: an arrangement in which a person approves or corrects a system's proposed action before it takes effect

Prompt injection: an attack in which instructions embedded in retrieved content are treated by a model as commands from the user

Hallucination: generated content that is not supported by the source material it purports to summarise

Attribution: the property that a generated statement is genuinely supported by the source it cites

Faithfulness: the consistency of a generated summary with the document it summarises

Approval fatigue: the decline in scrutiny that occurs when a user is asked to approve actions frequently

Privilege separation: an architectural defence that isolates a trusted component from untrusted data or code

3.2 Detailed Literature Review

There are seven themes. The first three cover the core behaviour of the system: resuming tasks, bounded agents and trust, and agentic planning. The next two cover particular capabilities, voice and code assistance. The last two are about whether generated text is correct, covering hallucination and the accuracy of citations.

3.2.1 Task Interruption and Resumption

The story starts with a simple observation. In a field study of ordinary office computer use, researchers saw that when people get pulled away from a task, cues on the screen, such as a window left open and

not minimised, are what help them pick it up again later [5]. They studied real working conditions, not a lab. But the study ended at diagnosis. It offered no system for the case where leaving a window open stops being practical, for example after a restart or a gap of several days.

Later work turned that diagnosis into a tool. ScreenTrack takes a time-lapse of the screen in the background, and the user scrolls through it to find and reopen something they remember seeing. The reasoning is that people remember a run of visual scenes better than file names or timestamps [6]. In a controlled lab study it did well for recent activity that looked distinct.

The strengths and weaknesses of this pair are complementary. The field study's strength is ecological validity, and its weakness is that it produced no intervention. ScreenTrack's strength is that it operationalises the finding, and its weaknesses were acknowledged candidly by its authors: retrieval slows as the history lengthens, the approach depends on tasks being visually distinguishable and so suits text-heavy work poorly, and it was never tested in prolonged multi-application use. There is also an evident tension between a continuous screen recording and the privacy expectations of the users this project targets.

The relationship to the proposed work is one of deliberate divergence rather than extension. The resumption feature specified in this report does not record the screen at all. It operates from the file index and from notes the user has written, matching on file name, recency and explicit text, which makes it independent of whether two screens resemble one another. Because a workspace is bounded by construction, the feature also avoids the problem of searching a long history, since only a short list of recent and relevant items exists to present. The honest qualification is that the resumption benefit remains a design claim rather than a demonstrated one, since the system has not undergone field validation comparable to the original study.

3.2.2 Bounded Artificial Intelligence Agents and Trust

Another thread asks how to keep an agent's autonomy inside limits that a person can trust. One answer is aimed at developers. A trace-based assurance framework writes down every action an agent takes as a structured trace, with machine-checkable contracts on each step. A violation can then be traced to an exact point and replayed the same way each time. Each action is allowed, rewritten or blocked according to limits set for that agent [7].

A recent security survey examines the same space from the perspective of systems that depend on human sign-off [8]. It analyses privilege separation as an architectural defence, drawing an analogy to the separation an operating system maintains between a trusted kernel and untrusted programs, and applies this to isolating a planning component from untrusted external data. It then examines the limits of human-in-

the-loop approval itself, and its conclusion is unfavourable: frequent approval prompts produce decision fatigue that undermines the safety benefit they exist to provide, and most users lack the background to judge what they are being asked to approve, so they either approve risky actions without real scrutiny or reject harmless ones. The survey names this as an open problem and offers no solution.

The good news from this body of work is that the machinery for bounding an agent already exists and is well understood. The catch, for our purposes, is that it was built for organisations that have security staff and supporting infrastructure, not for one person running an assistant on their own machine. The assurance framework also has no way to undo an action once it has run.

The relationship to the proposed work runs in both directions. This project's workspace model is a plain-language rendering of the same principles: a folder the user selects rather than a sandbox an administrator defines, and a readable preview rather than a machine-checked contract. Against the assurance framework the project adds precisely what that framework lacks, which is reversal after the fact, and this is among the most direct correspondences in the review. Against the security survey the project must concede a point rather than claim one. It does not solve approval fatigue. It relies on the same mechanism the survey identifies as unresolved, and nothing in the current design distinguishes a routine approval from a risky one or reduces prompts as trust accumulates. This limitation is inherited by choosing this approach at all, and the evaluation must test for it directly rather than assume the bounded workspace dissolves it.

3.2.3 Agentic Reasoning, Planning and Undo

A substantial body of work has moved language models beyond producing text toward reasoning about a goal, planning steps and acting on them. Reflexion allows an agent to evaluate its own past attempts, write verbal reflections and store them as episodic memory to improve later attempts without retraining, improving performance on decision-making, reasoning and coding tasks [9]. Toolformer takes a narrower approach, teaching a model in a largely self-supervised manner when and how to call an external tool such as a calculator or a search engine [10]. Generative Agents extends the memory-and-reflection idea further, demonstrating that a population of agents can behave coherently over simulated days by storing, retrieving and reflecting on experience, while also reporting failures including incorrect retrieval and fabricated memory detail [11]. ReAct interleaves reasoning, acting and observing within a single loop and shows that the combination improves both performance and interpretability, while proving prone to repetitive action loops and poor recovery when an action returns an unhelpful result [12]. Tree of Thoughts permits a model to search across multiple reasoning paths and backtrack from unpromising ones, producing substantial gains on puzzle-style tasks at a real computational cost [13].

The strength of this literature is that it establishes reasoning, tool use and reflective memory as a workable substrate for agent behaviour. Its shared limitation is the setting in which each system is evaluated: Reflexion on its own past attempts, Toolformer on isolated tool calls it cannot readily chain, Generative Agents within a sandbox world, and ReAct and Tree of Thoughts on benchmark tasks. None maintains a bounded workspace for a real user across ordinary interrupted work, and none requires the kind of approval that would render consequential actions reversible.

Our work sits in that gap. We take the same mix of reasoning, tool use and memory, but point it at a user's real files, notes and stated intentions and not at the agent's own history or a simulated setting. We also wrap the multi-step plans in preview, approval and undo, which none of these systems provide. The failures this research reports are a warning to us as well. Made-up memories and repetitive reasoning loops are exactly the errors the resume and synthesis features must guard against. They also argue for keeping few unsupervised steps between points where the user can check, instead of assuming a longer autonomous chain is safe.

3.2.4 Voice Interaction Parity

This theme concerns whether people can carry out practical computer work by voice as effectively as by typing, and how voice interaction should be evaluated. Two systematic reviews establish the state of the field. The first conducts a bibliometric analysis of user experience with voice interfaces and proposes a classification of six research categories, identifying voice-related factors including gender, accent, personality and style [14]. The second reviews voice assistant usability against the ISO 9241-11 framework of effectiveness, efficiency and satisfaction [15].

Three empirical studies give concrete design advice. The first shows that speech carries a real-time signal of how the user is experiencing the system, one that after-the-fact questionnaires miss. Pause length and the rate of disfluencies can serve as extra measures [16]. The second looks at spoken indicators during waits. For a long wait, an indicator that explains what is going on and adds a little humour works best, while for a short wait either one is enough and extra information only adds load [17]. The third looks at delay in free conversation with virtual agents and found that natural fillers beat artificial tones [18].

What this literature gives us is usable thresholds and design rules, including a target of four seconds for latency. What it lacks, as both reviews admit, is a whole picture: voice usability is studied in pieces, and standard usability measures leave out voice concerns such as trust and mental load.

For us it matters in two ways. On the design side, the findings on waiting indicators and fillers decide what the voice mode does while a plan is being made: it should speak an explanation, not stay silent or play a generic tone. On the evaluation side, the studies agree that voice as a full way of doing practical

multi-step tasks has hardly been studied. That is our contribution. We compare voice with typing on ease of use, task time, mental workload and trust. The speech-analysis technique is an optional extra for the evaluation and not a design requirement.

3.2.5 Code Assistance with Human Review

Five studies address the conditions under which language model assistance with code is useful. HULA demonstrates a plan, generate and human-approve loop in a real deployment, reporting an 82 percent plan approval rate and finding that output quality depends heavily on the quality of the input context [19]. HiLDe operates at a finer granularity, steering generation token by token, and reports two cautions: regeneration can silently overwrite earlier user edits, and presenting too many alternatives overwhelms users [20]. A study of suggestion timing examines when an inline assistant should display a suggestion and warns against optimising for acceptance rate at the expense of usefulness [21]. A taxonomy of expert steering strategies identifies the moves by which a person redirects a model, including pointing out a specific error, clarifying a requirement and re-orienting the approach, while noting that some strategies presuppose expert knowledge [22]. Finally, an evaluation of language models as code reviewers finds systematic overcorrection, with models inventing requirements and flagging correct code as wrong, and finds that requesting longer explanations worsens rather than improves this [23].

This literature shows that a human-approval loop is a good way to interact and not a fallback. Its shared weakness, put most sharply by the overcorrection study, is that a model's judgement about code helps but cannot be trusted alone, and longer explanations do not make it more reliable.

The link to our work is direct. The code assistant in this report follows HULA's shape but is narrower. It reads source in the workspace, answers questions, explains errors and proposes a fix as a difference for the user to approve. It does not open pull requests or act across a repository. HiLDe's warnings became two design rules: show one proposed change at a time, and never change code the user has not approved. Because the user calls the assistant deliberately and it does not suggest continuously as they type, the suggestion-timing problem never comes up, which is a deliberate benefit of the on-demand model. The overcorrection finding is the direct reason we never apply a change automatically. Our difference-plus-approval design reduces that limit but does not remove it.

3.2.6 Hallucination in Artificial Intelligence Summarisation

Four studies trace this problem from diagnosis to meta-evaluation. The first established that hallucinated content appeared in every abstractive summarisation system tested, and identified entailment between source and output as the appropriate check [24]. FactCC followed with a lightweight, trainable consis-

tency checker, which remains the closest thing in the literature to a component that could be adopted directly [25]. FRANK then benchmarked the available factuality metrics and found that no single metric detects every category of error [26]. SummEdits extended the question to frontier models and found that even the strongest models judge faithfulness poorly [27].

The strength of this literature is the consistency of its conclusion across a decade: hallucination is common, conventional overlap metrics do not detect it, no single detector is uniformly reliable, and models are weak judges of their own faithfulness. Its weakness from an engineering standpoint is that it supplies no complete solution, only partial detectors and the recommendation to combine them.

We should call this what it is: a gap, not something we have resolved. The synthesis feature works on a small set of documents the user picks. That is a narrower, more controlled task than open-domain summarising, so there is less room for hallucination. Less is not none. Our current design has no faithfulness check, so the gap stays open and the bounded workspace does not close it. The direction the literature points to is an entailment-style or FactCC-style check after generation, together with keeping the user in the loop. The SummEdits finding also rules out the tempting shortcut of asking the model to check its own citations.

3.2.7 Retrieval-Augmented Generation Citation Accuracy

Do the sources a text cites really support what it says? Three studies look at this. The first gives a workable definition of attribution: a statement passes if, according to the cited source, it is true [28]. ALCE then tested how well models write text with citations and found that incomplete citation is common [29]. An audit of deployed generative search engines found that about a quarter of citations did not hold up, and about half of sentences were not fully supported [30].

This literature gives both a standard and evidence of how far real systems fall short of it. It stops at measurement, though, and offers no way to enforce correctness.

Our relationship to it is partial. The synthesis feature removes half of the retrieval problem, because the user chooses the documents, and the citation map in Chapter 6 traces each chunk back to its file and line. What the design does not do is check that a citation really supports its claim, or that every claim has one. The audit of deployed products is a useful reality check. Mature products with dedicated citation engineering still fail these checks at meaningful rates. So citation checking is necessary work for us, and citing from a small known set should not be assumed safe.

3.2.8 Related Development Projects and Applications

Outside the research papers, three kinds of existing application matter to us. Inline coding assistants sit inside a development environment and suggest as you type. They are quick, but they only handle code and can reach whatever the editor can reach. Agentic command-line assistants run multi-step plans across a developer's machine. They are really capable, but they need broad access and are hard to reverse. General conversational assistants are capable too, but they have no lasting, structured link to the user's filesystem, so they cannot resume work or act on files unless the material is handed to them each time.

Our own research shows this is about positioning, not only a technical comparison. Many free-text answers in our questionnaire said the idea looked already covered by developer assistants, so we have to argue the difference openly. The things that set us apart are the enforced workspace boundary, local-first operation by default, destructive operations being structurally unavailable, and a disclosure panel that lets anyone check the trust claim and not simply take it on faith.

3.3 Literature Review Summary Table

Table 3.1 summarises the reviewed studies, identifying the theme each addresses, the contribution it makes, its relevance to this project and its limitations.

Table 3.1: Summary of the reviewed literature.

Study	**Theme**	**Relevance to this project**	**Limitations**
Iqbal and Horvitz [5]	Resumption	Establishes that resumption depends on persistent cues, motivating the session resume feature	Diagnosis only; proposes no system; cues fail after restart or a multi-day gap
Hu and Lee [6]	Resumption	Demonstrates a resumption tool, whose weaknesses this project avoids structurally	Slow retrieval over long histories; requires visually distinct tasks; privacy cost of screen recording
Paduraru et al. [7]	Bounded agents	Contract mediation and capability limits inform the policy engine	Built for organisations with security staff; no mechanism to undo an executed action

Table 3.1: continued from previous page

Study	Theme	Relevance to this project	Limitations
Kim et al. [8]	Bounded agents	Privilege separation informs the planner and dispatcher split; documents approval fatigue	Names approval fatigue as an open problem without offering a solution
Shinn et al. [9]	Agentic planning	Episodic written memory as a substrate for agent behaviour	Evaluated on the agent's own past attempts, not on real user work
Schick et al. [10]	Agentic planning	Self-supervised tool invocation informs the tool contract	Isolated tool calls that cannot readily be chained into plans
Park et al. [11]	Agentic planning	Memory, retrieval and reflection architecture informs the memory design	Sandbox world; reports fabricated memory and incorrect retrieval
Yao et al. [12]	Agentic planning	Interleaved reasoning and acting informs the bounded reflection loop	Repetitive action loops; poor recovery from unhelpful observations
Yao et al. [13]	Agentic planning	Search over reasoning paths informs the cycle cap on planning	Substantial computational cost; benchmark tasks only
Deshmukh and Chalmeta [14]	Voice	Classification of voice user experience research informs evaluation measures	Voice usability studied in fragments; no unified framework
Dutsinma et al. [15]	Voice	ISO 9241-11 framing of usability informs the evaluation instrument	Standard metrics omit trust and cognitive load
Ma et al. [16]	Voice	Speech as a real-time experience signal; optional evaluation enhancement	Requires turn-by-turn audio capture, which this project does not plan
Zhang et al. [17]	Voice	Waiting-indicator design for periods while a plan is produced	Findings specific to the wait durations tested
Wang et al. [18]	Voice	Natural fillers preferred over tones during response delay	Studied in free-form conversation rather than task execution

Table 3.1: continued from previous page

Study	Theme	Relevance to this project	Limitations
Takerngsaksiri et al. [19]	Code assistance	Validates the plan, generate and approve loop at 82 percent plan approval	Output quality heavily dependent on supplied context
Anaya Gonzalez et al. [20]	Code assistance	Cautions against silent overwriting and excessive alternatives	Token-level granularity finer than this project requires
Mozannar et al. [21]	Code assistance	Establishes that on-demand invocation avoids the suggestion-timing problem	Applies to inline assistants; acceptance rate is a misleading objective
Chidambaram et al. [22]	Code assistance	Steering taxonomy informs follow-up prompts when a fix is rejected	Some strategies presuppose expert knowledge
Jin and Chen [23]	Code assistance	Direct justification for never applying a code change automatically	Models overcorrect; longer explanations worsen reliability
Maynez et al. [24]	Hallucination	Entailment identified as the appropriate faithfulness check	Hallucination present in every system tested; no complete remedy
Kryscinski et al. [25]	Hallucination	FactCC is the most directly adoptable consistency checker	Not currently used by this project; a stated open gap
Pagnoni et al. [26]	Hallucination	Warns against relying on a single faithfulness metric	No single metric detects every error category
Laban et al. [27]	Hallucination	Rules out self-checking of citations by the model	Even frontier models judge faithfulness poorly
Rashkin et al. [28]	Citation	Supplies the attribution standard for synthesis output	A standard rather than an enforcement mechanism
Gao et al. [29]	Citation	Benchmarks citation generation; motivates completeness checks	Incomplete citation common even in purpose-built systems

Table 3.1: continued from previous page

Study	Theme	Relevance to this project	Limitations
Liu et al. [30]	Citation	Calibrates expectations against deployed products	A quarter of citations unsupported; half of sentences not fully supported

The table gives the theme, contribution, relevance to this project and stated limitations of each study.

3.4 Conclusion

The reviewed literature establishes three conclusions for this project.

The project fills a genuine gap in four respects. It applies multi-step agentic planning to a bounded, user-chosen desktop workspace rather than to a simulated world, a single task or an open repository, a combination the planning literature leaves open. It makes agent memory visible and user-editable through the disclosure panel, where the reviewed systems run memory as an opaque background process and report fabricated and mis-retrieved entries. It provides reversal after a previewed action, which the assurance framework lacks entirely. It evaluates voice as a complete interaction mode over practical multi-step tasks, which both voice reviews identify as under-explored.

The project fills three further gaps partially. Its bounded capability scoping is a plain-language rendering of the capability limits and privilege separation the security literature describes, trading rigour for usability. Its resumption feature operationalises a well-established finding without having reproduced that study's field validation. Its code assistance adopts a validated human-approval loop while remaining subject to the underlying model's errors, which approval mitigates rather than removes.

The project inherits three limitations it does not solve, and these are stated here rather than deferred. It does not solve approval fatigue, which the security survey documents as an open problem in exactly the mechanism this design depends upon, and the evaluation must test for it over repeated sessions. It has no faithfulness check for synthesised text, so the narrower task reduces but does not eliminate the exposure the summarisation literature documents. It does not verify that citations support their claims or that every claim carries one, which the citation literature shows to be unreliable even in polished deployed products.

These conclusions shape the requirements in Chapter 4 and the design in Chapter 6, and they set the terms on which the evaluation in the second phase of the project must report its results.

Chapter 4 Software Requirement Specifications

Here we say what the system has to do. The chapter lists the features and the functional requirements with the users they serve, then the quality attributes and non-functional requirements and the assumptions behind them. After that come the main use cases, the hardware and software we need, the interface and the database design, and the risks. We wrote every requirement so that it can be checked: once a feature is built, you can test it against the requirement and do not have to judge it by feel.

Each requirement has a prefix and a number. Functional requirements run FR-1 onwards, non-functional requirements NFR-1 onwards and quality attributes QA-1 onwards.

4.1 List of Features

The system provides the following features, listed in the order in which they are built. The order is derived from measured demand, reported in Chapter 3, together with technical risk, so that the components on which every other feature depends are constructed and validated first.

- Workspace management, by which a user grants a folder, inspects what the system holds about it, and revokes it together with everything derived from it
- Action gating, by which every proposed action is classified by risk, previewed, approved or rejected, and reversed if required
- Session resume, which restores the user’s context after an interruption by reporting what was recently active and what was intended next
- Folder tidy, which proposes a set of file moves that the user previews and approves as one unit
- Repeated task detection, which offers to automate a manual sequence after it has been observed three times
- Notes synthesis, which gathers material across several documents with each claim carrying a citation to its source
- Code assistance, which explains errors and proposes fixes as differences for review
- Web search, performed only on the user’s explicit request
- Mail composition, in which a draft is prepared and sent only after the user approves it
- Document drafting, which produces a first draft as a new file
- Speech input and output, providing voice as an alternative interaction mode subject to identical

approval gates

- The disclosure panel, which reports everything the system currently holds and permits deletion of any part of it

The order follows measured demand, shown in Figure 4.1. People asked most for session resume, notes synthesis and folder tidy, and speech came last by a wide margin, so we build it last.

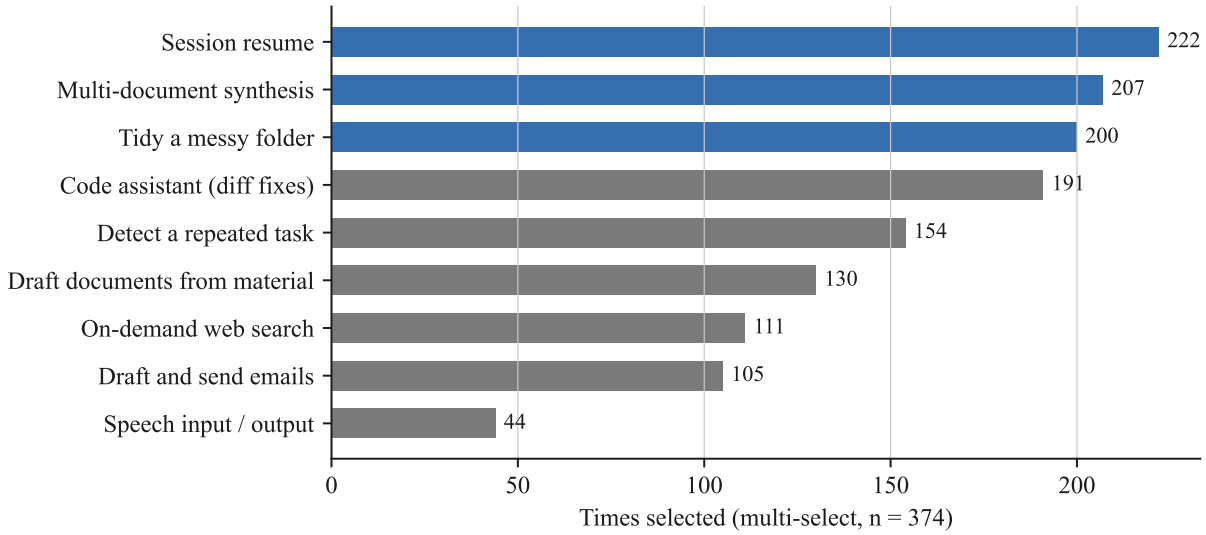


Figure 4.1: Feature demand (n = 374).

Demand is concentrated on session resume, notes synthesis and folder tidy.

4.2 Functional Requirements

The functional requirements describe the external behaviour of the system. Each is stated with the user who exercises it.

4.2.1 Workspace Management

- **FR-1** The system shall allow the user to grant a folder as a workspace, recording its name, canonical root path, permission level and cloud policy. User: all primary users.
- **FR-2** The system shall refuse to resolve any path whose canonical form lies outside the root of an active grant. User: all primary users, exercised indirectly.
- **FR-3** The system shall canonicalise a path, resolving symbolic links and relative segments, before testing whether it is contained within a grant. User: all primary users, exercised indirectly.
- **FR-4** The system shall refuse to resolve paths matching a deny list of credential and system locations, even where such a path lies within a granted folder. User: all primary users, exercised

indirectly.

- **FR-5** The system shall allow the user to revoke a workspace, deleting its index entries, embeddings, sessions, memory and rules, while leaving the user's own files untouched. User: all primary users.
- **FR-6** The system shall maintain an index of the files within each granted workspace, updating it incrementally as files change. User: all primary users, exercised indirectly.

4.2.2 Action Gating and Recovery

- **FR-7** The system shall classify every proposed action into exactly one of four risk tiers, being read, reversible, outward and destructive. User: all primary users, exercised indirectly.
- **FR-8** The system shall execute read actions within a granted workspace without requiring approval. User: all primary users.
- **FR-9** The system shall hold every reversible action for approval, unless the user has previously chosen to permit that operation in that workspace without asking. User: all primary users.
- **FR-10** The system shall hold every outward action for approval in all circumstances, and shall not permit this requirement to be configured away. User: all primary users.
- **FR-11** The system shall refuse every destructive action, providing no approval path by which it may proceed. User: all primary users.
- **FR-12** The system shall present, before any gated action, a preview stating the workspace affected, the operations proposed, the risk tier, whether anything leaves the machine, and whether anything is deleted. User: all primary users.
- **FR-13** The system shall allow the user to approve, approve and remember, edit or reject a pending plan. User: all primary users.
- **FR-14** The system shall scope a remembered approval to one operation within one workspace, and shall never remember an outward or destructive operation. User: all primary users.
- **FR-15** The system shall record how to reverse each change before that change is made. User: all primary users, exercised indirectly.
- **FR-16** The system shall allow the user to reverse an approved plan as a single unit, restoring its steps in reverse order. User: all primary users.
- **FR-17** The system shall never approve a pending action automatically, whether by timeout, by default or under load. User: all primary users, exercised indirectly.

- **FR-18** The system shall record every proposal, refusal, approval, rejection and execution in an append-only, hash-chained audit log. User: all primary users; auditor.

4.2.3 Capability Features

- **FR-19** The system shall report, on request, the files recently active within a workspace and the next actions the user recorded, so that interrupted work may be resumed. User: all primary users.
- **FR-20** The system shall propose a grouping of files within a folder, presenting every move for preview before any is performed. User: all primary users.
- **FR-21** The system shall detect a manual sequence repeated three or more times and offer to automate it, and shall not install an automation without the user's acceptance. User: all primary users.
- **FR-22** The system shall synthesise material across documents within a workspace, attaching to each claim a citation identifying its source file and location. User: all primary users.
- **FR-23** The system shall render as unsupported any claim in a synthesis that cannot be traced to a source. User: all primary users.
- **FR-24** The system shall answer questions about source code within a workspace and present any proposed change as a difference, and shall never write or execute code autonomously. User: developer.
- **FR-25** The system shall perform a web search only in response to an explicit user request, and shall not browse in the background. User: all primary users.
- **FR-26** The system shall compose electronic mail as a draft and send it only after the user has approved that draft. User: all primary users.
- **FR-27** The system shall write a drafted document as a new file and shall never overwrite an existing one. User: all primary users.
- **FR-28** The system shall accept spoken input and produce spoken output, subject to the same approval gates as typed interaction. User: all primary users.
- **FR-29** The system shall record audio only while a voice session has been explicitly armed by the user. User: all primary users.

4.2.4 Disclosure and Model Access

- **FR-30** The system shall present a panel reporting every workspace granted, file indexed, chunk embedded, fact remembered, session retained, automation rule, policy exception, action taken and item of data sent outside the machine. User: all primary users.
- **FR-31** The system shall allow the user to delete any item shown in that panel, with the exception of the audit log. User: all primary users.
- **FR-32** The system shall route model requests to a local model by default. User: all primary users, exercised indirectly.
- **FR-33** The system shall not transmit workspace content to a model tier that the workspace's cloud policy has disabled. User: all primary users, exercised indirectly.
- **FR-34** The system shall exclude from retrieved context any material outside the active grant, and shall record each such exclusion. User: all primary users, exercised indirectly.
- **FR-35** The system shall produce an inspectable plan without executing it, and shall permit execution only through the dispatcher and only where a matching policy verdict exists. User: all primary users, exercised indirectly.

4.3 Quality Attributes

The following quality attributes state the properties the system must hold, together with the means by which each is assessed.

- **QA-1 Confinement.** No operation resolves a path outside an active grant. Assessed by tests that attempt traversal using relative segments, symbolic links pointing outside the grant, absolute paths, and sibling directories sharing a name prefix, each of which must be refused and logged.
- **QA-2 Non-destructiveness.** No execution path exists that deletes or overwrites user data without a snapshot. Assessed by inspection of the dispatcher's registered operations and by tests confirming that destructive operations are refused rather than gated.
- **QA-3 Recoverability.** Every approved reversible plan can be reversed. Assessed by tests that execute a plan, reverse it, and compare the resulting filesystem state against the state recorded before execution.
- **QA-4 Transparency.** Everything the system holds is reportable and, save for the audit log, deletable. Assessed by comparing the panel's reported counts against direct queries of each store.

- **QA-5 Auditability.** The record of actions cannot be altered undetectably. Assessed by recomputing the hash chain and confirming that any modification or removal of an entry breaks it.
- **QA-6 Responsiveness.** The interactive path does not block on background work. Assessed by measuring time to first token while an initial workspace ingestion is in progress.
- **QA-7 Degradability.** Withdrawal of any external capability produces a stated limitation rather than a failure. Assessed against the degradation contract presented in Chapter 6.
- **QA-8 Modality parity.** Voice interaction traverses identical gates to typed interaction. Assessed by confirming that the introduction of the voice mode required no additional approval gate.

4.4 Non-Functional Requirements

- **NFR-1 Performance.** The interactive path shall produce a first token within one second when using a local model and within two seconds when using a cloud model, measured on the reference hardware specified in Section 4.7.
- **NFR-2 Performance.** Incremental index updates shall be debounced and shall not block interactive requests. Editing one line of a file shall cause only the affected chunk to be re-embedded.
- **NFR-3 Capacity.** The system shall operate within configurable ceilings on memory, index size and processor share, and shall remain usable on a consumer laptop while indexing.
- **NFR-4 Concurrency.** The system supports one interactive user per host instance. Background work shall be executed through a bounded priority queue that sheds the lowest-priority work when full rather than growing without limit.
- **NFR-5 Availability.** Core operation shall continue without internet connectivity. Capabilities requiring the network shall fail as explicit, retryable actions rather than being silently dropped.
- **NFR-6 Privacy.** User documents, embeddings, session history, memory, credentials and audit logs shall not be transmitted to any project-operated server.
- **NFR-7 Security.** Credentials shall be held in the operating system keyring and never written to the system's own storage tree. Data at rest shall be encrypted.
- **NFR-8 Security.** The system shall not open an inbound listening port other than on the loopback interface.
- **NFR-9 Usability.** Approval granularity shall be user-configurable, permitting approval of every action, approval by category, or session-scoped trust after initial approvals.

- **NFR-10 Usability.** Background work shall report visible, cancellable progress rather than an indeterminate indicator.
- **NFR-11 Reusability.** Capability features shall be implemented as plugins behind a uniform tool contract, such that a new feature may be added without modifying the gateway.
- **NFR-12 Extensibility.** Model providers shall be treated as configuration rather than as code branches, so that any compatible endpoint may be registered without modification to the router.
- **NFR-13 Portability.** The system shall operate on Windows, macOS and Linux, and shall not depend on any platform-specific filesystem behaviour for its confinement guarantee.
- **NFR-14 Maintainability.** Path resolution shall occur in exactly one component and execution in exactly one other, so that each guarantee has a single enforcement point.

4.5 Assumptions

The specification rests on the following assumptions.

- The user is a single individual operating their own machine, with the authority to grant access to the folders they select
- The host machine possesses sufficient resources to run a small local language model, or the user has enabled cloud fallback for the workspaces concerned
- The operating system provides a keyring suitable for credential storage and a canonicalisation routine that resolves symbolic links correctly
- The filesystem supports the metadata on which incremental indexing depends, namely modification times and stable path identity
- Folders granted as workspaces are not concurrently modified by another automated process in a manner that would invalidate a snapshot between its capture and the subsequent operation
- The user is capable of reading a preview stating what will occur and deciding upon it, which the literature notes is an assumption rather than a certainty and which the evaluation must examine
- Documents within a workspace may be adversarial, since a file may originate from any source, and the design therefore treats retrieved content as data rather than as instruction

4.6 Use Cases

This section presents the use cases that exercise the central functionality of the system or that illustrate a delicate point of the architecture. Four are given: granting a workspace, which establishes the boundary; tidying a folder, which exercises the gating and recovery path end to end; attempting to reach outside a workspace, which demonstrates the confinement guarantee; and sending mail, which demonstrates that an outward action cannot be configured past its gate.

Table 4.1: Use case UC-1: grant a folder as a workspace.

Name	UC-1 Grant a folder as a workspace		
Actors	Student, Developer, Freelancer		
Summary	The user selects a folder and opts it in as a workspace. The system records the grant, indexes the folder, and reports what it now holds.		
Pre-Conditions	The folder exists and is readable by the user. The folder is not already granted.		
Post-Conditions	A grant exists with a canonical root, a permission level and a cloud policy. The folder's files are indexed and the grant is recorded in the audit log.		
Special Requirements	The root is stored canonically, so that a later symbolic link cannot alter the region the grant covers.		
Basic Flow			
Actor Action		**System Response**	
1	The user selects a folder and names it.	2	The system canonicalises the path and records the grant with its permission and cloud policy.
3	The user confirms the permission level.	4	The system indexes the folder in the background, reporting visible progress, and writes the grant to the audit log.
Alternative Flow			
1-A	The user selects a folder already granted.	2-A	The system reports that the folder is already a workspace and makes no change.

Table 4.2: Use case UC-2: tidy a folder.

Name	UC-2 Tidy a folder
Actors	Student, Developer, Freelancer

Table 4.2: continued from previous page

Summary	The user asks the system to tidy a folder. The system proposes a plan of moves, the user previews and approves it, and the system executes it with undo available afterwards.
Pre-Conditions	The folder lies within a granted workspace with write permission. The workspace has been indexed.
Post-Conditions	The approved moves have been performed, the inverse of each is recorded, and the plan may be reversed as one unit. Nothing has been deleted.
Special Requirements	The snapshot is recorded before any move is performed. Undo operates at the granularity of the plan, not of the individual move.

Basic Flow			
Actor Action		**System Response**	
1	The user asks the system to tidy a folder.	2	The system lists the folder, reads metadata, and produces a plan of proposed moves.
		3	The system classifies each step, and presents a preview stating the risk tier, that nothing leaves the machine and that nothing is deleted.
4	The user reviews the moves and approves the plan.	5	The system records the inverse of each move, performs the moves, writes the audit entries and reports that undo is available.
Alternative Flow			
4-A	The user rejects the plan.	5-A	The system performs nothing, records the rejection in the audit log and halts cleanly.
4-B	The user approves, then later requests undo.	5-B	The system reverses every move in the plan in reverse order and reports any step it could not reverse.

Table 4.3: Use case UC-3: attempt to reach outside a workspace.

Name	UC-3 Attempt to reach outside a workspace
Actors	Capability service; indirectly, an adversarial document
Summary	A proposed action references a path outside the grant, whether through a relative segment, a symbolic link or an absolute path. The system refuses it and records the attempt.

Table 4.3: continued from previous page

Pre-Conditions	A workspace is granted and active.
Post-Conditions	No filesystem access occurred outside the grant. The refusal is recorded in the audit log with its reason.
Special Requirements	Canonicalisation precedes the containment test, so that a path which merely appears contained is resolved to its true location first.

Basic Flow			
Actor Action		**System Response**	
1	A service requests a path containing a traversal segment.	2	The broker canonicalises the path and finds it outside the grant root.
		3	The broker records the refusal and raises it, so no access occurs.
Alternative Flow			
1-A	The path is a symbolic link whose target lies outside the grant.	2-A	Canonicalisation resolves the link before the containment test, so the refusal is identical.
1-B	The path names a credential file inside the grant.	2-B	The deny list refuses it even though it lies within the granted folder.

Table 4.4: Use case UC-4: draft and send electronic mail.

Name	UC-4 Draft and send electronic mail
Actors	Freelancer, Student, Developer
Summary	The user asks the system to prepare a message. The system drafts it and holds it until the user approves the exact content and recipient.
Pre-Conditions	A mail account is configured and its credentials are held in the keyring. Any material referenced lies within a granted workspace.
Post-Conditions	The message is sent only if approved. The egress is recorded in the audit log with its recipient and time.
Special Requirements	This gate is not configurable. A remembered approval cannot be recorded for an outward operation.
Basic Flow	

Table 4.4: continued from previous page

Actor Action		System Response	
1	The user asks for a message to be prepared.	2	The system drafts the message and classifies the send step as an outward action.
		3	The system presents the recipient, subject and full body for approval.
4	The user reads the draft and approves it.	5	The system sends the message and records the egress in the audit log.
Alternative Flow			
4-A	The user requests that the approval be remembered.	5-A	The system declines to record the rule, because outward operations are never remembered.
4-B	The user rejects the draft.	5-B	Nothing is sent and the rejection is recorded.

Between them the four use cases touch the main parts of the architecture. UC-1 sets up the boundary and the index. UC-2 goes the full length of the request lifecycle: classification, preview, approval, snapshot, execution, audit and undo. UC-3 tests confinement against the three ways a bounded system can be escaped. UC-4 shows that the outward gate is built into the architecture and is not a setting someone can change.

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

Table 4.5 states the minimum and recommended hardware for the host machine. The minimum configuration is intended to run the system with a small local model, and the recommended configuration is the reference against which the performance requirements in Section 4.4 are measured.

Table 4.5: Host hardware requirements.

Component	Minimum	Recommended
Processor	Quad-core, 2.0 GHz	Eight-core, 2.5 GHz or higher
Memory	8 GB	16 GB or more, permitting a larger local model to remain resident
Storage	10 GB free, solid-state	25 GB free, solid-state, for indexes and snapshots

Table 4.5: continued from previous page

Component	Minimum	Recommended
Graphics	Not required; inference on processor	Dedicated graphics with 6 GB memory or more, for local inference
Audio	Microphone and output device, for the voice mode	As minimum
Network	Not required for core operation	Broadband, for cloud fallback and web search

4.7.2 Software Requirements

Table 4.6 states the software required to develop and operate the system, together with the purpose each serves.

Table 4.6: Software requirements.

Component	Version	Purpose
Operating system	Windows 10 or later, macOS 12 or later, or a current Linux distribution	Host platform
Python	3.11 or later	Implementation language for the entire system
LangGraph [1]	Current	Multi-step planning with an explicit pause before consequential calls
Ollama [2]	Current	Local model runtime, the default inference tier
SQLite [3]	3.35 or later	Local system of record for metadata, sessions, rules and audit
Local vector index	Current	Per-workspace embedding storage and retrieval
Typer	0.12 or later	Command line interface
Gradio	6.0 or later	Local graphical control panel served on the loopback interface

Table 4.6: continued from previous page

Component	Version	Purpose
Watchdog	4.0 or later	Filesystem change detection for incremental indexing
Operating system keyring	Platform-provided	Credential storage outside the system's own tree
Pytest	8.0 or later	Automated verification of the boundary, policy, approval and undo guarantees

4.8 Graphical User Interface

There are three ways to reach the system, all built over one shared view model so they always show the same state. The command line is for scripting, configuration and diagnosis. The local graphical control panel, served on the loopback interface only, is the main surface. The optional remote web app, described in Chapter 6, shows the same view model on another device and carries no extra privilege.

The control panel has five views. In the chat view you type or speak a request and see the reply and any plan. The approval view shows a pending plan with its preview: the workspace affected, the proposed operations, the risk tier, whether anything leaves the machine and whether anything is deleted. From there the user can approve, approve and remember, edit or reject. The disclosure view meets FR-30 and FR-31 by listing every category of data the system holds with its count and letting the user delete it. The workspace view meets FR-1 and FR-5, listing grants and offering revocation. The activity view shows the audit log and undo history for FR-16 and FR-18.

Navigation follows the life of a request. The user starts in the chat view. When a plan needs a decision, they are taken to the approval view, and afterwards they return to the chat with the result and an undo button. The disclosure, workspace and activity views can be opened at any time and are not on the request path. In the implementation each view is labelled with the requirements it meets, so the interface can be checked against the specification and not judged by impression.

Figure 4.2 shows the navigation flow, and Figures 4.3 to 4.10 show the screens of the principal surface. The Work screen and the approval card cover the request path (FR-8 to FR-16 and FR-28), the Folders screen covers FR-1 and FR-5, and the What it knows screen covers FR-30 and FR-31. The Settings screen holds the model, mail and remote access choices, and the interface can be switched to a dark theme.

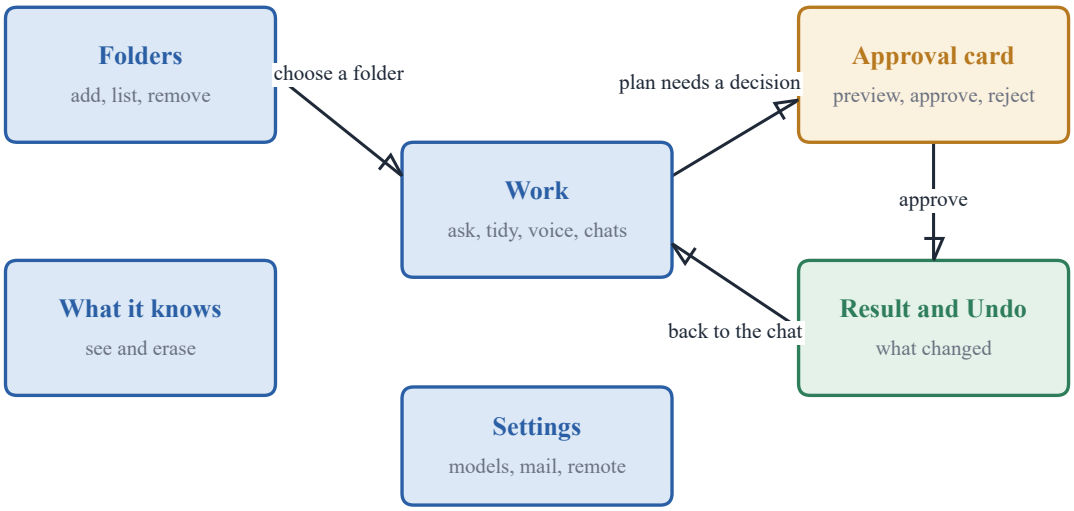


Figure 4.2: Navigation flow of the control panel.

The Work screen is where a request begins. The user chooses a folder, types or speaks what is wanted and presses Ask, or uses one of the shortcut buttons for a tidy-up, for what changed and for repeated tasks. The voice controls sit in a collapsible panel, and the saved chats are listed in the sidebar.

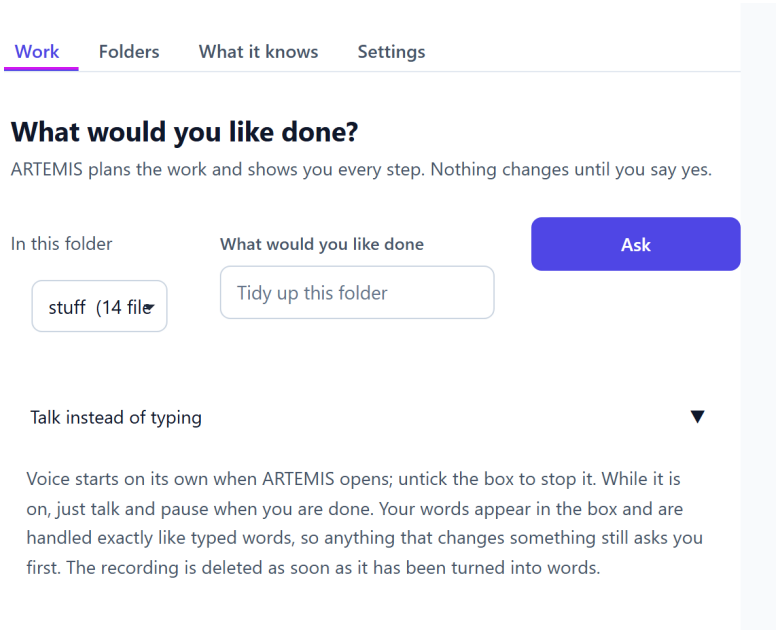


Figure 4.3: Work screen: ask, tidy and voice.

The Folders screen lets the user opt a folder in by typing its path or browsing to it. Each granted folder is listed with the number of files it holds, the permission it carries and whether cloud models are allowed for it. A folder that has not been added here cannot be seen by the assistant at all.

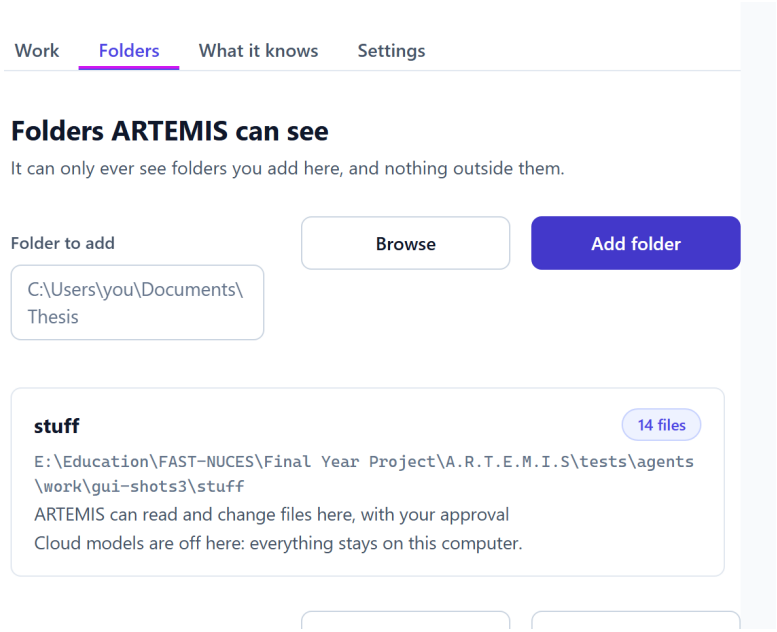


Figure 4.4: Folders screen: add and list folders.

The What it knows screen implements the disclosure requirement. For the chosen folder it lists every file the assistant has indexed, offers a check for changes, and offers a button that forgets the folder. Forgetting removes what the assistant holds and never touches the user's own files.

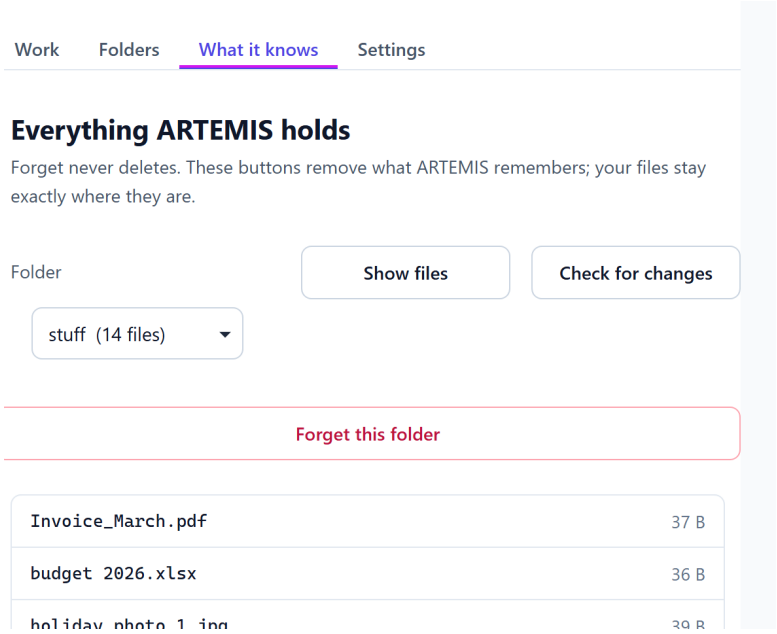


Figure 4.5: What it knows screen: see and erase.

When a request would change files, the assistant does not act. It shows a plan in plain words and three choices: Approve, Approve and do not ask again for this kind of change, or No, leave it. Nothing has happened until the user presses one of them.

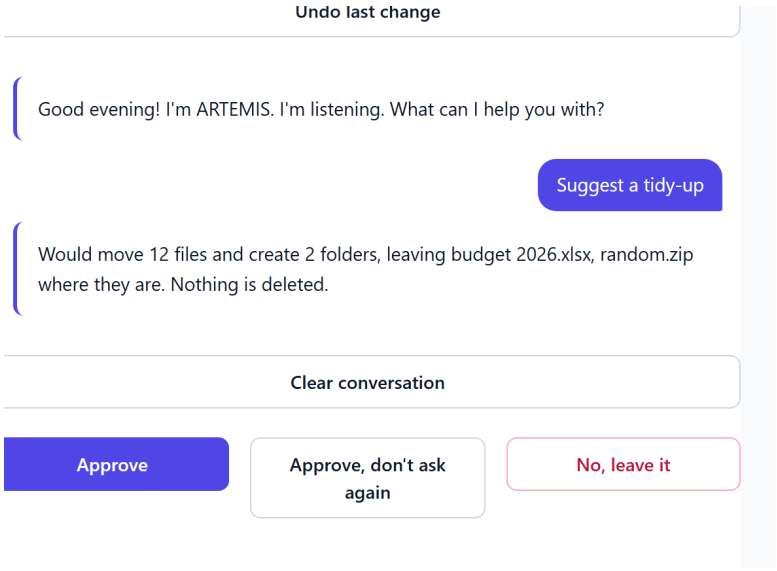


Figure 4.6: Approval card for a tidy-up plan.

After approval the assistant reports what it did, and the Undo last change button reverses the whole plan as one unit. Here a tidy-up that created two folders and moved twelve files is undone in a single step.

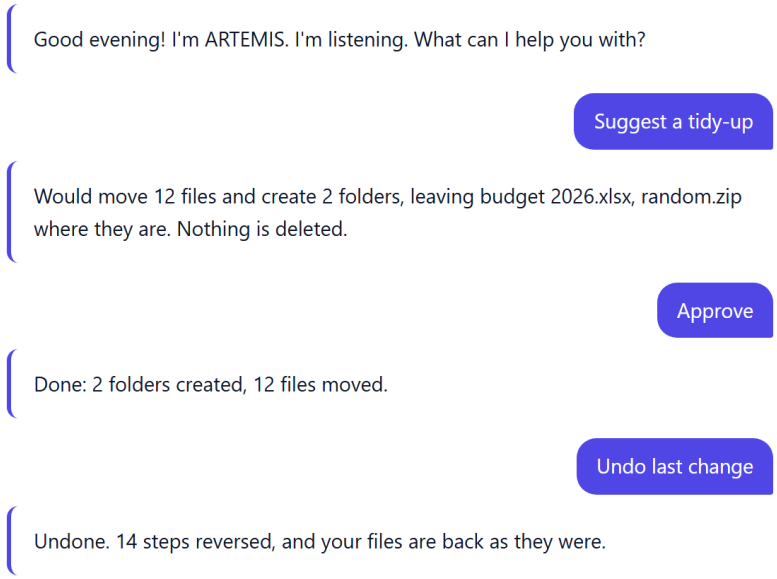


Figure 4.7: A tidy-up, its approval and its undo.

The Settings screen holds the choices that are not part of everyday use: where the assistant keeps what it knows, which models it may use, the mail account, and the remote access controls.

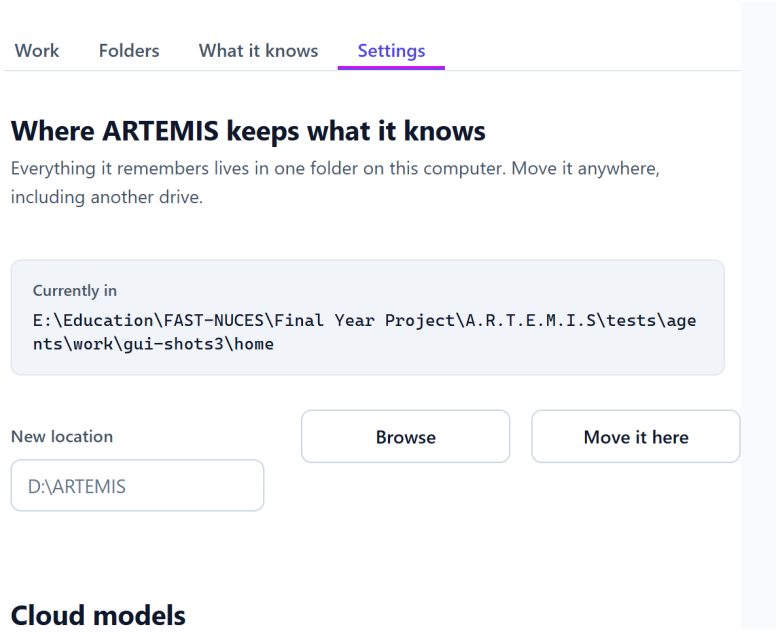


Figure 4.8: Settings screen: data location and cloud models.

Remote access is optional and off until the user connects to a relay. The screen states that messages are scrambled on the phone and only unscrambled on the computer, and it lists the folders a paired device may use, none by default.

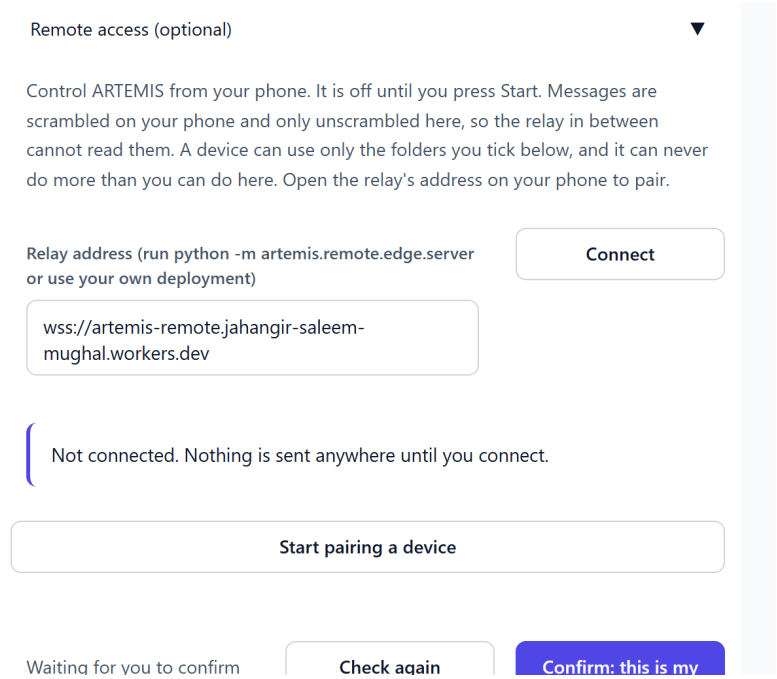


Figure 4.9: Remote access controls in Settings.

The interface can be switched between a light and a dark theme with one button, and the same screens and controls are shown in both.

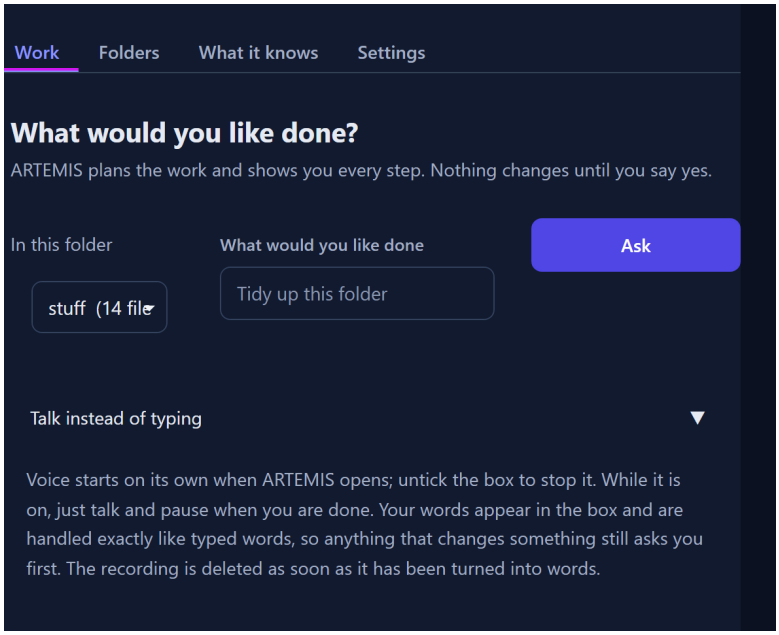


Figure 4.10: The Work screen in the dark theme.

4.9 Database Design

SQLite is the system of record, and each workspace also has its own vector index on the filesystem. The relational store is small and heavily normalised. Every table except the workspace table belongs to a grant, so revoking a grant is one cascading deletion.

4.9.1 ER Diagram

Everything in the ER diagram centres on the workspace. A workspace has many indexed files, sessions, memory entries, automation rules, policy rules and undo journal entries. Each of these is a one-to-many relationship, and deleting the workspace cascades to all of them. A session has many turns, which cascade in the same way. The audit log sits outside this on purpose. It refers to a workspace but does not belong to one, because the audit record of a revoked workspace has to outlive the revocation. A log that its subject can erase is not an audit log.

This arrangement builds two requirements into the structure. FR-5 becomes the cascade, so revoking a grant has to remove everything derived from it, with no cleanup routine that might be incomplete. FR-18 becomes the missing cascade on the audit log, so the record of what happened cannot be deleted by deleting its subject. Figure 4.11 shows the result.

4.9.2 Data Dictionary

Table 4.7 defines each table, its purpose and its principal attributes.

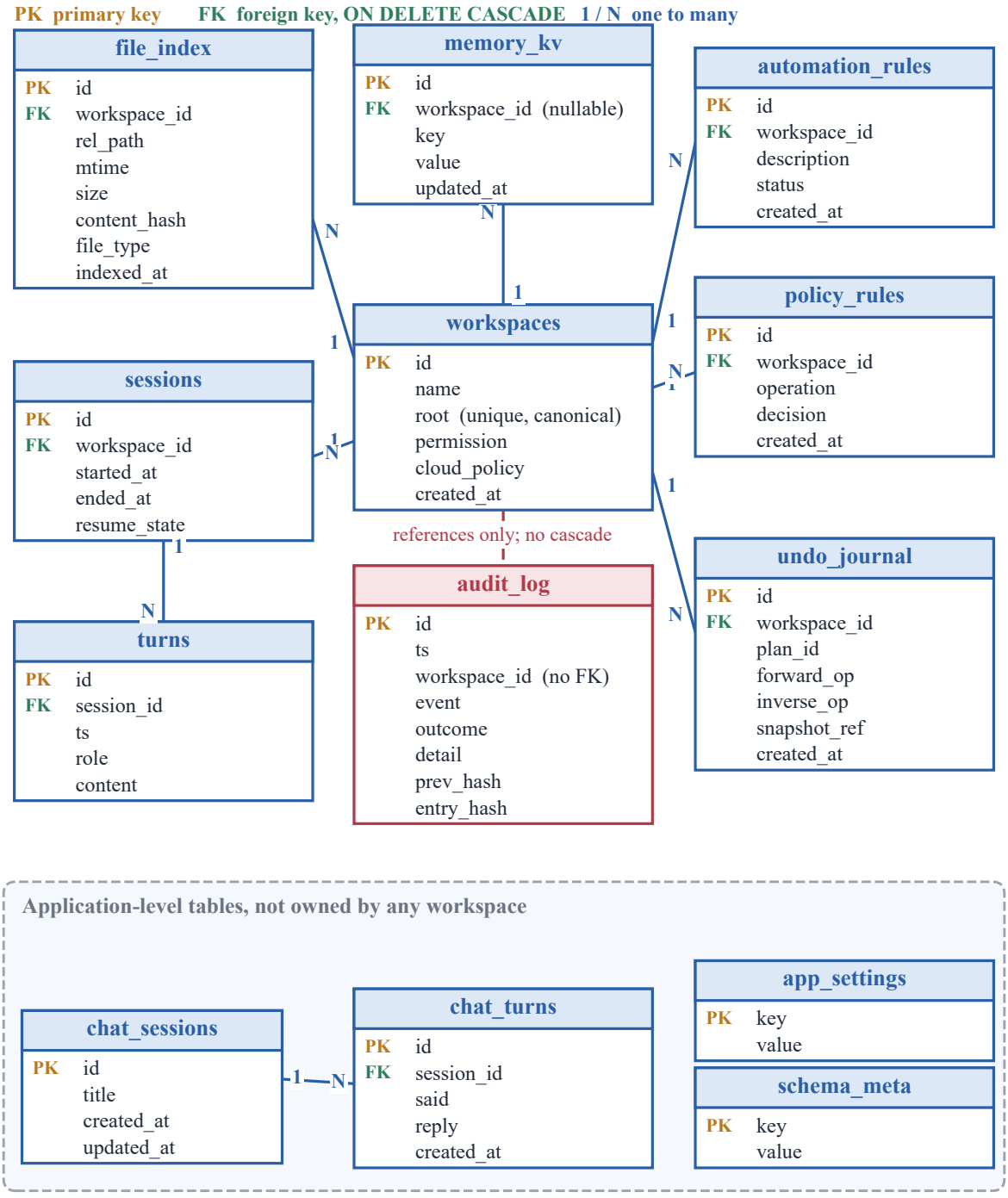


Figure 4.11: Entity relationship diagram.

Table 4.7: Data dictionary of the relational system of record.

Table	Purpose	Principal attributes
workspaces	One row per folder the user has granted	id; name; root, stored canonically and unique; permission, being read only or read and write; cloud policy; created at
file_index	What exists and where, mechanical and rebuildable	id; workspace id; relative path; modification time; size; content hash; file type; indexed at. Unique on workspace and relative path
audit_log	Append-only, hash-chained record of proposals, refusals and actions	id; timestamp; workspace id; event; outcome; detail; previous hash; entry hash
sessions	Session lifecycle and resume state	id; workspace id; started at; ended at; resume state
turns	Conversation history within a session	id; session id; timestamp; role; content
memory_kv	Interpretive long-term memory: preferences, facts, named entities	id; workspace id; key; value; updated at. Unique on workspace and key
automation_rules	Proposals and accepted rules from repeated task detection	id; workspace id; description; status, being proposed or accepted; created at
policy_rules	User-configured approval preferences, scoped to one operation in one workspace	id; workspace id; operation; decision; created at
undo_journal	Forward and inverse operations with snapshot references	id; workspace id; plan id; forward operation; inverse operation; snapshot reference; created at

Two points in this schema carry the trust claim, so we state them outright. First, referential integrity has to be switched on for every connection. The database engine leaves it off by default, so the cascade clauses would do nothing and revoking a workspace would look successful while leaving data behind. Second, the hash chain in the audit log links each entry to the one before. Changing or removing an entry breaks the chain in a way that can be detected, so the integrity of the record is something you can verify and not assume.

Beyond the relational store, each workspace holds its own vector index in a separate directory. This isolation is deliberate. It means a workspace's embeddings cannot be retrieved while another workspace is active even in the event of a query defect, since the index is not open, and it renders revocation of

a workspace a directory deletion. Content-addressed snapshots are held in a shared blob store, so that repeated snapshots of an unchanged file cost nothing.

4.10 Risk Analysis

Table 4.8 lists the main risks to the project, with the likelihood and impact of each and what we do about it.

Table 4.8: Project risks.

ID	Risk	Likeli-hood	Impact	Mitigation
R-1	A local model small enough for a student laptop proves inadequate for the planning tasks	Medium	High	Bounded failover to a cloud tier under per-workspace opt-in; capability probing in the router; degradation to a stated limitation rather than an error
R-2	Approval fatigue erodes the safety benefit the gates provide	High	Medium	Configurable granularity and scoped remembered approvals; outward gates remain fixed; the evaluation tests for fatigue directly rather than assuming its absence
R-3	A confinement defect permits access outside a grant	Low	Critical	Resolution confined to one component; canonicalisation before containment; adversarial tests for traversal, symbolic links, absolute paths and name-prefix siblings
R-4	Synthesised text contains unsupported claims	High	Medium	Citation map carried through to the answer; unsupported claims rendered as such; a faithfulness check is identified as an open gap rather than claimed as solved
R-5	Prompt injection from an adversarial document causes an unintended action	Medium	High	Provenance tagging; a structured plan as the interface; policy evaluated out of band; user preview of outward actions; blast radius bounded by the workspace

Table 4.8: continued from previous page

ID	Risk	Likeli-hood	Impact	Mitigation
R-6	Index growth degrades retrieval or rebuild time	Medium	Medium	Per-workspace indexes; incremental re-embedding by content hash; two-stage retrieval; configurable ceilings
R-7	The scope proves too large for three members across two semesters	Medium	High	Staged build order with the boundary and gates first; the remote plane is optional by construction and may be dropped without affecting the rest
R-8	The audience perceives no differentiation from existing assistants	Medium	Medium	Bounded workspace and local-first operation stated explicitly in the product; the disclosure panel makes the claim falsifiable
R-9	Evaluation participants are drawn from one institution, limiting generalisability	High	Low	Stated as a limitation; segment analysis reported; conclusions framed to the studied population
R-10	Undo fails to restore state after a partially completed plan	Low	High	Snapshots recorded before mutation; reversal in reverse order; irreversible steps reported rather than silently skipped

4.11 Conclusion

To sum up, this chapter set out what the system must do and under what conditions. It gave the features and the 35 functional requirements, each stated so it can be verified. It then covered the quality attributes and non-functional requirements, the assumptions, four use cases that exercise the central architecture, the hardware and software needed, the interface, the database design and the main risks. These give Chapter 7 the criteria for its test cases and Chapter 9 the standard for judging the prototype. Chapter 5 now describes how the requirements are met.

Chapter 5 Proposed Approach and Methodology

This chapter explains how we turn the specification in Chapter 4 into a working system and how we run the project. It covers the main architectural idea and why we chose it, the step-by-step procedures behind the behaviour that matters most, our development method and build order, and how we plan to evaluate the result. We stay on reasoning and procedure here. The implementation detail is in Chapter 6.

5.1 Overall Approach

Our whole approach rests on one idea: the limit on what the assistant can reach has to be built into the structure of the system and not left to the assistant's good behaviour. It should be impossible by construction, not merely discouraged by instruction.

The difference matters. Suppose we only tell a model not to touch anything outside a workspace. That is a preference, and a clever document, an odd phrasing or a mistake in the model can break it. Now suppose only one broker is able to turn a reference into a path on disk, and that broker works out the real location before it checks whether the path is allowed. That is a rule of the system. The first can be talked around. The second cannot, because no code path exists for the argument to travel along.

We apply this idea in three places, and together they define the system.

The first is confinement. Every path goes through one component, and that component finds the true location before it tests whether the path is inside the grant. This is what makes it a boundary and not a suggestion. A path with a traversal segment, or a symbolic link that points outside the grant, is resolved to where it really leads before the check happens. If the two steps were the other way round, a path that only looked contained would be let through.

The second is refusing destruction. We do not put deletion behind a confirmation box. We simply provide no operation that deletes. There is no approval dialogue for deleting because there is no code path to it. A gate the user can click through is a policy, while a gate that does not exist is part of the architecture. This is why we can say the system deletes nothing as a fact about the system, not as a promise about how a model will behave.

The third is keeping proposing and executing apart. The planner makes a plan but cannot run it. The dispatcher runs calls but cannot plan. In between, the policy engine gives a verdict on each proposed call, and the dispatcher refuses any call that has no verdict or whose verdict belongs to a different call. This is how we limit prompt injection. Text inside a document cannot become an instruction, because

an instruction has to be a well-formed declaration that passed the policy check outside the model, and no amount of convincing writing produces one.

5.2 The Risk Classification Procedure

The policy engine puts every proposed call into exactly one of four tiers. This classification is the main decision the system makes, and its properties are chosen on purpose.

The order of checks is fixed, and refusals are checked before permissions. Each rule returns as soon as it matches, so the first match decides and a later rule can never allow something an earlier one refused. The order is:

1. If any path the call touches lies outside the active grant, deny it as destructive
2. If the operation is itself destructive, deny it
3. If the operation leaves the machine, gate it, and do so regardless of configuration
4. If the operation changes state, gate it unless the user has recorded a rule permitting that operation in that workspace
5. If the operation is a recognised read, allow it
6. Otherwise deny it, because an unrecognised operation is treated as unknown rather than as harmless

The last rule needs a word of explanation. If the system does not recognise an operation, it refuses it instead of allowing it. Quietly allowing unknown operations is how a boundary wears away over time, because a system that defaults to yes slowly collects abilities it was never designed to have.

The containment test in the first rule is handed to the broker and not written again here. We did this deliberately so that only one containment check exists. Two copies could drift apart as the code changes.

Table 5.1 lists the four tiers, how each is treated by default and whether it can be recovered.

Table 5.1: The four risk tiers.

Tier	Definition	Default	Recovery
T0 Read	Reads within the granted workspace. No change, no egress.	Executes automatically	Not applicable

Table 5.1: continued from previous page

Tier	Definition	Default	Recovery
T1 Reversible	Changes workspace state and is fully invertible from a snapshot.	Configurable: automatic, batch approval, or per action	Full
T2 Outward	Leaves the machine: mail, web request, cloud model call carrying workspace content.	Always gated. Not configurable.	Partial; logged, and recallable only where the protocol permits
T3 Destructive	Deletion, overwriting without a snapshot, writing outside the workspace.	Refused. Not gated, because structurally unavailable.	Not applicable

Only T1 can be configured, and this is how we respect the finding from Chapter 2. Respondents were split almost evenly on whether an assistant should ask before acting, so approval granularity is a setting and not a fixed decision. A remembered approval covers one operation in one workspace. Approving a folder tidy does not allow the assistant to send mail, and the engine refuses to record such a rule for an outward or destructive operation, whatever the user asks.

5.3 The Recovery Procedure

Recovery follows two rules. Each one answers a common way in which undo features fail.

First, the snapshot is written before the change, never after. If the inverse is recorded afterwards, it captures the state the operation already produced, and undoing that restores nothing. So every path through the journal writes its entry first and only then lets the caller act.

Second, the method depends on cost. A small file is copied into a content-addressed store, so repeated snapshots of an unchanged file cost nothing after the first. A large file is hard-linked where the filesystem allows it. A move or rename needs no copy at all, because its inverse is just the reverse move. Creating a new file only needs a record that it can be removed.

Undo works on the whole plan, not on single operations. Undoing a folder tidy puts back all of its moves together, in reverse order, because the plan is what the user was asked to approve. The reverse order is needed and not just tidy. If a plan creates a directory and then moves files into it, the files have to be moved back before the directory can be removed. A step that cannot be reversed is reported and not

raised as an error, so one awkward file does not stop the rest from being restored.

5.4 The Prompt Injection Defence

The system reads documents, source code and web results, and an attacker may have influenced any of them. The security literature in Chapter 3 names this as the main threat to an agentic system. We do not try to detect attacks, because detection is unreliable. We make it structurally impossible for retrieved content to become an authorised instruction. Five defences apply, and each would be enough on its own.

- Every context chunk is tagged with its provenance as user, workspace file, web or tool result, and only user chunks may originate an intent. All others are treated as data
- The planner emits a structured plan rather than prose, so a tool call must be a well-formed declaration and text within a document cannot become one by being persuasive
- The policy engine reads the tool call and never the model's reasoning text, so nothing written in a document can alter a verdict
- Every outward action presents the user with a concrete preview of what will occur, including recipient and content, so an attempted exfiltration must survive a person reading it in plain language
- The workspace boundary limits the consequences of a wholly successful injection to one granted workspace, with no deletion and no egress beyond that workspace's policy

The last defence is the strongest argument for our hypothesis, because it limits the worst case and not only the typical one.

5.5 Development Methods

We work in two-week sprints. The backlog is written as user stories with Fibonacci point estimates. Each sprint delivers one stage of the build order and ends with a stage gate, a test that must pass before the stage counts as done. A stage is finished when its gate passes, not when its code exists.

We chose this method because of the project's main risk and not out of habit. The risk is that the trust machinery cannot be added later once features depend on it. So we build that machinery first and check it against inert tools before any model is involved.

We considered two other methods and rejected both. Waterfall did not fit, because our specification depends on findings that only appear during construction, such as whether a local model of a workable size can plan well enough. Building the visible features first and adding safeguards afterwards was rejected for a stronger reason. The boundary and the gates are exactly the parts that cannot be added

late, because every feature built before them would need rewriting to pass through them.

Table 5.2 shows the build order. One principle sits behind it: a property has to exist before anything can be required to hold it.

Table 5.2: The build sequence.

Stage	Deliverable	Reason for this position
1	Workspace foundation: the store, the workspace broker, the file watcher and the disclosure panel. No model involved.	The boundary must exist before anything can be bounded.
2	Policy engine, approval broker and undo journal, exercised against placeholder tools.	Inexpensive now and near-impossible to retrofit; provable before any model introduces non-determinism.
3	The gateway: model router, context engine, agent planner and tool dispatcher.	The highest-risk stage, validated against gates that already function.
4	Session resume, folder tidy and repeated task detection.	The first real capabilities, exercising the read and reversible tiers end to end.
5	Notes synthesis and code assistance, with the index, retrieval and citation map.	Introduces the vector index only once the boundary is proven.
6	Web search, mail composition and document drafting.	The first outward actions, permitted only after the gates are demonstrated.
7	Speech input and output over the existing turn normaliser.	Should require no new gate. If it does, the interaction layer was designed wrongly, so this stage tests the parity claim.
8	Remote access plane.	Genuinely optional; a complete product exists without it.

Putting stage 2 before the model work is the most important decision in this order. It means the trust machinery can be tested and demonstrated before a model introduces any unpredictability. A failing gate at that point is a plain defect with a short way to reproduce it, and that stops being true once a model is generating the plans.

The stage 7 gate is worth noting because it is written so that it can fail. We claim voice works the same

as typing by construction, and we test that by requiring that the speech stage adds no new approval gate. If one were needed, the claim would be false and the interaction layer would need redesigning. A gate that cannot fail tests nothing.

5.6 Evaluation Methodology

The evaluation will take place in the second phase. It tests the central hypothesis and the limitations that the literature review said we inherit. It has four parts.

First, task-based measurement. Participants do typical multi-file tasks with and without the system, and we record completion time and success rate. Second, mental workload, measured with an established subjective questionnaire after each task. Third, usability and trust, measured by questionnaire, with the trust questions aimed at the bounded-scope claim so that the hypothesis is tested and not assumed.

Three more tests deal directly with the limitations we inherit, because the literature suggests each one would otherwise be claimed and not shown. Approval fatigue is tested over repeated sessions, since a single session cannot show it. Faithfulness of the synthesis and accuracy of citations are measured wherever source metadata allows, and any shortfall is reported as a known limitation. Code assistance is checked to confirm that no change is applied without explicit approval and that no earlier user edit is silently overwritten.

By the end of the first phase we had carried out the parts of this plan that do not need participants. They are reported in Chapter 9. The parts that do need participants are planned for the second phase and listed in Chapter 10.

The voice evaluation follows the guidance from the literature. We set a target response time of four seconds, use natural spoken fillers and not tones while the user waits, and report workload and trust next to task time and success rate, because the reviews showed that standard usability metrics leave out exactly these voice concerns.

5.7 Conclusion

This chapter has described the approach and the methodology. It stated the strategy of placing the guarantees in structure, gave the procedures for risk classification, recovery and the defence against prompt injection, described the incremental method and the build sequence in which each stage must pass its gate before the next begins, and outlined the evaluation. The procedures set out here are the ones that the implementation follows and the evaluation tests. The next chapter presents the design that realises them.

Chapter 6 High-Level and Low-Level Design

This chapter shows how the system is designed. We start with an overview and the things that limit the design. Then we go through the architecture layer by layer and component by component, the strategies that shaped it, the domain model and the policies we follow when building it. Diagrams are used throughout, and each important decision is tied back to the requirement or finding behind it.

6.1 System Overview

Think of the system as a local process that keeps running and sits between the user and one bounded part of their filesystem. A request comes in by typing or speech. The system pulls context from inside the granted workspace, makes a multi-step plan, rates each step by risk, asks for approval where it is needed, records how each change could be reversed, runs the approved steps and finally writes a permanent record of everything it proposed and did.

The design is organised as six layers on the execution path with two planes crossing all of them, as shown in Figure 6.1. The presentation layer provides the command line, graphical and optional remote surfaces over one shared view model. The interaction layer normalises voice and text into a single modality-neutral object. The gateway layer routes model requests, assembles context, classifies risk, plans, and dispatches. The capability layer implements the features. The data layer persists everything locally. The remote access layer is drawn outside the stack because nothing in the layers beneath it depends on its presence.

Layers one to five make up the local execution path. Two planes cut across every layer. The remote access plane is drawn outside the stack, since the system still works fully without it.

Security and performance are drawn as planes and not as layers, and we did that on purpose. A layer is a stage a request moves through. Drawing security as a layer would suggest a request can get past it, and that is the very mistake behind confused-deputy defects. As planes, they show that every arrow in the architecture passes through both.

6.2 Design Considerations

6.2.1 Assumptions and Dependencies

The design depends on the following, each of which would require reconsideration were it to change.

- The operating system’s canonicalisation routine resolves symbolic links correctly, since the con-

finement guarantee is built upon it. The design does not depend on any platform-specific behaviour beyond this

- The relational store enforces referential integrity when instructed to do so, since revocation is expressed as a cascading deletion. Because the engine disables this by default, it must be enabled explicitly on every connection
- A local model runtime is available, or the user has enabled cloud fallback for the workspaces concerned. The router treats providers as configuration, so a change of provider is not a change of design
- The user is a single individual with authority over the folders they grant. The design does not address shared or concurrent access by multiple users
- Documents within a workspace may be adversarial, since a file may have originated anywhere. The design therefore treats all retrieved content as data rather than as instruction
- Functionality is expected to change most in the capability layer, as features are added. The uniform tool contract exists so that such change does not propagate into the gateway

6.2.2 General Constraints

The following constraints shaped the design, and the impact of each is stated.

- **Resource envelope.** The system must remain usable on a consumer laptop, which requires that expensive work be asynchronous and that every queue be bounded. The impact is the execution plane described in Section 6.4
- **Offline operation.** Core functionality must not require connectivity, which requires that a local model be the default tier and that networked capabilities degrade explicitly. The impact is the degradation contract in Table 6.3
- **Data residency.** User data must not reach a project-operated server, which places egress enforcement in the router, where workspace policy is known, rather than in the tool layer
- **No inbound port.** The remote plane must dial outward, which determines the topology in Section 6.4 and removes any requirement for firewall configuration or network address translation traversal
- **Credential handling.** Secrets must reside in the operating system keyring, which excludes them from the system's own storage tree and from any backup of it
- **Verification.** Each guarantee must be testable, which requires that each have exactly one enforcement point. This is the reason resolution occurs in one component and execution in another

- **Team and timeline.** Three members across two semesters requires that the optional components be separable, which is why the remote plane is architecturally detachable

6.2.3 Goals and Guidelines

Four principles dominate the design, and the reason for each is stated because none is self-evident.

Rule one: a property has to be structural, not promised. Where a guarantee can be written as the absence of a code path and not as a check, we write it that way. That is why destructive operations have no approval dialogue. A check can be beaten by a bug or an unexpected path, but a capability that is absent cannot be used.

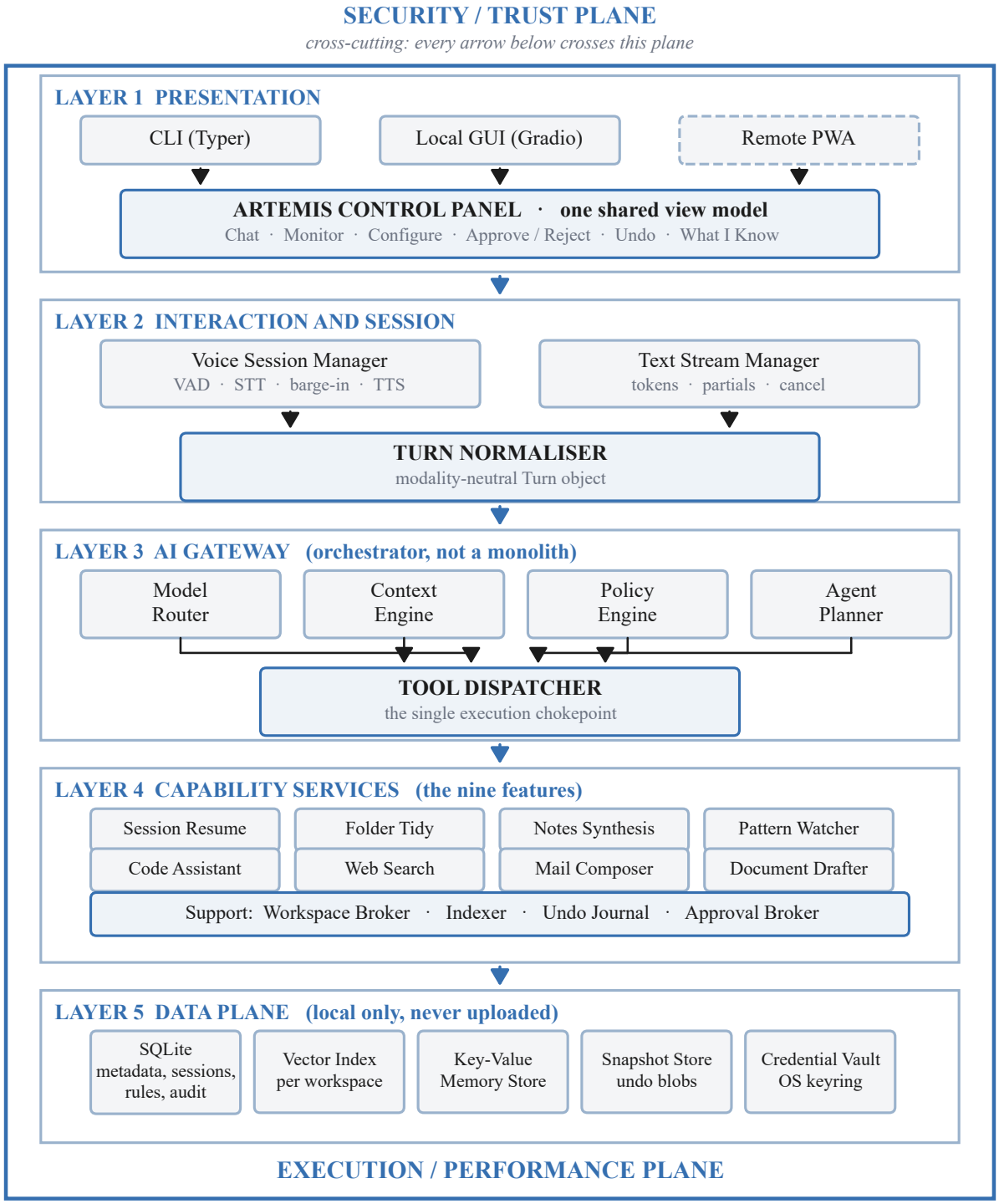
Rule two: each guarantee has one enforcement point only. Paths are resolved in the broker and nowhere else, and actions run in the dispatcher and nowhere else. A guarantee with two enforcement points can drift apart and has no single place to audit, and we want to be able to verify each one.

Rule three: being able to see what the system does is part of trust. Background work shows progress that can be cancelled. The disclosure panel runs a live query and is not a written summary. The audit log records refusals as well as actions. A system whose behaviour cannot be observed cannot be trusted, however correct it may be.

Rule four: when something goes wrong, the system refuses. It denies an operation it does not recognise, refuses a path it cannot resolve, openly drops work when a queue is full, and never executes an approval that has expired. If a boundary-holding system fails permissively, a defect becomes a breach.

6.2.4 Development Methods

We did not copy a reference architecture. We derived the structure from the research findings and the specification. The layered form with cross-cutting planes is a common one. The split of the gateway into components came from a specific problem. Our first design gave one component responsibility for routing, context, memory, planning, policy and dispatch all at once, and that could not be tested because no single guarantee could be isolated. We split it into four components that share a single execution point, so each guarantee can be checked on its own. The same revision made the policy engine a component in its own right, which turned the approval-first finding from interface behaviour into enforced structure.



LAYER 6 · REMOTE ACCESS PLANE is drawn separately, because nothing in Layers 1 to 5 depends on it.

Figure 6.1: The layered architecture.

6.3 System Architecture

The responsibilities are divided so that exactly one component holds each guarantee. Table 6.1 is the inventory: for every component it gives the layer, the responsibility and the one thing it must never do. The last column is the one that matters. It states the rule whose violation would be a defect in the architecture and not just in the code.

Table 6.1: Component inventory.

#	Component	Layer	Responsibility	Must never
1	Command line interface	1	Scriptable control surface	Bypass the approval broker
2	Graphical control panel	1	Visual control surface, loopback-bound	Bind to a non-loopback interface
3	Control panel view model	1	Single source of interface truth across surfaces	Hold state the backend does not own
4	Voice session manager	2	Activity detection, transcription, interruption, synthesis	Record when not explicitly armed
5	Text stream manager	2	Token streaming and cancellation	Buffer past a cancellation
6	Turn normaliser	2	Produce the modality-neutral turn	Leak modality downstream
7	Model router	3	Select a tier and fail over; enforce egress	Send workspace data to a disabled tier
8	Context engine	3	Assemble bounded, cited context	Include content outside the grant
9	Policy engine	3	Classify risk and decide gating	Be consulted after execution
10	Agent planner	3	Produce a multi-step plan	Execute anything itself
11	Tool dispatcher	3	Sole execution point	Accept a call lacking a policy verdict
12	Capability services	4	The features	Touch the filesystem directly
13	Workspace broker	4	Path resolution and confinement	Resolve a path outside a grant
14	Indexer	4	Incremental indexing and embedding	Block the interactive path
15	Undo journal	4	Snapshots and inverse operations	Record an action after the fact

Table 6.1: continued from previous page

#	Component	Layer	Responsibility	Must never
16	Approval broker	4	Hold pending actions and fan out to surfaces	Approve on timeout
17	Data plane stores	5	Local persistence	Serialise outside the host
18	Remote access plane	6	Authenticated relay	Store or read plaintext user data

How the components work together is shown by the request lifecycle in Figure 6.2, the most important path in the system. Spoken or typed, local or remote, every request goes through exactly this sequence. There are no shortcuts.

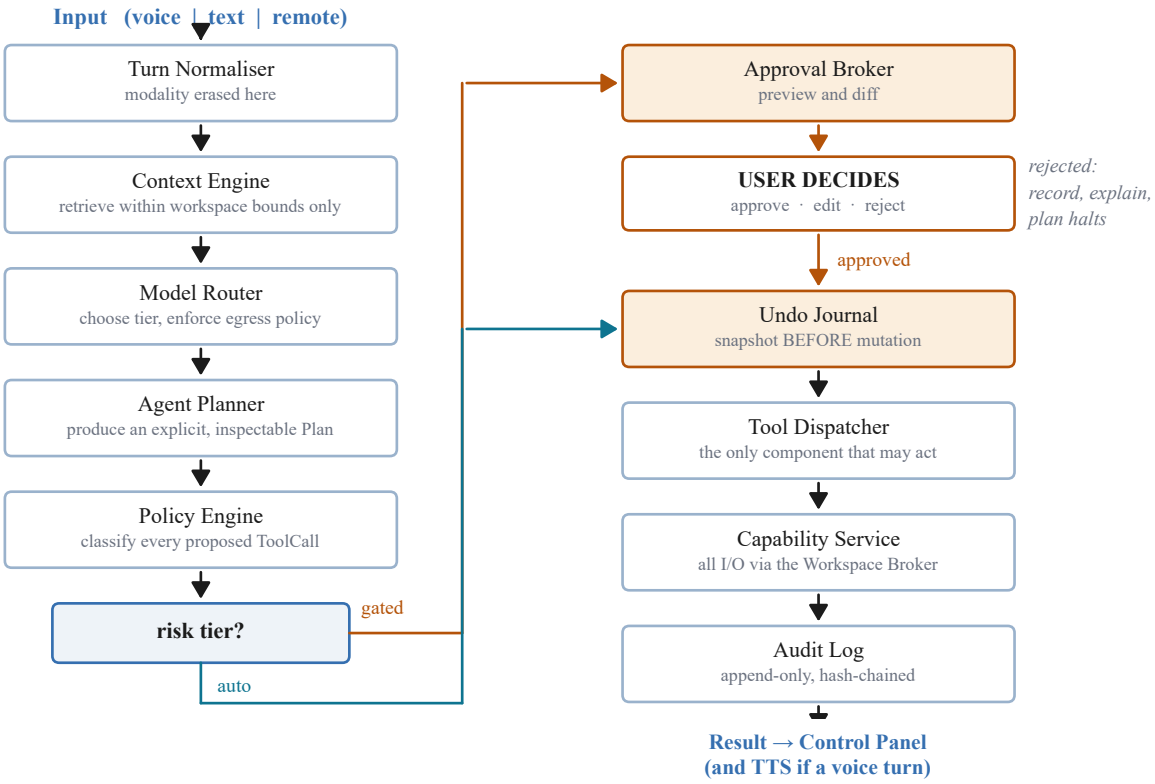


Figure 6.2: The request lifecycle.

Modality is erased at the turn normaliser, the policy engine is consulted before the dispatcher, the snapshot precedes the mutation, and the audit entry is written whether the action proceeds or is refused.

Five invariants are enforced along this path, and together they constitute the architecture's guarantees. The policy engine is consulted before the dispatcher and never after. The undo journal records its snapshot before the mutation and never after. The audit entry is written even when the action is rejected, so the log records what was proposed rather than only what occurred. A call lacking a matching policy

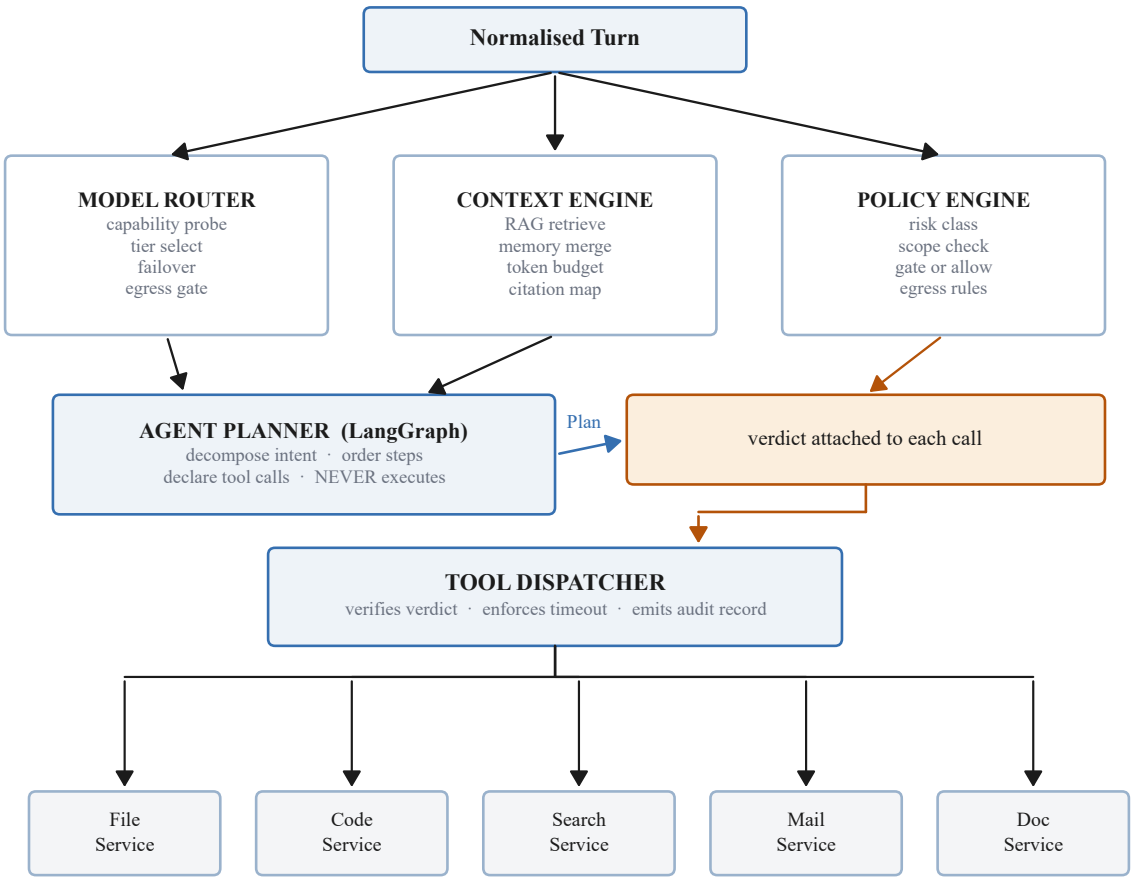


Figure 6.3: Internal structure of the gateway.

verdict is refused by the dispatcher. Voice and text turns are indistinguishable from the planner onward, which is what renders voice parity structural rather than a second implementation.

We looked at two other ways to divide the work and turned both down. One was a single orchestrating component, which we rejected because no individual guarantee could be isolated for testing, and a bug anywhere inside it would break every guarantee at once. The other was a per-feature arrangement, where each capability keeps its own gating logic. That would give as many enforcement points as there are features, none of them auditable from one place, and each able to drift.

6.3.1 Subsystem Architecture: The Gateway

The gateway is decomposed into four components with a single shared execution point, as shown in Figure 6.3. The planner produces a plan and cannot execute it; the dispatcher executes and cannot plan; the policy engine renders a verdict on each proposed call; and the router determines which model tier may be used and whether data may leave the machine.

The planner produces a plan but never executes it, and every call converges on the dispatcher, which is the sole execution point.

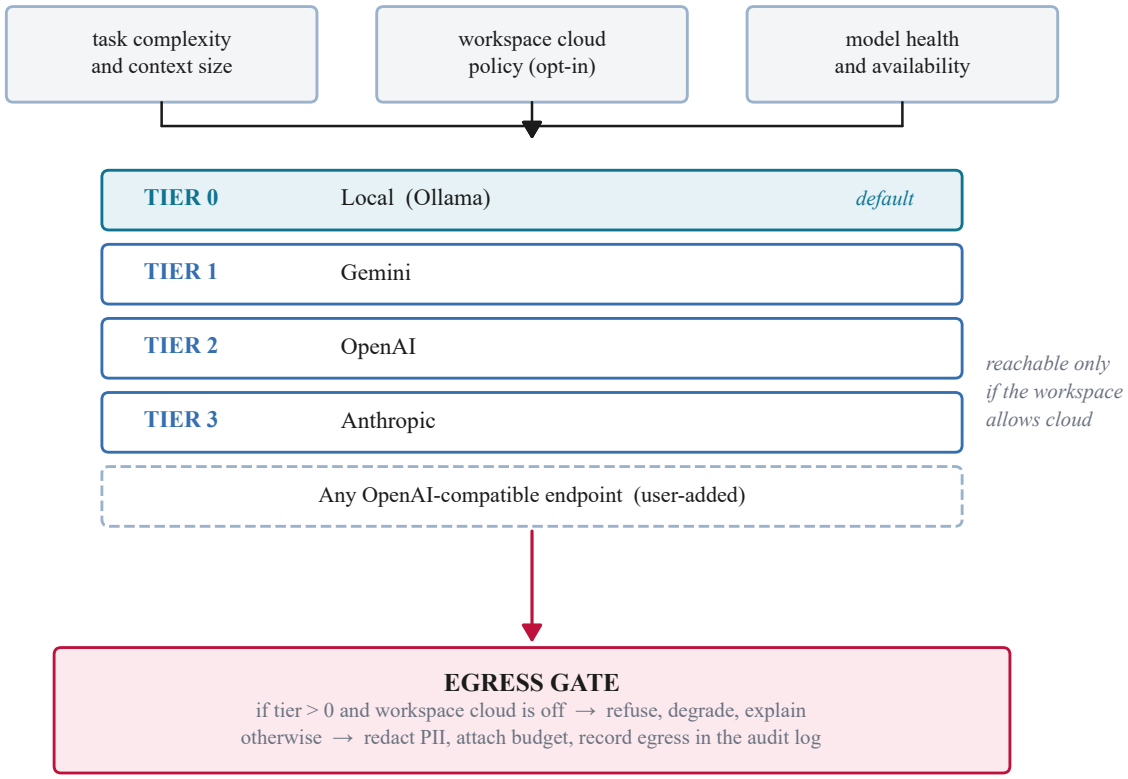


Figure 6.4: Tier selection and the egress gate.

The model router owns tier selection and egress enforcement. Egress is enforced here, rather than in the tool layer or in the prompt, because the router is the component that knows a given workspace's cloud policy. Tier zero is the local model and is the default. Higher tiers are reachable only where the workspace permits, and any compatible endpoint may be registered as an additional tier without modification to the code, since providers are treated as configuration rather than as branches. Failover is bounded at one retry per tier before advancing, so a workspace with cloud disabled possesses exactly one tier and its failure produces a clean degradation message rather than an indefinite wait. Figure 6.4 shows the selection and the egress gate.

Tiers above zero are reachable only where the workspace's cloud policy permits, and every egress is recorded.

The context engine assembles the model's working context and enforces bounded scope on the read side, as the broker does on the write side. Every retrieved chunk carries the identifier of the workspace it came from, and any chunk not matching the active grant is discarded and the discard recorded. A token budgeter weights by recency and relevance but never truncates a citation source silently. A citation map carries each chunk back to its file and line, and this map is what makes the synthesis feature verifiable, since a claim that cannot be traced through it is rendered as unsupported rather than presented as established. Figure 6.6 shows this assembly.

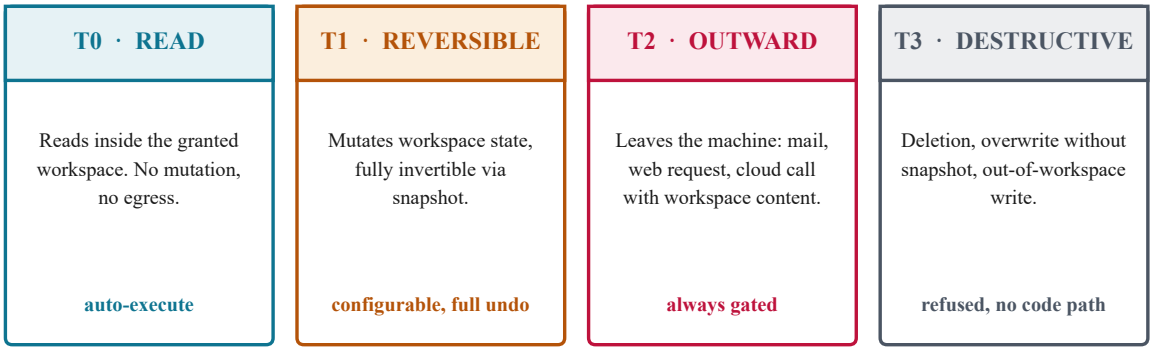


Figure 6.5: The four risk tiers.

Every retrieved chunk carries a workspace identifier, any chunk outside the active grant is discarded and the discard logged, and the citation map is carried through to the answer.

The policy engine rates each proposed call using the procedure from Chapter 5. Figure 6.5 shows the four tiers, and Figure 6.7 shows the order of evaluation, with refusals before gates and gates before permissions.

Only the reversible tier is user-configurable; outward actions are always gated and destructive actions are structurally unavailable.

Denials are evaluated before gates and gates before permissions, so that the first match is the decision and no later rule can grant what an earlier rule refused.

Figure 6.8 walks through a folder tidy. The read steps run automatically, the two steps that change state are gated, and the resulting preview says what will happen and, just as important, what will not.

Read operations execute automatically while the reversible operations are gated, and the preview states that nothing leaves the machine and nothing is deleted.

The agent planner builds plans with LangGraph and never runs them. Its reflection loop has a hard cap on cycles and a time budget. This answers the looping failure that the planning literature describes: an agent that cannot settle on a plan shows what it has so far and asks the user, instead of going round indefinitely.

6.3.2 Subsystem Architecture: Capability Services

Each feature is a service behind the same tool contract. None of them touches the filesystem, the network or the database directly. They all go through a shared base made up of the broker, the journal, the approval broker and the data plane, as Figure 6.9 shows.

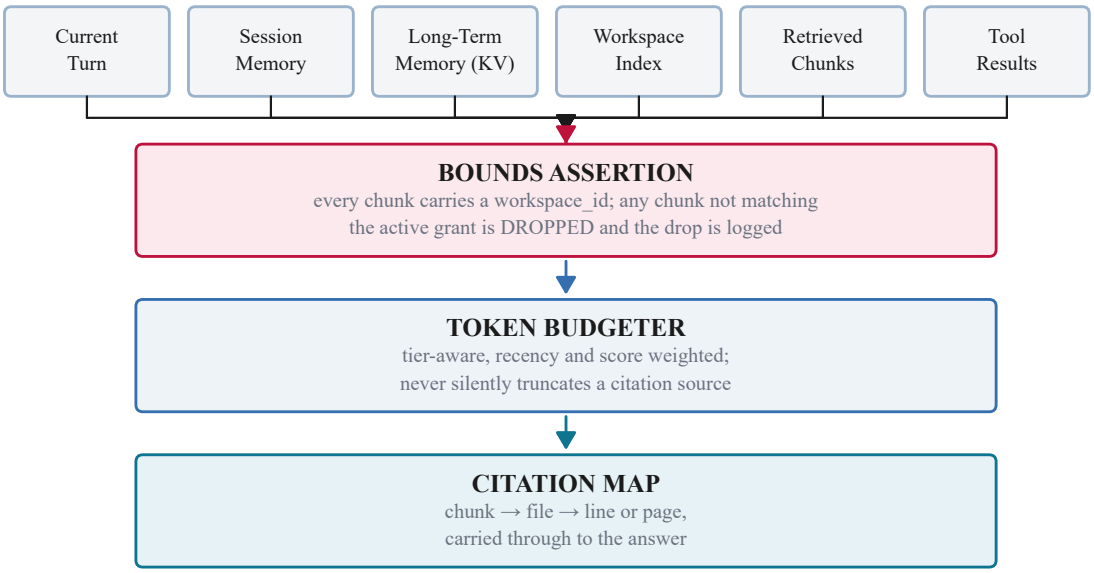


Figure 6.6: Context assembly.

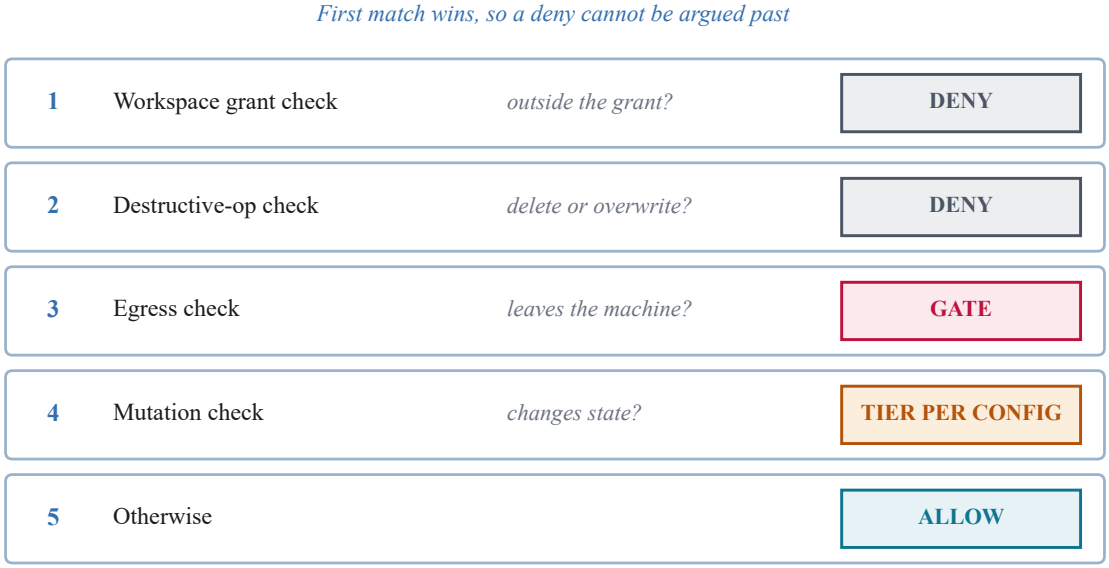


Figure 6.7: Policy evaluation order.

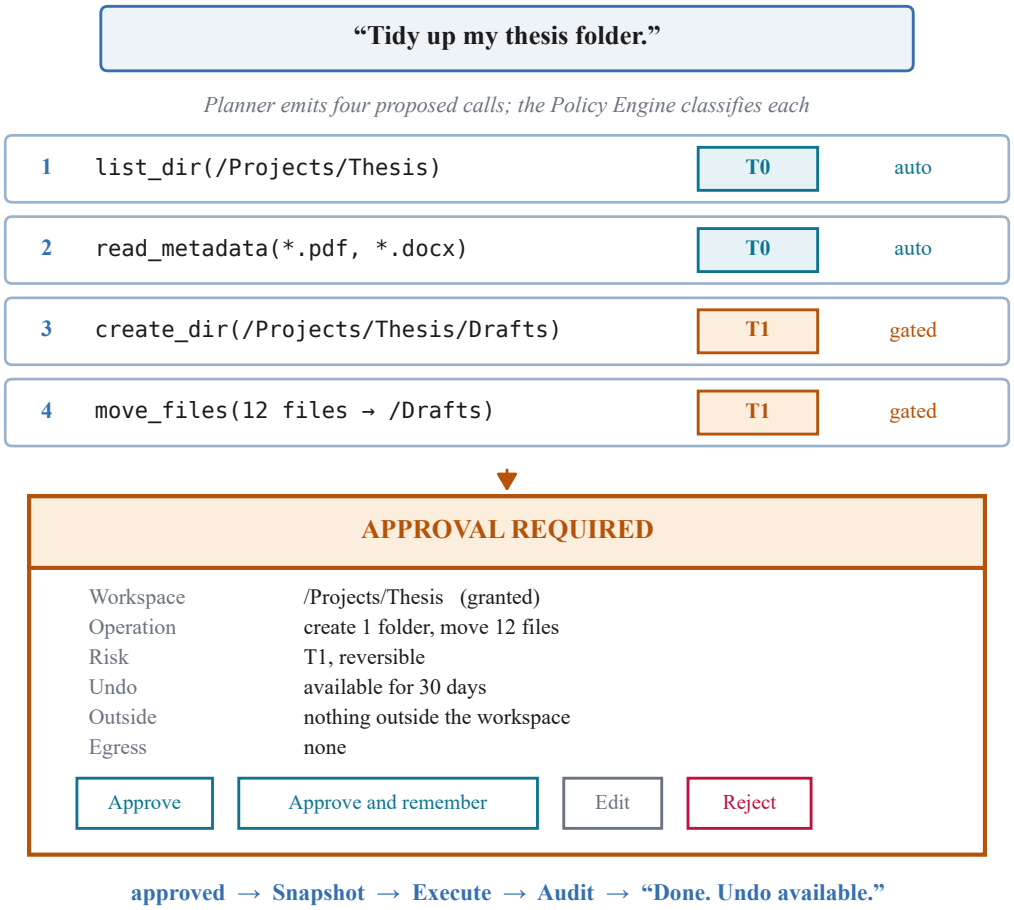


Figure 6.8: A worked classification for a folder tidy.

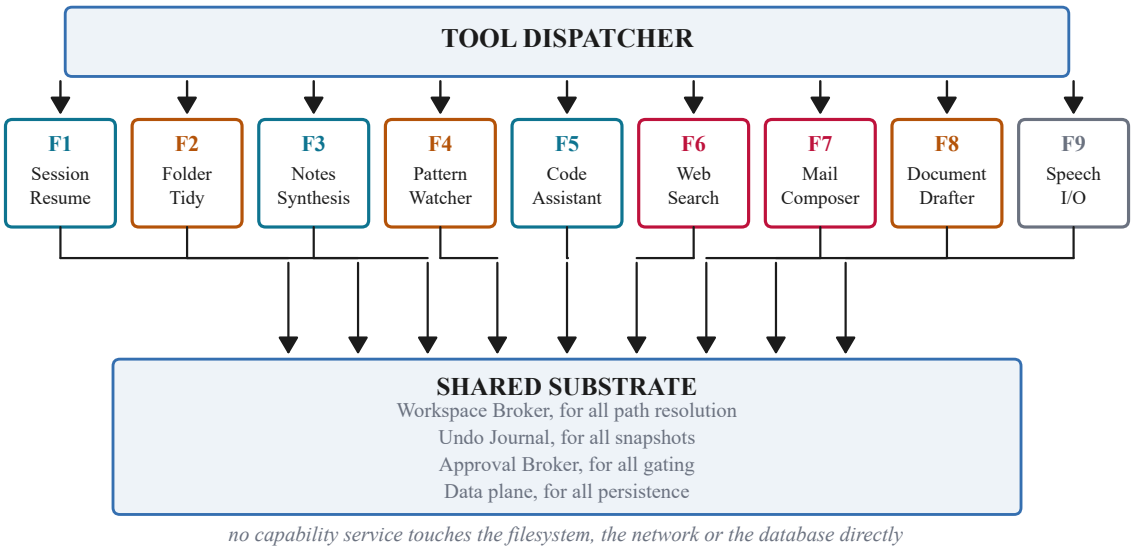


Figure 6.9: Capability services and their access path.

Capability services are reachable only through the tool dispatcher and reach the filesystem only through the shared substrate, so no service holds its own path to the filesystem.

Table 6.2 lists the highest tier each service may ask for. Treat it as a design constraint and not as a description. A service cannot request a tier above its maximum, which keeps the damage from a bug inside any one feature contained.

Table 6.2: Capability services and risk tiers.

#	Service	Max tier	Notes
F1	Session resume	T0	Deterministic replay of recorded session state; no model involved
F2	Folder tidy	T1	Moves and groups only; no code path to deletion
F3	Notes synthesis	T0	Reads and generates; output remains a proposal until the drafter writes it
F4	Pattern watcher	T1	Detects three or more repetitions and may only propose a rule; cannot install one
F5	Code assistant	T0	Explains and produces a difference; never writes and never executes
F6	Web search	T2	On demand only; every query recorded
F7	Mail composer	T2	Draft, approve, send; sending is gated regardless of configuration
F8	Document drafter	T1	Always writes a new file; never overwrites an existing one
F9	Speech input and output	Not applicable	Not a service but a modality at layer two, over identical gates

The ninth feature, voice, is not a service, and that is deliberate. Voice is turned into the same turn object as text at the interaction layer, so it picks up every approval gate automatically. We get parity by design and not by rebuilding the gates along a second path, and the stage seven gate from Chapter 5 exists to test exactly this.

The workspace broker turns a logical reference into a real path. Figure 6.10 shows its five assertions. The order of the second and third is the whole point: finding the true location first resolves a symbolic link or a traversal segment before the containment test runs.

Canonicalisation occurs before the containment test, which is what distinguishes a boundary from a

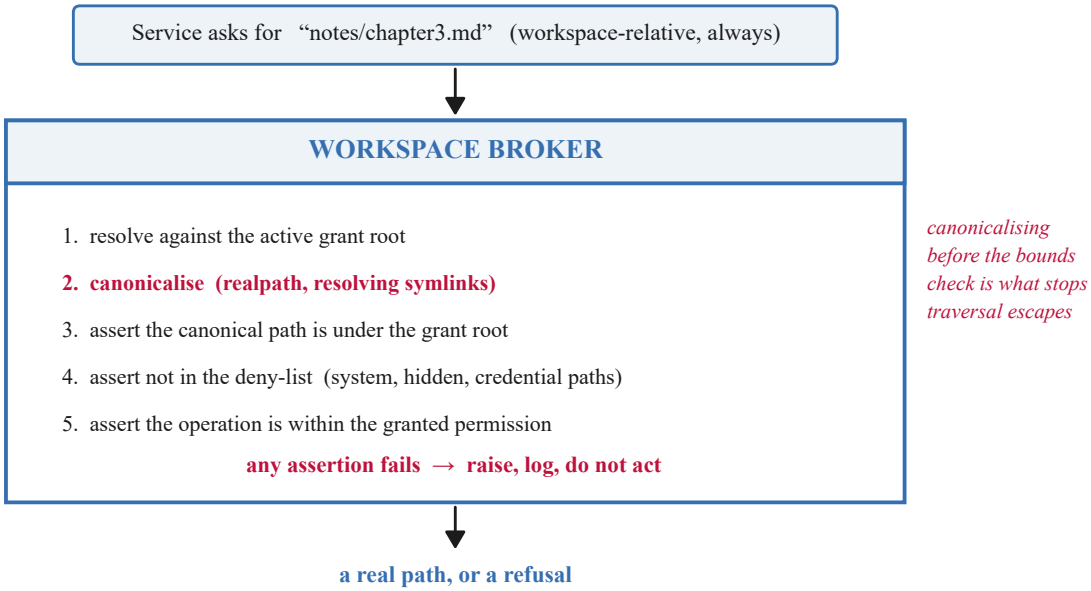


Figure 6.10: Path resolution and confinement.

suggestion.

6.4 Architectural Strategies

This section states the strategies that shaped the architecture, the reasoning behind each, and the alternatives considered and rejected.

6.4.1 Local-First Operation with Optional Egress

By default the system uses a local model, and cloud access is opt-in per workspace. There are three reasons. The research named local-first as the most common deployment preference. Privacy came up both as a reason people might stop using the system and as a theme in free-text answers. And offline operation is one of our constraints.

We turned down cloud-first with a local fallback, because it flips the guarantee around. Under that setup, user data is sent by default and the user has to act to stop it. Under ours, the user has to act to allow it. The price is capability, since a small local model does worse than a big hosted one. We reduce that with bounded failover where the workspace permits it, and with honest degradation where it does not.

Table 6.3 is the degradation contract. It says what must happen as each outside capability is taken away.

Every row is an acceptance criterion we can test, and together they define what correct means when conditions are bad.

Table 6.3: The degradation contract.

Withdrawn capability	Required behaviour
Internet connectivity	Full local operation continues. Cloud model calls, web search and mail sending are disabled, each failing as an explicit retryable action rather than being silently dropped
All cloud model providers	Falls back to the local model. Features exceeding local capability degrade to a stated limitation rather than an error
Local model	Falls forward to the cloud chain, if and only if the workspace permits cloud access
Remote device	The host continues uninterrupted. Approvals in flight revert to pending and appear on the host surface
Remote edge	Local operation is entirely unaffected, because the remote plane is not on the execution path
Workspace grant	The system cannot see the folder. Not declines to touch it: the path is not resolvable at all

6.4.2 Separation of Memory from the File Index

The system keeps two different kinds of knowledge, and mixing them up is what turns a bounded assistant into a surveillance tool. Figure 6.11 shows the five kinds of memory, and Figure 6.12 shows the distinction that matters most.

The kinds are distinguished by lifetime and by store, and all feed a single context engine that merges, budgets and asserts bounds.

The two answer different questions and carry different consequences. The index is mechanical and rebuildable; memory is interpretive and, once deleted, lost.

The file index answers one question: what exists, and where? It is mechanical, comes straight from the filesystem, can be rebuilt at any time and holds no interpretation. Memory answers a different one: what has the system learned about this workspace, this task and this user's preferences? That is interpretive, builds up over time, is lost if deleted and needs consent. We keep them in separate stores with separate lifecycles, and every memory entry can be seen and deleted in the disclosure panel. This also covers the most requested missing feature, that the system learns how the user organises things. That learning lives in per-workspace memory, with the same visibility and deletion as everything else, so the boundary does not widen.

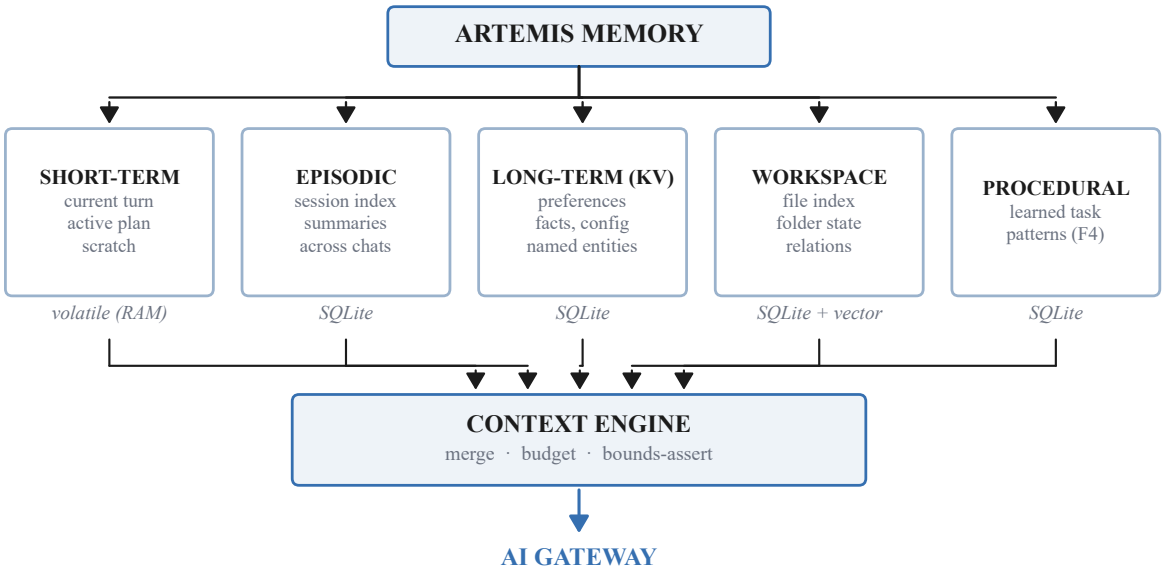


Figure 6.11: Five kinds of memory.

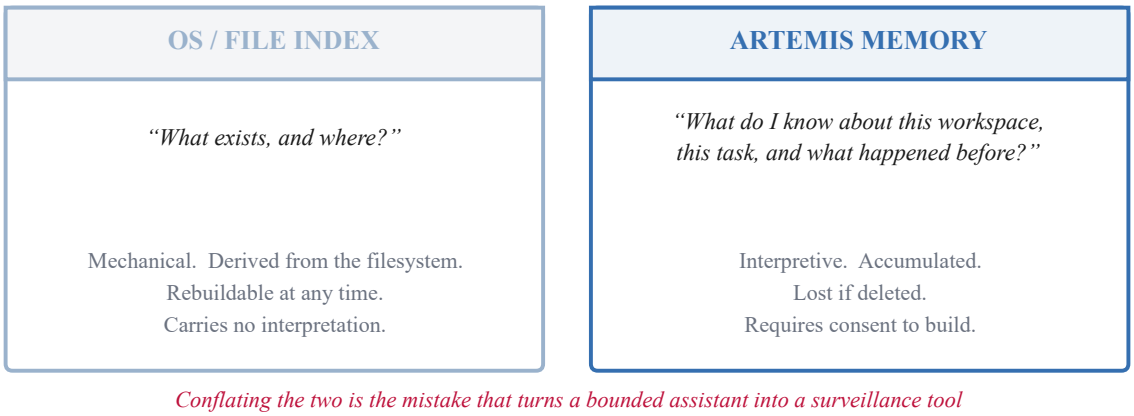


Figure 6.12: The file index compared with system memory.

6.4.3 Disclosure as a Live Query

The disclosure panel is a live query across every store, not a summary the system writes about itself. A written summary can be out of date or wrong, but a query cannot disagree with the data it reads. Every row can be deleted, and deletion is a real cascade. Revoking a workspace removes its index directory, its file rows, its embeddings and its memory entries.

We rejected a report generated now and then. It would make the trust claim persuasive and not falsifiable, and a claim that can be checked is the stronger one. The audit log is the one thing that cannot be deleted. A log that its subject can erase is not an audit log.

6.4.4 The Remote Plane as a Detachable Component

The remote access plane sits outside the core stack, and the test that this separation is real is simple: take the whole plane away and you still have a working single-machine system. Figure 6.13 shows its structure, and Figure 6.14 shows the pairing handshake.

The durable object records which authenticated device is connected to which host and relays encrypted frames it cannot read.

Host-side confirmation is deliberate, so that possession of the pairing code alone is never sufficient to pair a device.

The plane is a control surface rather than a privilege escalation. Remote sessions run at equal or lower privilege than local ones and never higher. A workspace may be marked local-only, rendering it invisible to remote sessions. Outward actions may be configured to require local approval regardless of where they were requested, and this is the default. Sessions expire on a fixed interval and on host sleep, revocation is instant and unilateral from the host, and every remote-originated action is tagged with its device in the audit log. The relay holds no plaintext, so the privacy claim survives compromise of the edge.

6.4.5 Asynchrony with Bounded Queues

The interactive path stays thin, and anything expensive runs in the background (Figure 6.15). Indexing is incremental and only re-embeds chunks whose content hash changed, so a one-line edit re-embeds one chunk and not a whole file. Retrieval has two stages: a cheap vector recall and then a reranking pass, which keeps the index small without losing quality. The local model stays loaded with an idle timer, so the first token does not wait for a cold start.

The interactive path never blocks on background work, and background progress is reported visibly and

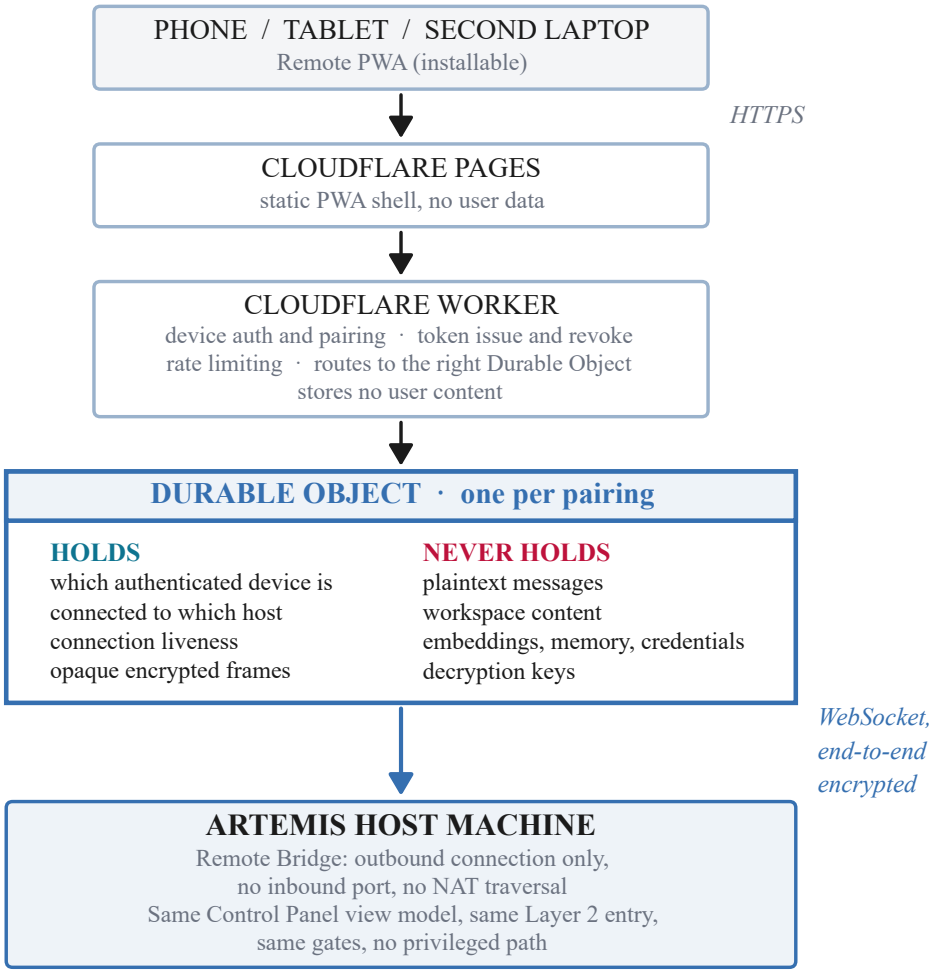


Figure 6.13: The remote access plane.

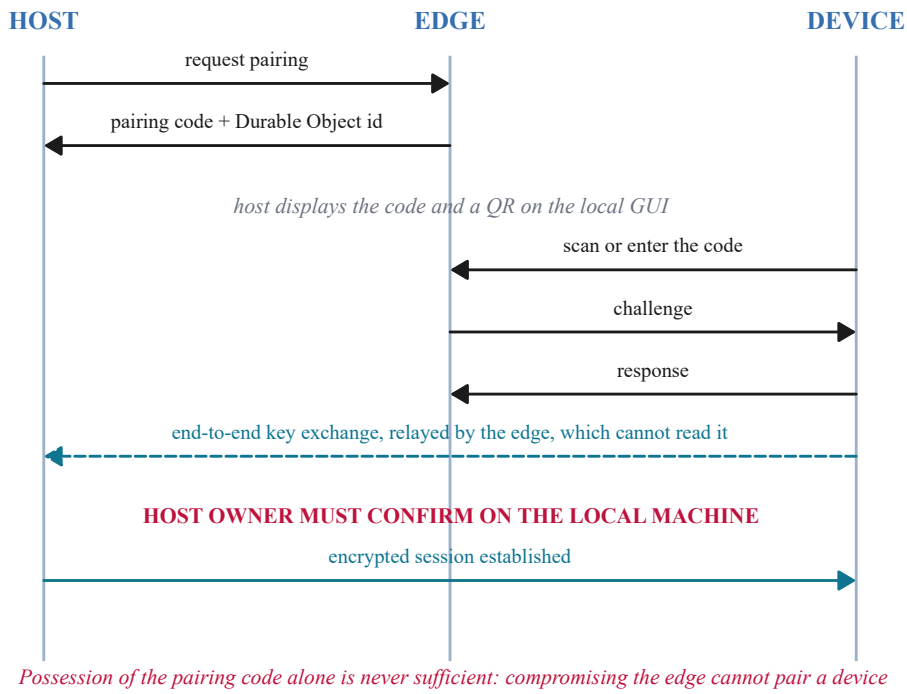


Figure 6.14: The pairing handshake.

remains cancellable.

Every queue has a limit. When one fills up, it drops its lowest-priority work and says so, and it does not grow without bound. An unbounded queue turns a load problem into a memory problem, and a system that quietly falls behind behaves in ways the user cannot predict.

6.5 Domain Model/Class Diagram

The domain model has four immutable value objects and five services that work on them. The objects are immutable on purpose, so nobody can attach a verdict to one call and then change the call before it runs.

A ToolCall represents one proposed action, carrying an operation name, a workspace identifier, a set of paths relative to that workspace, arguments and a unique identifier. Its paths are relative rather than absolute, so that a call cannot smuggle an absolute path past the boundary by careful construction. It provides a description for previews and logs, and a fingerprint identifying the shape of the call independently of the paths it touches, which is the scope at which remembered approvals apply.

A Verdict is the policy engine's conclusion about one call. It holds the identifier of the call it was issued for, the tier, the decision, the reason and the rule applied. It can say whether approval is needed and whether the call was refused, and it can confirm that it matches a given call. That match check is what stops a call being classified as a read and then run as something else.

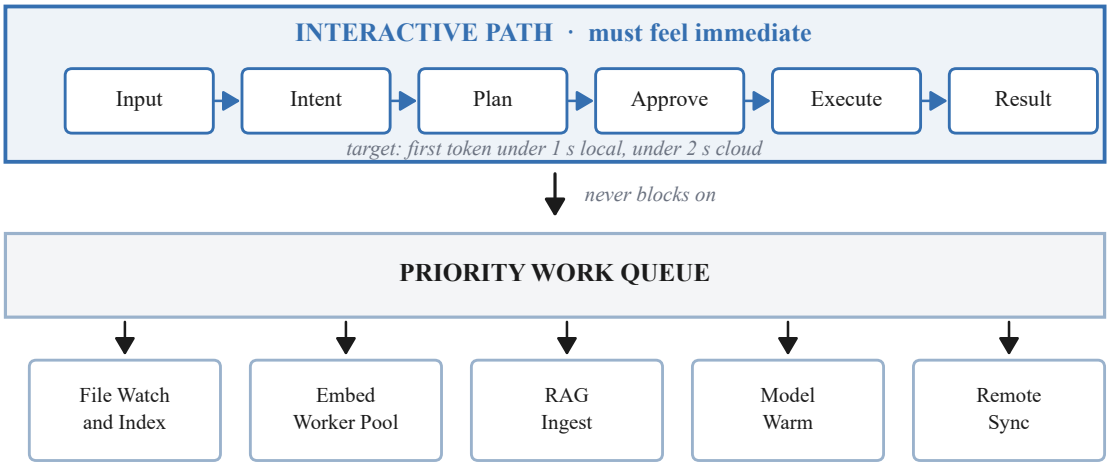


Figure 6.15: Asynchrony and background progress.

A Plan is an ordered collection of calls with an intent and an identifier, and it is the unit at which approval and reversal operate.

A Resolved path is one that has passed every assertion in the broker. Services take this type and not a plain path, so an unchecked path cannot reach the filesystem by mistake. If you hold one, the boundary was tested.

The services are the WorkspaceBroker, which resolves references or refuses; the PolicyEngine, which classifies calls and holds remembered rules; the ApprovalBroker, which holds pending plans, constructs previews and records decisions; the UndoJournal, which records inverse operations and replays them; and the ToolDispatcher, which is the only component that acts.

Two relationships in this model carry the architecture's guarantees. The dispatcher depends upon the approval broker rather than accepting an approval flag from its caller, so that a service cannot assert its own permission. The dispatcher also resolves paths through the broker a second time, despite the policy engine having already done so, because the dispatcher must not depend upon an earlier component having performed its function correctly; that assumption is the substance of confused-deputy defects.

6.6 Policies and Tactics

This section lists the policies and tactics we follow while implementing. They do not change the overall structure, but they decide details of the interface and the code.

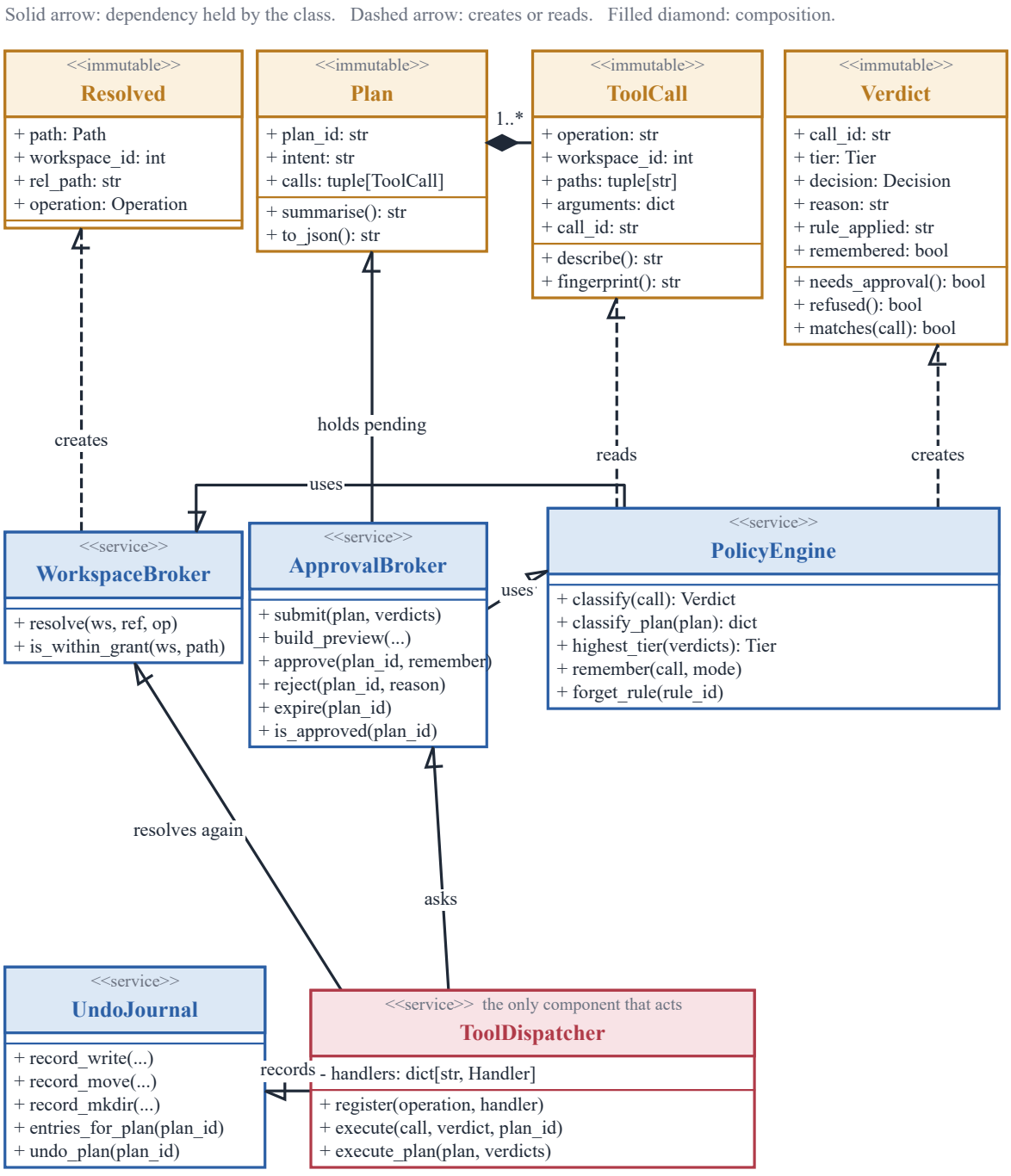


Figure 6.16: Class diagram.

6.6.1 Refusal as the Default Failure Mode

When a situation is ambiguous, we refuse. An operation the system does not recognise is denied, not treated as a read. A path that cannot be resolved is refused and not attempted. A call whose verdict is missing or does not match is refused. An expired approval can never run, which is why expiry is its own outcome and not a kind of rejection or a silent discard.

We rejected the opposite approach, permissive defaults with lists of denied operations. A denial list has to anticipate every dangerous operation, while a permission list only needs to name the safe ones, and forgetting an item has very different consequences in the two cases.

6.6.2 Single Enforcement Points

Resolving paths belongs to the broker alone and running actions to the dispatcher alone. Handlers are registered on an allowlist. An operation with no registered handler cannot run, so adding a capability is a deliberate step and not something that happens because a planner invented a name.

This policy has a maintenance implication that must be stated for any future developer: introducing a second path to the filesystem or a second execution point would not be a refactoring but a change to what the system guarantees, and either would invalidate the testing strategy, which depends upon each guarantee having one place at which it can be verified.

6.6.3 Ordering Constraints

Three orderings carry the load, and none may be changed. Canonicalisation comes before the containment test. The snapshot comes before the change. Policy evaluation comes before dispatch. Each is a matter of correctness and not of taste. Reverse any of them and you get a system that looks like it works while giving none of the guarantee it claims. So each has its own test, which is meant to fail if the order is disturbed.

6.6.4 Testing Strategy

We organise testing around the guarantees and not around the code. The boundary is attacked: relative traversal, symbolic links pointing outside the grant, absolute paths and sibling directories that share a name prefix. Every attempt must be refused and recorded. Policy classification is tested across all operation sets, including the unrecognised case. Approval is tested for two properties: nothing is approved by default, and outward operations cannot be remembered. Recovery is tested by running a plan, undoing it and comparing the result with the state recorded beforehand.

Every stage of the build order ends with a gate test. A stage is done when its gate passes, not when its code exists. The gates are written so they can fail. The clearest case is the speech stage gate, which fails if voice needed any new approval gate.

6.6.5 Concurrency Handling

The interactive path runs in one thread, but more than one thread reaches the store, because the file watcher flushes its debounce buffer on a timer. So the store puts a lock around every statement, and all access goes through its guarded methods and not straight to the connection. If we turned off the database engine's own thread guard without this lock, a visible error would become silent corruption, so the two go together.

The audit log's chain read and append occur under a single lock, so that two threads cannot chain onto the same predecessor and fork the chain.

6.6.6 Source Organisation

We organise the source by architectural role and not by feature, so the code lines up directly with the design. The broker, policy engine, approval broker, undo journal, dispatcher and disclosure panel are in one package. The data plane is in another. The interface surfaces are in a third. Anyone looking for where a guarantee is enforced will find it where the architecture says it is.

6.7 Conclusion

This chapter has presented the design of the system: the layers and their components, the strategies that give the architecture its shape, the domain model and the policies that hold the design together. Each significant decision was traced to a requirement or a research finding. The design places path resolution, classification and execution each in a single component, keeps the remote plane detachable, and treats refusal as the default failure mode, so that the guarantees the hypothesis depends on are properties of structure and can be tested as such. The next chapter describes the prototype that implements this design and the tests that exercise it.

Chapter 7 Implementation and Test Cases

This chapter describes the prototype that was built during the first phase of the project and the tests that exercise it. As the template allows for the first deliverable, the implementation section is a description of the prototype and not of a finished product. It gives the platform and interfaces used, the algorithms behind the components that are not apparent from the design, and the state of each stage of the build sequence. The second section sets out the design of the test cases and presents a representative set in the prescribed layout, each traced to the requirements of Chapter 4. The third section reports the test metrics and the traceability of requirements to tests. The chapter does not reproduce source code; it records the procedures that the code implements.

7.1 Implementation

7.1.1 Scope and Status of the Prototype

The prototype is a working desktop assistant written in Python 3.13. It comprises 56 modules and about 14,250 lines of source, organised by architectural role into five packages as Chapter 6 describes: the core enforcement layer (26 modules, about 5,400 lines), the local data plane (4 modules, about 850 lines), the capability services (10 modules, about 2,100 lines), the presentation surfaces (5 modules, about 4,450 lines) and the optional remote plane (10 modules, about 1,450 lines). A further relay program in JavaScript and a phone page complete the remote plane. It is accompanied by an automated suite of 554 tests.

All eight stages of the build sequence of Table 5.2 have been implemented, each ending in the gate test that Chapter 5 requires. Table 7.1 records the state of each stage. A stage counts as having its gate passing when the automated suite passes. It counts as validated through the interface only where the user-interface campaign of Chapter 9 supports it, and that campaign found a number of defects that the automated gates did not, so the two columns are reported separately.

Table 7.1: Implementation status by stage.

Stage	Deliverable	Automated gate	Interface campaign (Chapter 9)
1	Workspace foundation	Passing	Gate partly met; folder guard-rails missing
2	Policy, approval and undo	Passing	Gate partly met; undo defect on name collisions

Table 7.1: continued from previous page

Stage	Deliverable	Automated gate	Interface campaign (Chapter 9)
3	Gateway: router, context, planner, dispatcher	Passing	Gate met for execution; one retrieval isolation failure
4	Resume, tidy, repeated tasks	Passing	Gate partly met
5	Notes synthesis and code assistance	Passing	Gate not met: unsupported claims not always labelled
6	Web search, mail, drafting	Passing	Gate met; visibility of what left the computer weak
7	Speech input and output	Passing	Gate met (no new approval gate); recording control defect
8	Remote access plane	Passing	Not fully tested; one boundary defect

7.1.2 Platform and Interfaces

Table 7.2 lists the platform, libraries and services on which the prototype depends and the role of each. Every dependency of the core is local. The network is used only by the optional cloud providers, the web search service, the mail account and the remote plane, each behind its own gate.

Table 7.2: Platform, libraries and services.

Component	Role
Python 3.13, pytest	Implementation language and test runner (554 tests).
SQLite [3]	The system of record: workspaces, file index, audit log, undo journal, chat sessions, settings; one vector index database per workspace.
Ollama [2], model gemma4:e4b	Default local language model, reached over the loopback interface; model name is a user setting.
nomic-embed-text through Ollama	Local text embedder for the vector index.
Cloud providers (Gemini, OpenAI, Anthropic)	Optional fallback tiers, available only to workspaces that opted in.

Table 7.2: continued from previous page

Component	Role
LangGraph [1]	State-graph runtime for the planner, with an explicit pause before any gated call.
NumPy	Cosine similarity over a workspace's embedding matrix.
watchdog	File watcher feeding incremental indexing.
Gradio 6.12	The web interface with tabs Work, Folders, What it knows and Settings.
PySide6, Typer	A desktop panel and a command-line surface over the same components.
faster-whisper	Local speech recognition.
Kokoro, Piper, Windows speech	Speech synthesis, tried in that order.
cryptography, keyring	X25519, HKDF and AES-256-GCM for the remote plane; storage of keys and passwords in the operating system keyring.
Cloudflare Worker with a Durable Object	The deployed relay of the remote plane and the host of the phone page.

7.1.3 Implementation of the Workspace Broker

The broker is the only component that turns a reference into a path. It applies five assertions in a fixed order: the reference is resolved against the active grant root; it is canonicalised, so that symbolic links and parent-directory segments are replaced by the true location; the canonical path is tested for containment in the grant root; it is tested against a deny list; and the requested operation is compared with the permission of the grant. Canonicalisation precedes the containment test because a symbolic link inside the grant whose target lies outside it looks contained until it is resolved. The deny list covers credential and system locations such as `.ssh`, `.aws`, `.env` and key files, and always covers the application's own store, so that a grant on a parent folder cannot expose the audit log. A successful resolution returns a value that records the workspace, the relative path and the operation. Services accept only this value and not a bare path, so that an unchecked path cannot reach the file system by accident. Every refusal is written to the audit log before the refusal is raised.

7.1.4 Implementation of the Policy Engine and Approval Broker

The policy engine reads a proposed call and nothing else. It never sees the model's text, so nothing written inside a document can argue its way to a different verdict. Evaluation runs in a fixed order, being the grant check, the destructive check, the egress check, the mutation check and finally allow, and the first rule that matches decides. A call is placed in exactly one tier. Reads run without approval. Reversible changes are held unless the user has chosen to allow that operation without asking in that workspace. Outward actions are always held and can never be remembered. Destructive operations are refused with no approval path, and an operation the engine does not recognise is refused as well, so that omission can never widen what is permitted. A plan takes the highest tier among its steps. Classification never raises an error: a call that cannot be classified is a refusal.

The approval broker keeps each plan in one of four states, being pending, approved, rejected and expired, and none of the transitions out of pending is automatic. A plan cannot be decided twice, expiry does not count as approval, and a remembered approval is scoped to one operation in one workspace. The preview shows the workspace, the proposed operations, the tier, whether anything leaves the computer and whether anything is deleted, and for outward steps shows the exact search text or message body, so the user approves what will actually happen.

7.1.5 Implementation of the Dispatcher and the Undo Journal

The dispatcher is the only component that acts. It applies four refusals to every call in order: a call with no verdict is refused; a verdict issued for a different call is refused, because each verdict carries the identifier of its call; a denied call never executes; and a gated call is refused unless the approval broker holds a recorded approval for its plan. The approval is looked up and is never passed in by the caller. The dispatcher also resolves the call's paths through the broker at execution time, so the boundary holds even if an earlier layer were bypassed.

The undo journal records how to reverse a change before the change is made. A small file is copied into a content-addressed blob store, so identical contents are stored once, and a file above four mebibytes is hard-linked where the file system permits. A move or a newly created file needs no copy because its inverse is structural. Undo works at the granularity of a plan, reversing every recorded step in reverse order, and a folder that the user has filled in the meantime is left in place. Both the dispatcher and the journal write to an append-only audit log in which each row carries the hash of the previous row, so that removing or editing an entry breaks the chain, which can be verified on demand. Chain read and append occur under one lock so that two threads cannot fork the chain.

7.1.6 Implementation of the Gateway

A typed request first passes through an intent step that decides whether it is a question, a file task, a draft, a search, a mail request or a request about code. The step is rule-based, using patterns over the request text, so that routing is deterministic, fast and inspectable and does not depend on a model's judgement. The model router then chooses a provider. Providers are tried in tier order, local first, with one retry per tier before advancing, so that an unreachable chain degrades within seconds and with an explanation. The router is also the single place where egress is decided: a workspace that has not opted into cloud has exactly one tier available, and if the local model is unreachable the answer is a refusal and not a quiet promotion to a cloud provider.

The context engine assembles what the model is allowed to see. Each fragment carries its provenance, being the user's own words, a workspace file, a tool result, memory or the web. Only the user's words are authoritative; every other source is presented to the model as data. Retrieval is restricted to the active workspace, and the size of the context is budgeted using an estimate of 3.6 characters per token. The planner is a LangGraph state graph that asks the model for a structured plan at a low sampling temperature of 0.1, validates it against the set of plannable operations, allows up to three planning cycles within a time budget of 30 seconds, and returns the plan without executing it. The model may therefore propose, but it holds no handle by which it can act.

7.1.7 Implementation of Indexing, Retrieval and Grounding

Files in a workspace are cut into chunks of at most 800 characters, each carrying its source file and line range so that a citation can point back to it. Each chunk is embedded by the local embedder, normalised, and stored as a float32 vector in a database private to the workspace. A search embeds the question and computes cosine similarity against the workspace's matrix by a single matrix product, returning the best chunks. Indexing is incremental: a file whose content has not changed is not embedded again, files above two mebibytes are skipped, and a background worker indexes without blocking the interactive path. A file watcher with a debounce buffer notifies the index of changes.

The grounding gate is applied to every answer to a question about the user's files. The answer is split into claims, one per sentence, and a claim is supported only if it carries a citation that names a chunk which was actually sent to the model and which is a file chunk and not the user's own words. Any other claim is shown with an unsupported label. The gate establishes that a source exists and was provided, and it does not establish that the source says what the claim says, which is a limitation inherited from the literature and recorded as such in Chapter 3.

7.1.8 Implementation of the Capability Services

All services reach the file system only through the broker and the dispatcher. Resuming reports the files most recently active in a workspace together with a note that the user left for next time. Tidying groups loose files in a folder by kind, using extension tables for documents, spreadsheets, images and others, or by date on request. It plans the creation of each destination folder before the moves into it, leaves alone a file that has no sibling of its kind and files already filed in subfolders, gives a colliding name a new name and not an overwrite, and shows every move in the preview. The service holds no deletion operation to call. Repeated-task detection computes a signature for each approved plan, ignores undone plans and plans the user has declined, and offers an automation only when the same signature has occurred at least three times within a window of 30 days. Code assistance answers questions about code and returns any proposed change as a difference, and nothing in the service writes to a file. Web search runs only on an explicit request, through a public search endpoint, shows the query in the preview, records every query, and tags results as coming from the web. Mail composes a draft and sends it through the user's own mail account only after the draft is approved; the recipient is validated, header injection is refused, and the recipient and time of every send are recorded. Drafting writes a new file with a name that does not clash and never overwrites; undo removes it.

7.1.9 Implementation of Disclosure

The panel titled What ARTEMIS knows is a live query over the store and not a cached list. It reports each workspace, each indexed file, the embedded chunks, remembered facts, retained chats, automation rules, policy exceptions and the record of outward actions, and it offers deletion of each item, with a confirmation, other than the audit log. Revoking a workspace deletes the index entries, embeddings, sessions, memory and rules derived from it and leaves the user's own files untouched.

7.1.10 Implementation of Speech

Speech is a modality at the interaction layer and not a service. Audio is accepted only while a voice session has been armed by the user. A recording is transcribed on the computer by faster-whisper, deleted after it is read, and deleted even if transcription fails. The audit log records that voice was used and never what was said. The transcript is passed to exactly the same entry point as a typed request, so a spoken request meets the same gates, and a spoken email waits for approval like a typed one. Replies are read aloud by the first available of three engines, tried in order of naturalness. Before speech, the reply is stripped of the model label and of markup and is shortened, and a spoken chat is worded for the ear. The browser holds the hands-free loop: after a reply it listens, treats about 3.5 seconds of silence

after speech as the end of the user's turn, and, if the user says nothing for about four seconds, speaks first and then keeps listening.

7.1.11 Implementation of the Remote Plane

The remote plane lets a paired phone talk to the computer through a relay that carries opaque frames and knows nothing else. A device is paired by an exchange of X25519 keys in which the short pairing code is mixed into the derived key, so that a relay which substitutes keys gains nothing, and the owner confirms on the computer after comparing a short safety number. Frames are sealed with AES-256-GCM using a per-session key and counters, so a replayed frame is not answered twice. The phone's key is held as a non-extractable key in the browser, and the host's key in the operating system keyring. A session lasts one hour and must then be opened again. The bridge exposes only asking and answering: folders are hidden from a device unless the owner has ticked them, outward approvals are not accepted from a device and must be given on the computer, voice from a device is off by default, and every remote request is logged with its device. A device can be revoked at once. For voice, the phone streams short chunks of 16 kHz audio over the encrypted channel, the computer decides when the turn has ended by counting audio time, and the reply is returned sentence by sentence as text and speech. The plane is optional by construction, because nothing in the core imports the remote package and the main database holds no remote tables. The relay is deployed as a Cloudflare Worker with a Durable Object that also serves the phone page, and a runnable relay with the same rules is provided for use on a home network and for tests.

7.1.12 Implementation of the User Interface

The web interface has four areas. Work holds the request box, the folder selector, the approval cards and the saved chats, which are kept for seven days, together with the voice controls. Folders adds and lists workspaces by typed path. What it knows is the disclosure panel. Settings holds the model choices, the cloud policy per folder, the mail account and the remote access controls. Gradio 6.12 imposes some constraints on interaction, and the interface works within them by using hidden controls that scripts in the page trigger. The same components are reachable through a desktop panel and a command line.

7.2 Test case Design and description

The tests are organised around the guarantees of Chapter 4 and not around the code structure. Their common attributes are as follows. Every test runs against a throwaway store, selected by an environment variable, and against a throwaway folder that stands in for the user's files, so that no test touches real

data and tests cannot affect each other. No test needs the network, a cloud key or a running language model: model replies are supplied by stub providers, embeddings by a deterministic fake embedder, mail and search by stubs, and speech recognition and synthesis by stand-ins. The suite therefore tests the enforcement layer and the plumbing deterministically and cannot test the quality of what a real model says. Gate tests are written so that they can fail, so that a gate asserting a property is tested by an attempt to break it: escape attempts for the boundary, an unrecognised operation for the policy engine, a call with no verdict for the dispatcher, and a requirement for any new approval path in the case of speech. Each test case below states the tests it summarises, and the tables follow the layout prescribed by the template.

7.2.1 Test Cases

The nine test cases below are TC-01 to TC-09, in Tables 7.3 to 7.11.

Table 7.3: Test case TC-01: workspace boundary.

Workspace Broker		
Requirements FR-2, FR-3, FR-4; stage gate of the workspace foundation stage (tests/test_broker.py, 13 tests)		
Test Case ID: TC-01		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-1, UC-3
Revision History:	None; first version.	
Objective:	Establish that no reference can resolve to a path outside an active grant, that every attempt is recorded, and that the record is intact.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. The broker is the single component that turns a reference into a path.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	The machine can create symbolic links (checked at run time; the symbolic link case is skipped otherwise).	
Pre-Requisite:	A folder with two files is granted as a workspace; a second folder holding a file named secrets.txt exists beside it and is not granted.	
Step No.	**Execution description**	**Procedure result**
1	Resolve readme.txt inside the grant.	A resolved path is returned, with the workspace identifier and the relative path.

Table 7.3: continued from previous page

2	Resolve references that climb out of the grant with parent-directory segments.	A boundary refusal is raised with the reason outside grant.
3	Place a symbolic link inside the grant that points at the ungranted folder, then resolve a file through it.	Refused. The link looks contained; only its canonical target is not, which is what canonicalising first exists to catch.
4	Resolve an absolute path and a sibling folder whose name begins with the grant's name.	Both refused; a shared name prefix is not containment.
5	Read the audit log after a refused attempt.	One refused entry exists for the attempt and contains the reference text; the hash chain verifies.
Comments: All 13 tests passed, including the symbolic link case on the test machine.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.4: Test case TC-02: policy engine classification.

Policy Engine		
Requirements FR-7, FR-8, FR-9, FR-10, FR-11 (tests/test_policy.py, 17 tests)		
Test Case ID: TC-02		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-2, UC-3
Revision History:	None; first version.	
Objective:	Show that the four tiers behave as specified and that the destructive tier has no approval path.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. Classification reads the tool call only and never the model's reasoning.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Remembered approvals are off by default.	
Pre-Requisite:	A folder is granted as a workspace.	
Step No.	**Execution description**	**Procedure result**
1	Classify a read of a file inside the workspace.	Allowed to run without asking.

Table 7.4: continued from previous page

2	Classify a change to a file and a call that sends something out of the computer.	Both held for approval; the second is flagged as leaving the computer.
3	Classify a delete operation.	Tier destructive, decision refuse. No approval can change this.
4	Classify an operation the engine has never heard of (rm_rf).	Refused under the rule unknown-operation, not assumed safe.
5	Classify a read whose path escapes the grant.	Refused under grant-check, which runs before every other rule.
6	Ask to remember an outward action and a destructive action.	Both remain gated or refused; remembering is scoped to one operation in one workspace.
Comments: All 17 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.5: Test case TC-03: tool dispatcher.

Tool Dispatcher		
Requirements FR-17, FR-35; invariants of Chapter 6 (tests/test_dispatch.py, 11 tests)		
Test Case ID: TC-03		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-2
Revision History:	None; first version.	
Objective:	Show that the dispatcher cannot be talked into acting without a verdict and an approval, and that a full plan can be gated, executed and undone.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. The dispatcher is the only component that acts.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Handlers are registered by the services; unregistered operations have none.	
Pre-Requisite:	A granted workspace with a file readme.txt containing the word hello.	
Step No.	**Execution description**	**Procedure result**
1	Execute a read call with no verdict.	Refused: no policy verdict.

Table 7.5: continued from previous page

2	Execute a change to readme.txt after classification but without an approval for its plan.	Refused: needs approval. The file still contains hello.
3	Execute a call the policy denied, and a call for an operation with no handler.	Neither executes.
4	Execute a call whose path escapes the workspace, with a verdict that claims otherwise.	Refused again at execution, so the boundary holds even if an earlier layer is bypassed.
5	Run a full plan: classify, submit, approve, execute, undo.	Gated, executed once, snapshotted before the change, and restored by undo; every refusal reaches the audit log.
Comments: All 11 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.6: Test case TC-04: undo of an approved plan.

Undo Journal		
Requirements FR-15, FR-16 (tests/test_undo.py, 10 tests)		
Test Case ID: TC-04		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-2
Revision History:	None; first version.	
Objective:	Show that recovery information is recorded before a change and that undo restores the state the user approved from.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. Snapshots are kept in a local blob store; moves need no copy.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Files are below the hardlink threshold unless stated.	
Pre-Requisite:	A granted workspace with two files, readme.txt and notes/chapter3.md.	
Step No.	**Execution description**	**Procedure result**

Table 7.6: continued from previous page

1	Record the creation of a Drafts folder and the moves of both files into it, performing each change after its record is written.	Both files sit in Drafts; the originals are gone.
2	Undo the plan by its identifier.	Three steps reversed, no problems reported.
3	Inspect the workspace.	Both files are back at their original paths and the created folder is removed.
4	Undo a plan after the user has put a new file into the created folder.	The folder is kept, because removing it would destroy the user's file.
5	Undo an unknown plan, and undo the same plan twice.	Reported, not raised; a reversed plan is cleared from the journal; each undo is written to the audit log.
Comments: All 10 tests passed. The campaign of Section 9.4 later found a name-collision case that these tests do not cover.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.7: Test case TC-05: approval broker.

Approval Broker		
Requirements FR-12, FR-13, FR-14, FR-17 (tests/test_approval.py, 12 tests)		
Test Case ID: TC-05		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-2, UC-4
Revision History:	None; first version.	
Objective:	Show that a pending plan stays pending until a person decides, and that the preview tells the truth.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	None.	
Pre-Requisite:	A tidy plan with several moves, and a plan containing an email.	
Step No.	**Execution description**	**Procedure result**

Table 7.7: continued from previous page

1	Submit the plan.	State is pending, and remains pending without further action.
2	Read the preview.	It states what will and will not happen, and flags anything that leaves the machine.
3	Reject the plan; separately let another plan expire.	Neither becomes approved; the stored outcomes are rejected and expired.
4	Decide a plan a second time.	Refused; a plan cannot be decided twice.
5	Approve and remember a plan that contains an outward step.	Only the gated reversible calls are remembered; the outward one is refused for remembering.
Comments: All 12 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.8: Test case TC-06: grounding gate.

Grounding gate		
Requirements FR-22, FR-23 (tests/test_grounding.py, 7 tests)		
Test Case ID: TC-06		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** Notes synthesis (Chapter 4, feature F3)
Revision History:	None; first version.	
Objective:	Show that the gate marks any claim that cannot be traced to a chunk of the user's own files that was actually sent to the model.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer, applied to model output before display.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Citations take the form [n], where n numbers a chunk in the context.	
Pre-Requisite:	A context of three chunks: the user's question, report.md line 4, and a second file.	
Step No.	**Execution description**	**Procedure result**
1	Check the claim Revenue grew 10% [2].	Supported, with the source report.md:4.
2	Check a claim with no citation.	Unsupported.

Table 7.8: continued from previous page

3	Check a claim citing a chunk number that was never sent.	Unsupported.
4	Check a claim citing the user's own question as its source.	Unsupported; the user's words are not evidence.
5	Render a two-sentence answer with one unsupported sentence.	Only that sentence is prefixed with the label [unsupported].
Comments: All 7 tests passed. The gate checks that a citation exists and points at a sent chunk; it does not check that the chunk says what the claim says (an inherited limitation, Chapter 3).		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.9: Test case TC-07: outward actions.

Outward actions: mail, search and drafting		
Requirements FR-10, FR-25, FR-26, FR-27 (tests/test_sprint7_gate.py, 24 tests; tests/test_mail.py, 11 tests)		
Test Case ID: TC-07		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-4
Revision History:	None; first version.	
Objective:	Show that sending mail and searching the web are always gated and recorded, and that drafting never overwrites.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, services layer, over the same dispatcher.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	A stub mail server and a stub search provider replace the network.	
Pre-Requisite:	A granted workspace and a stubbed mail account.	
Step No.	**Execution description**	**Procedure result**
1	Compose a message.	Nothing is sent; a draft is shown.
2	Classify the send and the search.	Both gated as outward.
3	Approve and remember a send.	Refused: an outward approval can never be remembered.
4	Reject the mail and then dispatch it directly.	Never sent, even when dispatched directly.

Table 7.9: continued from previous page

5	Approve and send; then read the log.	Sent once; the recipient and time are recorded.
6	Draft a document whose name already exists, and one whose name appeared after planning.	The drafter picks a new name; the existing file is untouched; undo removes the new file.
Comments: All 35 tests passed. Header injection in the subject and a malformed recipient are refused.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.10: Test case TC-08: remote device limits.

Remote plane: bridge and plane		
Remote access stage gate (tests/test_remote.py, 20 tests; tests/test_remote_network.py, 2; tests/test_remote_voice.py, 9)		
Test Case ID: TC-08		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** Remote access (Chapter 6, Section on the remote plane)
Revision History:	None; first version.	
Objective:	Show that remote access is optional, encrypted end to end, and strictly below local access.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, remote package. Nothing in the core imports it.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	A local relay (in-memory or a real WebSocket on the loopback interface) stands in for the deployed edge.	
Pre-Requisite:	A workspace is granted; a device is paired and confirmed on the computer.	
Step No.	**Execution description**	**Procedure result**
1	Pair without the owner's confirmation; pair with a wrong code.	Not paired; repeated wrong codes lock the pairing.
2	Capture every frame the relay carries.	No plaintext appears; a replayed frame is not answered twice.

Table 7.10: continued from previous page

3	Ask for a change from the device.	A plan is held; nothing changed on disk until the device approves, and then only the approved change occurs.
4	Ask for an email and approve it from the device.	The answer is waiting on the computer; the plan is still pending locally.
5	Send settings, forget_key, add_folder, erase and set_model requests.	Every one returns an error.
6	Revoke the device, and, separately, delete the remote package and start ARTEMIS.	The device is cut off at once and for good; ARTEMIS runs unchanged.
Comments: All 31 tests passed. The campaign of Section 9.4 found a case this suite missed: a waiting plan still ran after its folder was un-shared.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.11: Test case TC-09: speech and the approval gates.

Speech input and output		
Requirements FR-28, FR-29 (tests/test_speech.py, 14 tests)		
Test Case ID: TC-09		**QA Test Engineer:** Project team (automated suite)
Test case Version: 1.0		**Reviewed By:** Project team
Test Date: 9 November 2026		**Use Case Reference(s):** UC-2, UC-4
Revision History:	None; first version.	
Objective:	Show that voice is another way to type: it is never recorded unless armed, and it cannot approve, send or delete.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, services layer. Speech engines are replaced by stand-ins for the test.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS_HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Transcription and speech synthesis are stubbed.	
Pre-Requisite:	A granted workspace and a voice session in its default, disarmed state.	
Step No.	**Execution description**	**Procedure result**
1	Offer audio to a session that has not been armed.	Refused unread.

Table 7.11: continued from previous page

2	Arm the session, read a recording, then disarm.	Listening stops; the recording file is deleted after it is read, and also when transcription fails.
3	Speak a request to move a file.	A plan is held for approval exactly as for a typed request.
4	Speak a request to email someone.	A draft waits for approval; nothing is sent.
5	Read the audit log.	It records that voice was used and never what was said.
Comments: All 14 tests passed. These tests use stand-in engines, so they establish the gates and not the quality of recognition or speech; the campaign testers could not hear audio either.		
Passed [X] Failed [ ] Not Executed [ ]		

7.3 Test Metrics

Table 7.12 gives the test metrics for the automated suite at the end of the first phase, using the metrics prescribed by the template. The suite was executed in full on 9 November 2026 and took 42 seconds.

Table 7.12: Test metrics.

Metric	Value	Basis
Number of test cases	554	Collected from 39 test files, one file per component or stage gate.
Number of test cases passed	554	Full run of 9 November 2026.
Number of test cases failed	0	Same run.
Test case defect density	0%	Failed multiplied by 100, divided by executed: $0 \times 100/554$.
Test case effectiveness	0% against interface defects	Defects detected by the test cases multiplied by 100, divided by total defects detected, taken over the 71 distinct defects of the interface campaign: all were found while the automated suite was green, so $0 \times 100/71$. See the discussion below.

The first three figures describe a suite that passes completely, and the fourth shows why that is not sufficient. The defects that the later interface campaigns found were not detected by the automated tests, because those tests checked each component in isolation and the defects were in the way components are joined to the interface. In the first hands-on round, 359 unit tests were passing while only one of nine

features worked end to end. The effectiveness metric is therefore reported as it is. It is computed over the interface defects only, because the defects that the unit tests caught during development were not logged separately. The lesson drawn from the figure is a practice that continues into the second phase: every defect found at the interface is to be given a regression test before it is closed.

7.3.1 Traceability Matrix

Table 7.13 traces each group of requirements of Chapter 4 to the test files that verify it, and to the result of the interface campaign. Each group of requirements from FR-1 to FR-35 has automated tests. The audit requirement FR-18 has no file of its own because it is asserted inside the tests of the broker, the policy engine, the approval broker, the dispatcher and the undo journal, each of which checks that its refusals or actions reach the log. The remote plane, which carries no numbered functional requirement, is verified by 31 tests.

Table 7.13: Requirements traceability.

Requirement group	FRs	Verifying tests (count)	Campaign
Workspace grant, boundary, index	1 to 6	broker (13), workspace and index (13), watcher (5), vector index (11)	Partial
Classification, approval, execution	7 to 14, 17, 35	policy (17), approval (12), dispatch (11), gateway (11), planner (23)	Partial
Recovery	15, 16	undo (10)	Partial
Audit	18	Assertions within the five files above	Partial
Resume, tidy, repeated tasks	19 to 21	resume (17), tidy (12), patterns (16)	Partial
Synthesis, grounding, code	22 to 24	grounding (7), answer (6), chunking (9), code diff (11)	Fail on the gate
Outward actions	25 to 27	sprint 7 gate (24), mail (11)	Pass
Speech	28, 29	speech (14), remote voice (9)	Fail on recording control
Disclosure	30, 31	panel (18)	Partial

Table 7.13: continued from previous page

Requirement group	FRs	Verifying tests (count)	Campaign
Model access and context	32 to 34	router (18), cloud policy (3), providers (2), model configuration (29), context (10)	Partial
Interface and conversation (not numbered)	–	intent (39), chat (34), assistant (32 and 15), web (29), capabilities (10), credentials (12), others	Partial

7.4 Conclusion

The prototype implements all eight stages of the build sequence and its enforcement layer behaves as specified under 554 automated tests, none of which fails. The implementation follows the design in the respects that carry the hypothesis: path resolution occurs in one component, classification in one, and execution in one, and the model proposes without acting. The test design is adversarial where the guarantees are concerned and is isolated from the network and the model, which makes it fast and repeatable and also limits it, since it cannot judge what a real model says or how the interface behaves under use. The metrics show this limit candidly. The gaps between a green suite and the defects of the interface are the subject of the next two chapters.

Chapter 8 User Manual

The user manual will be added in FYP-II, as the template provides. The prototype can be started and used today from the instructions in the repository, and the running instructions and a description of the four areas of the interface are kept there. A complete manual, with annotated screens, one walkthrough for each feature and a section on how to inspect and erase what the assistant holds, will be written in the second phase once the interface has been stabilised after the defects reported in Chapter 9 have been fixed, so that the manual describes the interface that is finally delivered and not one that is still changing.

8.1 Conclusion

The user manual is deferred to the second phase for the reasons given above, so that it is written once against the stabilised interface.

Chapter 9 Experimental Results and Discussion

This chapter reports the experiments that were conducted in the first phase and discusses what they show about the central hypothesis, that restricting an assistant's scope to explicitly granted folders increases trust and adoption without meaningfully reducing usefulness. The controlled evaluation designed in Chapter 5, which measures task time, workload, usability and trust with participants, is scheduled for the second phase and is not reported here. What the first phase supports is a different and smaller set of questions: whether the guarantees hold in the prototype, whether the prototype performs acceptably, how it fares when used as a person would use it, and how prospective users react to the idea and to the prototype. Six experiments address these questions, and they are summarised in Table 9.1. Each is reported with its method, its results in tables and its limits.

Table 9.1: The experiments reported in this chapter.

ID	Experiment	Question answered	Evidence type
E1	Automated gate and unit tests	Do the guarantees hold under adversarial tests?	554 deterministic tests
E2	Performance measurements	Are the enforcement and retrieval costs acceptable?	Timed runs on one machine
E3	Two hands-on rounds through the interface	Does the whole system work for a user?	Scripted interface sessions
E4	Exploratory interface campaign, 27 sessions	What breaks when the system is used in a human-like and adversarial way?	Automated agents driving the real interface
E5	Observations of prospective users, 30 people	Is the idea trusted and would the product be adopted?	Reported conversations
E6	Exploratory persona simulation, 15 personas	Does the idea hold under sceptical review?	Synthetic evidence

9.1 E1: Automated Gate and Unit Tests

Method. The full suite was run on 9 November 2026 on the development machine. **Result.** All 554 tests passed in 42 seconds. The policy engine, approval broker, dispatcher and undo journal account for 50 of the tests, the speech tests for 14 and the remote plane for 31, with the remainder spread over the other components listed in Table 7.13. The properties asserted include that traversal, symbolic link, absolute

and sibling-prefix escapes are refused and logged (TC-01); that the destructive tier has no approval path and unrecognised operations are refused (TC-02); that nothing executes without a verdict and an approval (TC-03); that an approved plan reverses as a unit (TC-04); that a claim without a real source is labelled (TC-06); that voice creates no new approval path (TC-09); and that a paired device cannot approve an email (TC-08). **Limit.** The suite proves that each guarantee holds where it is applied, and not that it is applied everywhere. Section 9.4 shows the difference.

9.2 E2: Performance Measurements

Method. A script measured the enforcement path and retrieval on one Windows 10 machine, using a throwaway store, folders of 100, 400 and 1,600 small text files, and the deterministic embedder. Timings for the enforcement path are medians of 300 to 2,000 repetitions. The numbers therefore describe the cost of ARTEMIS's own machinery, excluding the language model and the real embedding model, which were not part of this measurement.

Table 9.2: Enforcement and retrieval cost.

Operation	Median (ms)	95th percentile (ms)	Maximum (ms)
Broker resolves a valid reference	0.32	0.37	1.22
Broker refuses a traversal, including the audit write	1.14	2.76	27.1
Policy classifies a read	0.34	0.40	0.56
Policy refuses a delete, including the audit write	0.96	2.86	9.40
Search over 100 files	2.16	2.30	2.63
Search over 400 files	2.16	2.26	3.12
Search over 1,600 files	2.17	2.25	5.47
Verify the audit chain of 2,306 rows	10.9	–	–

Table 9.3: Time to scan and to index a folder, by size.

Files	Scan (s)	Embed and index (s)
100	0.62	0.48
400	1.96	2.26
1,600	7.98	7.64

Discussion. The boundary and the policy engine add well under a millisecond to an ordinary call, and

a refusal that must also write to the hash-chained log costs about a millisecond at the median, so the guarantees are not a measurable burden. Search latency stayed at about 2 milliseconds from 100 to 1,600 files, which is consistent with a single matrix product over a few thousand vectors and supports the requirement that retrieval stay fast as a folder grows, within that range. Scanning and indexing grew roughly in proportion to the number of files, at about 5 milliseconds per file for each step. **Limit.** Real embedding is slower than the stand-in used here, and an embedding model was not installed on the machine used for the interface campaign, so end-to-end indexing time on real hardware remains to be measured. The latency of the local language model was also not measured under controlled conditions in this phase, and it is the dominant delay a user will perceive.

9.3 E3: Hands-on Rounds Through the Interface

Method. On 8 November 2026 the whole system was driven through the real web interface in a headless browser, as a user would, and the result of each action was checked on disk. The first round used the build that had 359 unit tests passing, and a second round repeated every test after the defects had been fixed.

Table 9.4: Results of the two hands-on rounds.

Measure	Round 1	Round 2
Unit tests passing at the time	359	468
Features working from the interface	1 of 9	8 of 9 (speech not yet built)
Interface tests passing	Not applicable (first run)	36 of 36
Defects recorded	15 (one blocker, several major)	All 15 retested and fixed

Discussion. The first round found that approval buttons were hidden after a typed request, so a user could never approve anything, and that code questions, cited answers, web search, mail and drafting were unreachable from the interface even though their services passed their unit tests. The safety core nevertheless held from the start: deletion was refused, reads outside the folder were refused, and tidying and undo restored files exactly. All fifteen defects were fixed and retested on the interface, and every one of the 36 tests passed in the second round. The retest also exposed seven further problems that the fixes had introduced or revealed, among them a defect in the interface framework that froze tab clicks whenever a page-load event was used and a local model that spent its whole token budget reasoning and

returned an empty answer. All seven were corrected, which is a reason the retest was run on the real interface and not only in unit tests. **Limit.** These rounds were scripted and run by one tester, so they show that known paths work and say less about unanticipated use, which is what the next experiment addresses.

9.4 E4: Exploratory Interface Campaign

Method. Each of the nine areas of the prototype was tested by three automated testing agents, each driving the real web interface through a browser with a different disposition: a newcomer who follows the obvious path and makes ordinary mistakes, an impatient power user who double-clicks, refreshes and switches tabs, and an adversarial tester who supplies hostile, empty, very long and contradictory input and tries to make the system do what it promises never to do. This gave 27 sessions. Each used a private copy of the application and a private data folder, acted only through the interface, and verified outcomes on disk and in the server log. Each reported bugs and, separately, behaviour that works but that a user would not want. A consolidating step merged duplicates and compared the findings with the backlog.

Table 9.5: Session verdicts by area and stage gate status.

Area	**Newcomer**	**Power user**	**Adversarial**	**Stage gate**
Workspace foundation	Issues	Issues	Issues	Partial
Trust machinery	Issues	Issues	Fail	Partial
AI gateway	Issues	Issues	Issues	Pass
Core file features	Issues	Issues	Fail	Partial
Document and code understanding	Issues	Issues	Fail	Fail
Outward actions	Issues	Issues	Issues	Pass
Speech input and output	Issues	Fail	Issues	Pass
Remote access plane	Issues	Issues	Issues	Not tested
Conversation and everyday use	Issues	Issues	Issues	–

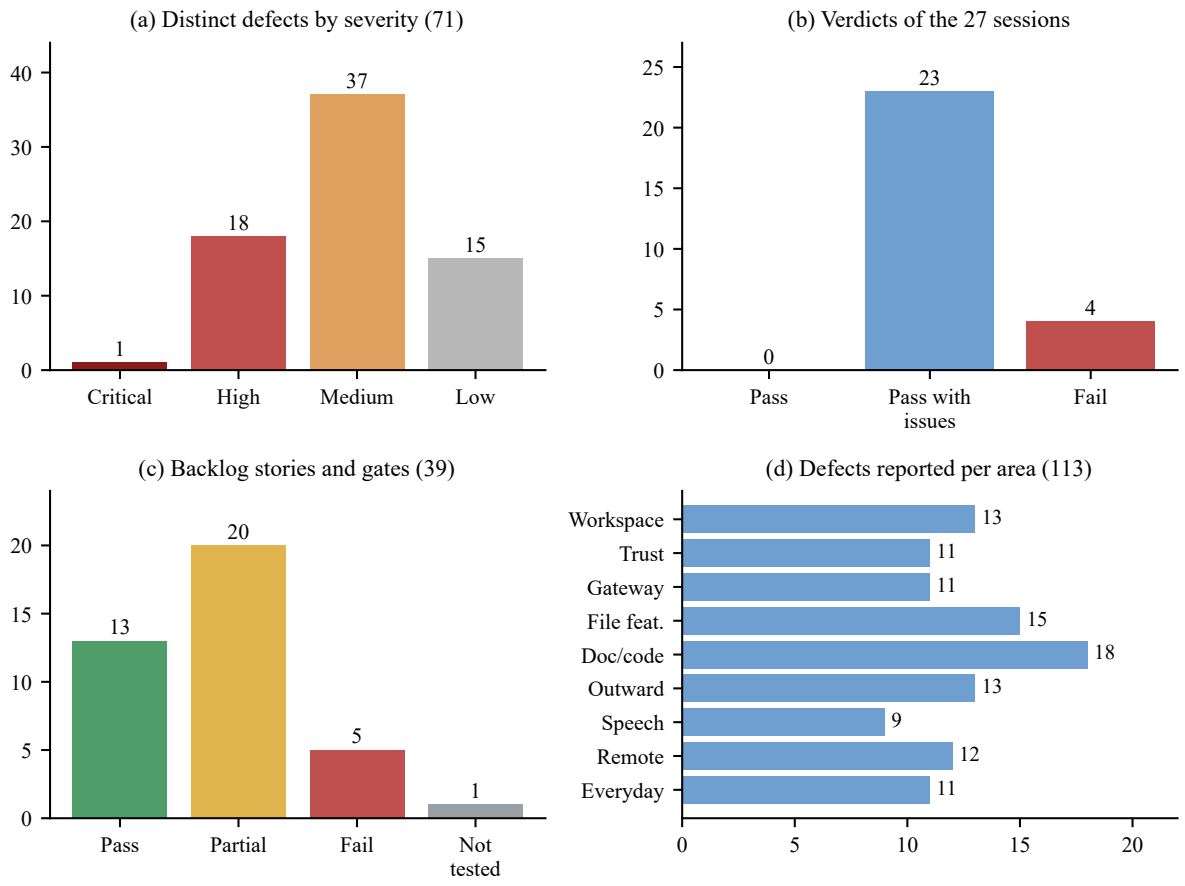


Figure 9.1: Results of the interface campaign.

Results. Figure 9.1 summarises the campaign. No session found a clean pass: none of the 27 sessions ended with the verdict pass, 23 ended with pass with issues and 4 with fail (panel b). Of the 39 backlog stories and stage gates, 13 passed, 20 passed in part, 5 failed and one could not be tested (panel c). The sessions reported 113 defects, which merged into 71 distinct defects, being 1 critical, 18 high, 37 medium and 15 low (panel a), together with 63 behaviours that work but that a user would not want. The reports were spread across all nine areas (panel d), with document and code understanding drawing the most (18) and speech input and output the fewest (9).

What held. Across many phrasings the system never deleted anything and refused any attempt to reach outside the chosen folder in plain words. Nothing left the computer, by web search or mail, without an approval card, and double clicks and stale clicks did not run a plan twice. Code questions produced only a difference and never an edit. Voice could not approve, delete or send. A phone could not approve emails or searches, saw only the folders ticked for it, and could be removed instantly. Tidy previewed every step, undo worked in ordinary use, and the rule that automation is offered only after three repetitions was exact.

What failed. The principal defects, with their severity, were the following.

- **Critical.** If a tidy meets two files that would take the same destination name, undo can lose one of them while reporting that the files are back as they were. This contradicts the promise that nothing is lost, even though no deletion operation exists, and it is the first defect to correct.
- **High, visibility.** The record of what ARTEMIS did exists on disk but no screen shows it in full, the page that reports changes can say that nothing changed immediately after files were moved, and a banner could continue to say that nothing had been sent after a search.
- **High, boundary.** The microphone could stay live after voice was switched off; a plan that a phone had left waiting still ran after its folder was un-shared; a conversation could carry the content of one folder into another; and any drive, including the system drive, could be added as a folder.
- **High, trustworthy answers.** The system sometimes gave an answer with no source, said that something was not found when it was in the files, cited the wrong file, or wrote an email that mentioned an attachment that did not exist. The gate that labels unsupported claims did not always fire.
- **Medium and low.** The voice panel crowded the page, messages sent while the system was busy or before a reload could vanish silently, and new files were not visible until the user asked the system to check for changes.

Discussion. The pattern is informative. The mechanisms designed to carry the hypothesis, which are the single resolution point, the tiers, the dispatcher and the undo journal, withstood adversarial use, and most of the serious defects lie in the joins around them: in how an interface control is wired to a gate, in what a screen reports, and in situations the unit tests did not model, such as a name collision or a folder un-shared while a plan is waiting. This supports the architectural choice of building the enforcement first and also shows that enforcement correctness in isolation is not sufficient to deliver the trust that the hypothesis predicts, since several of these defects would damage trust in practice. **Limits.** The testers were automated agents and not people, so their reports show what breaks and not how a person feels about it. They could not hear audio, did not see a cloud model answer or a real email leave, did not test the relay’s own side, the seven-day retention of chats or the removal of the remote plane, and ran on a machine without the embedding model, which probably inflates the retrieval failures. Several of the worst findings come from a single session and are marked as such in the consolidated report, so they warrant a re-check after correction. All 71 defects are carried forward as the work of the second phase.

9.5 E5: Observations of Prospective Users

Method. Members of the team spoke with 30 prospective users, who were students, developers, human resources staff and non-technical users. The idea was explained using the project pitch and participants tried the current prototype. They were asked whether they would trust an assistant that works as ARTEMIS describes, and they were free to say what they thought of the idea and of the prototype. The responses below are those reported by the team from these conversations.

Table 9.6: Stated trust, 30 prospective users.

Response	People	Share of 30
Fully agree they would trust it	10	33%
Partly agree they would trust it	16	53%
Total expressing trust	**26**	**87%**
Did not express trust	4	13%

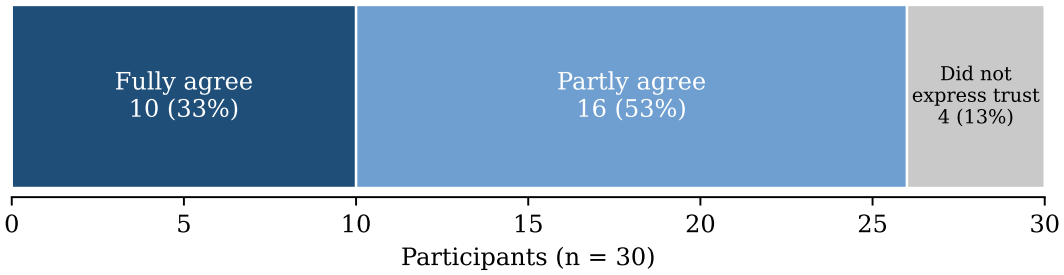


Figure 9.2: Stated trust among 30 prospective users.

Results. The overall idea was well received across all four backgrounds. Twenty-six of 30 expressed trust, 10 of them fully, and the partial agreement was consistently conditional: participants were willing to trust the assistant provided that it does exactly what it says. Participants who tried the application described it, in its current form, as a proof of concept. They separated this judgement from the idea, and they said they would be willing to adopt the final product as described in the pitch.

Discussion. The condition that participants attached to their trust is the condition that the architecture is built to satisfy, since the assistant's trustworthiness is meant to rest on the boundary and the approval steps behaving as promised and not on its good intentions. It is also the condition that the defects of the previous section threaten, which means that the same promise carries both the favourable result and the work that remains. The result is in the same direction as the foundational questionnaire, in which 65 percent of 374 respondents rated a workspace-only assistant more trustworthy than a full-access one,

although the two questions differ and the figures should not be combined. **Limits.** The participants are 30 people reached through the team's own contacts and are not a representative sample. Responses were gathered in conversation and summarised by the team, so no individual scores or recordings are available. The responses were not tallied by background, so no figure is claimed for any group. The reasons of the four participants who did not express trust were not recorded. The responses measure stated attitudes after a pitch and a short trial and not sustained use. They are best read as directional evidence.

9.6 E6: Exploratory Persona Simulation

Method. To test the idea under sceptical review and to expose objections that friendly conversation might not, 15 simulated personas, including students, developers, a freelancer, employees and deliberately sceptical non-adopters, were given a neutral description of the idea and a private copy of the prototype, and each reported its first impressions, what it did, its ratings and its objections in a structured report. Each worked independently and the agents did not see each other's reports. The study uses language-model-based simulated user agents to explore potential user perceptions and behavioural patterns. The participants are simulated personas and not recruited human subjects, so the results are exploratory synthetic evidence and must not be presented as empirical human-subject findings or as statistically representative of any population. They are reported here only as a stress test of the argument.

Table 9.7: Persona simulation results.

Question	Result
Would an assistant limited to chosen folders be trusted more than one with full access?	6 agree, 9 partly, 0 disagree
Would that make them more willing to use it?	1 agree, 14 partly, 0 disagree
Does the limitation cost too much usefulness?	6 no, 3 partly, 6 cannot say, 0 yes
Is wanting broad access a barrier for them?	10 agree, 2 partly, 3 disagree
Would they accept one mistake if undo is easy?	7 agree, 8 partly, 0 disagree
Ask before every change, or act and let me undo?	8 ask first, 6 mixed, 1 act then undo
Perceived relevance of the problem (1 to 5)	Mean 3.27, median 3
Perceived value of the solution (1 to 5)	Mean 2.8, median 3
Adoption	14 might try, 1 would not use, 0 would use or recommend

Discussion. The simulation is more reserved than the observations of real people, and that is its use. No persona disagreed that bounding scope would raise trust, which agrees in direction with E5, but only 1 of 15 would be clearly more willing to use it, and most attached conditions. Concerns were raised by all 15: 13 described failed, wrong or partial answers, and 12 noticed that a cloud key appeared as already saved in a clean copy, which could not be resolved as a defect or as an artefact of the shared test machine. Approval fatigue and long-term retention cannot be observed in a single session and were not. **Limits.** The personas are played by one underlying model, so their views are not independent, and such agents can be agreeable. The sample is small, no statistical test is used, and defects of the prototype, notably the failures of the folder box, probably coloured the personas' reactions. The results are therefore a check on the reasoning and are not evidence of adoption.

9.7 Discussion

9.7.1 Findings against the Central Hypothesis

Table 9.8 sets the evidence against the three parts of the central hypothesis. The classification is a description of the weight of the first-phase evidence and not a statistical test.

Table 9.8: Assessment of the central hypothesis.

Part	Evidence	Assessment
Restricting scope increases trust	Questionnaire (65 percent of 374); E5 (26 of 30, 10 fully); E6 (6 agree, 9 partly, none disagree); E4 shows the boundary mechanisms hold under hostile use	Supported in direction; conditional on exact behaviour
Increases willingness to adopt	E5: willingness to adopt the final product as pitched; E6: 14 might try, 1 clearly more willing	Partly supported; rests on stated intention
Without meaningfully reducing usefulness	E4 and E6 show the limits were not what users objected to; no task measure exists yet	Not yet tested

On this evidence the first part is supported in direction and the second is partly supported, while the third cannot be judged until usefulness is measured on tasks. The hypothesis as a whole is therefore encouraged by the first phase and is not yet confirmed. In the stricter reading of the simulation, where

only a clear agreement counts, the adoption part would not be supported, which is a reason to give the second phase's controlled study the greatest weight.

9.7.2 Findings against the Research Question

The research question asks whether an assistant restricted to granted workspaces can deliver effective multi-step task automation while improving trust without reducing usefulness. The prototype shows that the restriction can be built as structure and not as policy, and that it can be built cheaply, adding well under a millisecond to a call. It shows that multi-step plans, tidying, cited answers, mail, search, drafting, voice and remote control can all run through the same three enforcement points. It does not yet show that the automation is effective for users, because that needs the task study, and the defects of E4 show that reliability, and not the restriction, is what currently limits the experience.

9.7.3 Threats to Validity

The evidence is of four kinds, each with its own weakness. The deterministic tests cannot exercise a real model. The interface campaign used automated agents that cannot hear and cannot feel, and one of the machines lacked the embedding model. The observations of 30 people were gathered by the team, who built the system and presented it, which risks a favourable response, and were not recorded with individual scores. The simulation is synthetic. These weaknesses point in different directions, which is why the four are reported separately and not merged, and none is presented as the controlled study that the second phase will provide.

9.7.4 Evaluation Not Yet Done

The following parts of the evaluation methodology of Chapter 5 have not been carried out and are planned for the second phase: task-based measurement with and without the system, the subjective workload instrument, the usability and trust questionnaires, approval fatigue over repeated sessions, measurement of synthesis faithfulness and citation accuracy with a real model, and the voice evaluation with the four-second latency target.

9.8 Conclusion

The first phase shows that the enforcement the hypothesis relies on can be built, runs at negligible cost and holds under hostile use, and that prospective users who were asked reacted to the idea with trust that was conditional on the system doing exactly what it says. It also shows that the prototype does not yet always do exactly what it says: the interface campaign found 71 distinct defects, one critical, and the

gap between a passing automated suite and a working interface was large. The evidence supports the direction of the hypothesis and its trust component, partly supports adoption, and leaves usefulness to be measured. The corrections of the defects found and the controlled study are the two main tasks of the second phase.

Chapter 10 Conclusions

10.1 Summary of the Work Done

The project set out to show that an assistant restricted to folders the user explicitly grants, which asks before it acts and can undo what it did, can be trusted and adopted without being much less useful than an assistant with full access. The first phase established the problem and the hypothesis through a questionnaire of 374 respondents and 13 interviews, reviewed 26 studies across seven themes, specified the requirements, and designed an architecture that turns the hypothesis into three enforcement points: a workspace broker that alone resolves paths, a policy engine that places every action in one tier, and a dispatcher that alone acts. It then built a working prototype of about 14,250 lines in eight stages, from the workspace foundation to an optional remote plane, and verified it with 554 automated tests, two hands-on rounds, a 27-session interface campaign and conversations with 30 prospective users.

10.2 Findings and Results

The enforcement layer works as designed and withstands adversarial use. Nothing was deleted in any session, nothing outside the chosen folder was reachable, and nothing left the computer without an approval, and these properties cost well under a millisecond per call. Prospective users reacted to the idea with conditional trust, with 26 of 30 expressing trust and 10 of them fully, and said they would adopt the final product as pitched. The prototype itself is not ready for users. The interface campaign found 71 distinct defects, including one critical defect in which undo after a name collision can lose a file, and showed that a passing automated suite can coexist with a largely broken interface. Answers to questions about files were sometimes unsupported or wrong, and the record of what the assistant had done was not visible enough.

10.3 Coverage of the Scope and Objectives

Table 10.1 states, for each objective of Chapter 2, whether it has been met.

Table 10.1: Status of the project objectives.

Objective	Status	Basis and remaining work
Operate strictly within granted folders	Largely met	Boundary holds in all tests; any drive can still be added, a chat can carry one folder's content to another, and a waiting remote plan outlived an unshared folder.

Table 10.1: continued from previous page

Objective	Status	Basis and remaining work
Reduce time spent on resumption and routine file work	Not yet measured	Needs the task study of the second phase.
Keep the user in control through preview, approval and undo	Largely met	Gates hold; undo can lose a file on a name collision.
Provide no code path to deletion	Met	No deletion operation exists and none was reachable in any session.
Make everything held visible and deletable	Partly met	The panel works; the activity record is not fully visible.
Support voice under the same gates	Met for gates	No new gate was needed; microphone control and recognition quality need work.
Evaluate on time, workload, usability and trust	Not yet done	Planned for the second phase.

The functional scope of Chapter 2 was covered in full as a prototype: every capability listed there exists and runs through the gates, including the optional remote plane. The scope was not covered in the sense of readiness, because several capabilities are unreliable, and two of the four deliverables are not yet complete, being the user manual, which will be written once the interface has stabilised, and the evaluation report, which depends on the study of the second phase.

10.4 Challenges Faced

Four challenges shaped the first phase. The first was the gap between component tests and a working interface. The first hands-on round found that nearly every feature was unreachable although all unit tests passed, and the interface campaign later found 71 defects, which taught that the interface must be tested as a whole and that each defect needs a regression test. The second was the nondeterminism of a small local model, which sometimes gave unsupported or wrong answers, and which the gates mitigate but cannot cure. The third was the constraints of the interface framework, which required hidden controls to be triggered by scripts for some interactions and made voice behaviour, such as repeated speech, speaking the model's label, and losing the thread of a conversation, difficult to get right, and these were corrected in turn. The fourth was the size of the task for a team of three in one semester, which required the highest-risk components to be built first and the evaluation to be deferred.

10.5 What Has Been Left Out and Why

The controlled evaluation, the user manual, the measurement of approval fatigue and the quality evaluation of synthesis and citations are left to the second phase, because each depends on a stable interface. Real audio, a cloud model answer, a real email and the relay's own side were not exercised in the interface campaign, because the testing agents could not hear, did not hold credentials and must not use real accounts. The 71 defects are known and recorded and have not yet been fixed.

10.6 Recommendations and Future Work

The first recommendation is to correct, before anything else, the defects that bear directly on the promises the system makes: make tidy and undo collision-safe so that a file is never overwritten and any step that cannot be restored is reported; refuse drive roots and system folders when adding a folder; stop the microphone completely when voice is off; recheck the folder grant when a waiting plan is approved from a phone; stop a chat from carrying one folder's content into another; and make the activity record and the page that reports changes truthful. The second is to make answers honest, so that an answer with no source is plainly labelled, a missing embedding model is detected and reported, and retrieval failures that say not found for facts that are present are removed. The third is to add a regression test for every defect found at the interface, and to run the interface scenarios of the campaign as part of the suite. The fourth is to carry out the controlled study of Chapter 5. Beyond the second phase, directions worth pursuing are configurable approval granularity, since the questionnaire found users evenly divided, onboarding that takes minutes and not tens of minutes, barge-in and lower latency in voice, and a check that a citation actually supports its claim and not only that it exists.

10.7 Plan for FYP-II

The second phase has about sixteen weeks. The plan in Table 10.2 orders the work so that the product is stable before it is evaluated, since a study of a prototype that is still changing would measure the changes and not the idea.

The principal risks to this plan are recruitment, which depends on the institution's networks as the risk analysis of Chapter 4 notes, and the possibility that correcting defects takes longer than allowed. The mitigation is that the order of work places stability first, so that if the study must be smaller it can be reduced in size without being run on an unstable product, and that the defect work is limited to those defects that bear directly on the project's promises.

Table 10.2: Plan of work for the second phase.

Weeks	Work	Deliverable and exit check
1 to 2	Correct the defects that bear on the promises: collision-safe tidy and undo, drive and system folder refusal, microphone control, remote grant recheck, cross-folder chat memory. Add a regression test for each.	All critical and high defects of the boundary and recovery groups closed; the suite grows by the regression tests.
3 to 4	Honest answers and visibility: label unsupported answers, detect and report a missing embedding model, repair false not-found answers, show a readable activity view and a live record of what left the computer.	Re-run of the campaign scenarios for these groups shows no repeat of the defects.
5 to 6	Onboarding and interface: guard-rails and background indexing in the folder box, voice panel layout, no silently lost messages, visibility of new files. Voice latency and barge-in.	Medium defects of the interface group closed; time for a new user to reach a first answer measured.
7	Re-run of the full interface campaign, including the remote plane's relay side, retention of chats and removal of the remote plane.	Updated defect register; stage gates that were not tested are now tested.
8	User manual (Chapter 8).	Complete manual written from the stabilised interface.
9 to 10	Design the controlled study: tasks, instruments for workload, usability and trust, consent and ethics approval, recruitment through the institution. Pilot with a small group.	Study protocol approved by the supervisor; pilot completed and the instruments revised.
11 to 13	Conduct the study: task completion with and without the system, repeated sessions for approval fatigue, measurement of synthesis faithfulness and citation accuracy, voice latency against the four-second target.	Data collected for the planned number of participants.
14	Analyse the results and classify the central hypothesis and each supporting claim against the criteria fixed beforehand.	Analysis tables and figures.
15	Complete the report: full Chapter 9, final Chapter 10, updated manual.	Final report draft to the supervisor.
16	Final fixes, demonstration preparation and defence.	Demonstration build and final submission.

10.8 Conclusion of the Report

The first phase has produced a prototype in which the central idea, that an assistant's scope can be limited by construction and its actions made reviewable and reversible, is implemented and holds under hostile use at negligible cost, and prospective users who were asked reacted with trust that depended on the system doing exactly what it says. The same phase showed that the prototype does not yet do so everywhere, and its defects are recorded. What remains is to make the product reliable and to measure, with participants and on tasks, whether the trust it earns comes at an acceptable cost in usefulness. That is the work of the second phase.

Bibliography

[1] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs." Available: https://langchain-ai.github.io/langgraph/ [Accessed September 06, 2026], 2024.

[2] "Ollama: Get up and running with large language models locally." Available: https://ollama.com/ [Accessed September 06, 2026], 2024.

[3] SQLite Consortium, "SQLite: Self-contained, serverless, zero-configuration, transactional SQL database engine." Available: https://www.sqlite.org/ [Accessed September 06, 2026], 2024.

[4] United Nations, "Transforming our world: The 2030 agenda for sustainable development." Available: https://sdgs.un.org/2030agenda [Accessed September 06, 2026], 2015.

[5] S. T. Iqbal and E. Horvitz, "Disruption and recovery of computing tasks: Field study, analysis, and directions," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '07)*, pp. 677–686, ACM, 2007. DOI: 10.1145/1240624.1240730.

[6] D. Hu and S. W. Lee, "ScreenTrack: Using a visual history of a computer screen to retrieve documents and web pages," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*, pp. 1–13, ACM, 2020. DOI: 10.1145/3313831.3376753.

[7] C. Paduraru, P.-L. Bouruc, and A. Stefanescu, "A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance," in *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2026)*, SciTePress, 2026.

[8] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song, "The attack and defense landscape of agentic AI: A comprehensive survey," in *Proceedings of the 35th USENIX Security Symposium (USENIX Security '26)*, 2026.

[9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[10] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[11] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, pp. 1–22, ACM, 2023. DOI: 10.1145/3586183.3606763.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023.

[13] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[14] A. M. Deshmukh and R. Chalmeta, "User experience and usability of voice user interfaces: A systematic literature review," *Information*, vol. 15, no. 9, p. 579, 2024. DOI: 10.3390/info15090579.

[15] F. L. I. Dutsinma, D. Pal, S. Funilkul, and J. H. Chan, "A systematic review of voice assistant usability: An ISO 9241–11 approach," *SN Computer Science*, vol. 3, no. 4, p. 267, 2022. DOI: 10.1007/s42979-022-01172-3.

[16] Y. Ma, X. Zhang, X. Zhang, N. Bartłomiejczyk, S. Je, A. Holzer, M. Fjeld, and A. M. Butz, "Beyond words: Measuring user experience through speech analysis in voice user interfaces," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, ACM, 2026. DOI: 10.1145/3772318.3791747.

[17] X. Zhang, Y. Ma, K. Li, Y. Zhou, M. Fjeld, and M. Obaid, "Exploring the design of human speech indicators to enhance waiting experience in voice user interface," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, ACM, 2025. DOI: 10.1145/3706598.3713090.

[18] J. Wang, X. Zhang, Y. Ma, and M. Fjeld, "Mitigating response delays in free-form conversations with LLM-powered intelligent virtual agents," in *Proceedings of the 7th ACM Conference on Conversational User Interfaces (CUI '25)*, ACM, 2025. DOI: 10.1145/3719160.3736636.

[19] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proceedings*

of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '25), 2025.

[20] E. A. González, R. Rothkopf, S. Lerner, and N. Polikarpova, "HiLDe: Intentional code generation via human-in-the-loop decoding," in *Proceedings of the 2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '25)*, 2025.

[21] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "When to show a suggestion? integrating human feedback in AI-assisted programming," in *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI-24)*, vol. 38, pp. 10137–10144, 2024. DOI: 10.1609/aaai.v38i9.28878.

[22] S. Chidambaram, L. E. Li, M. Bai, X. Li, K. Lin, X. Zhou, and A. C. Williams, "Socratic human feedback (SoHF): Expert steering strategies for LLM code generation," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 15491–15502, 2024. Available: https://aclanthology.org/2024.findings-emnlp.908/.

[23] H. Jin and H. Chen, "Are LLMs reliable code reviewers? systematic overcorrection in requirement conformance judgement," *Automated Software Engineering*, vol. 33, p. 90, 2026. DOI: 10.1007/s10515-026-00638-5.

[24] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pp. 1906–1919, 2020. DOI: 10.18653/v1/2020.acl-main.173.

[25] W. Kryściński, B. McCann, C. Xiong, and R. Socher, "Evaluating the factual consistency of abstractive text summarization," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pp. 9332–9346, 2020. DOI: 10.18653/v1/2020.emnlp-main.750.

[26] A. Pagnoni, V. Balachandran, and Y. Tsvetkov, "Understanding factuality in abstractive summarization with FRANK: A benchmark for factuality metrics," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2021)*, pp. 4812–4829, 2021. DOI: 10.18653/v1/2021.naacl-main.383.

[27] P. Laban, W. Kryściński, D. Agarwal, A. R. Fabbri, C. Xiong, S. Joty, and C.-S. Wu, "SummEdits: Measuring LLM ability at factual reasoning through the lens of summarization," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pp. 9662–9676, 2023. DOI: 10.18653/v1/2023.emnlp-main.600.

[28] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023. DOI: 10.1162/coli_a_00486.

[29] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pp. 6465–6488, 2023. DOI: 10.18653/v1/2023.emnlp-main.398.

[30] N. F. Liu, T. Zhang, and P. Liang, "Evaluating verifiability in generative search engines," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7001–7025, 2023. DOI: 10.18653/v1/2023.findings-emnlp.467.

Chapter A Research Instruments and Response Distributions

This appendix records the instruments used in the foundational research and the complete response distributions summarised in Chapters 2 and 3. It is included so that the findings cited throughout the report can be examined at the level of the underlying data.

A.1 Research Method

The foundational research comprised two instruments. A questionnaire of seventeen items drew 374 valid responses, covering the respondent's context, the problem, design and trust preferences, and feature demand, with attitudinal items using a five-point scale. An interview guide coded thirteen in-person conversations on five axes: the last occasion on which the participant lost their place in multi-file work, the workaround they already employ, their reaction to the concept, the effect a trust-breaking mistake would have, and whether they would continue using the system.

Recruitment was through the institution's student and practitioner networks, and the questionnaire ran from 22 to 27 August 2026. Four limitations apply and are stated because they bound every conclusion drawn from this data. The sample is self-selected and drawn from a single institution. Students predominate, so the professional signal rests on a smaller subsample. The system was described in text rather than demonstrated, so predicted use and predicted churn are stated intentions rather than observed behaviour. Some free-text responses contain near-identical phrasings, which suggests that certain answers were presented as suggested text, so theme counts should be read as indicative.

A.2 Respondent Profile

Table A.1 gives the composition of the sample and the frequency with which respondents perform multi-file work.

Table A.1: Respondent role and work frequency.

Response	Count	Percent
Role		
Student	302	80.7
Developer or software engineer	47	12.6
Freelance	12	3.2
Employed	7	1.9
Other	6	1.6

Table A.1: continued from previous page

Response	Count	Percent
Frequency of multi-file project work		
Daily	147	39.3
A few times a week	159	42.5
A few times a month	68	18.2
Prior use of an AI assistant		
Yes, regularly	172	46.0
Yes, occasionally	115	30.7
Tried once or twice	48	12.8
Never	39	10.4

A.3 Headline Findings

The nine findings established by the research, referenced throughout Chapters 2 and 4, are as follows.

- **F1.** The reorientation problem is real. 44 percent of respondents struggle to recall what they were doing on returning to a project after a break, and only 7 percent never do
- **F2.** File disorder outpaces manual cleanup for 59 percent of respondents
- **F3.** The problem costs real time at moderate scale, with 42 percent losing more than an hour a week and 14 percent more than three hours
- **F4.** The work is unautomated because of distrust of automation at 36 percent and lack of know-how at 28 percent, rather than because the tasks do not exist
- **F5.** The bounded-workspace hypothesis holds. 65 percent rate a workspace-only assistant more trustworthy than a full-access one, with a mean of 3.8, and 40 percent prefer local-first operation
- **F6.** Users are forgiving of a single mistake. 65 percent would continue using the system after one wrong suggestion and only 10 percent would abandon it
- **F7.** The principal retention risks are habit fit at 27 percent and an untrusted mistake at 24 percent; approval fatigue is real but smaller at 10 percent
- **F8.** Feature demand is concentrated on session resume, notes synthesis and folder tidy, with speech last by a wide margin
- **F9.** The interviews confirm the questionnaire and add effortless onboarding as a named condition

for continued use

A.4 Segment Analysis

Table A.2 presents selected measures by respondent segment. Role does not predict the interaction preference, since students and developers give near-identical workspace-trust ratings and identical approval-first shares. Experience moves it modestly: regular users of assistants prefer approval-first less often than light or non-users, which is consistent with heavier users being more comfortable delegating. The effect is small and neither group approaches unanimity, so the practical conclusion is unchanged, which is that the interaction model must be a setting rather than a fixed decision.

Table A.2: Selected measures by respondent segment.

Segment	**n**	**Workspace trust (mean)**	**Prefer approval-first**	**Prefer local-first**
All respondents	374	3.8	51%	40%
Students	302	3.8	51%	41%
Developers and engineers	47	3.7	51%	36%
Regular assistant users	172	3.9	44%	40%
Light or non-users	87	3.8	59%	41%

A.5 Interview Findings

Figure A.1 presents the coded interview responses. Ten of the thirteen participants reacted positively to the concept, with the remainder split between conditional, mixed and one negative response. Eight of thirteen said a single mistake would not stop them, several qualifying this with caution, while two said a trust-breaking mistake would cause them to stop. Six said outright that they would continue using the system and four said they would do so conditionally, with effortless onboarding named as the most common condition, followed by privacy and by the system reaching sufficient maturity.

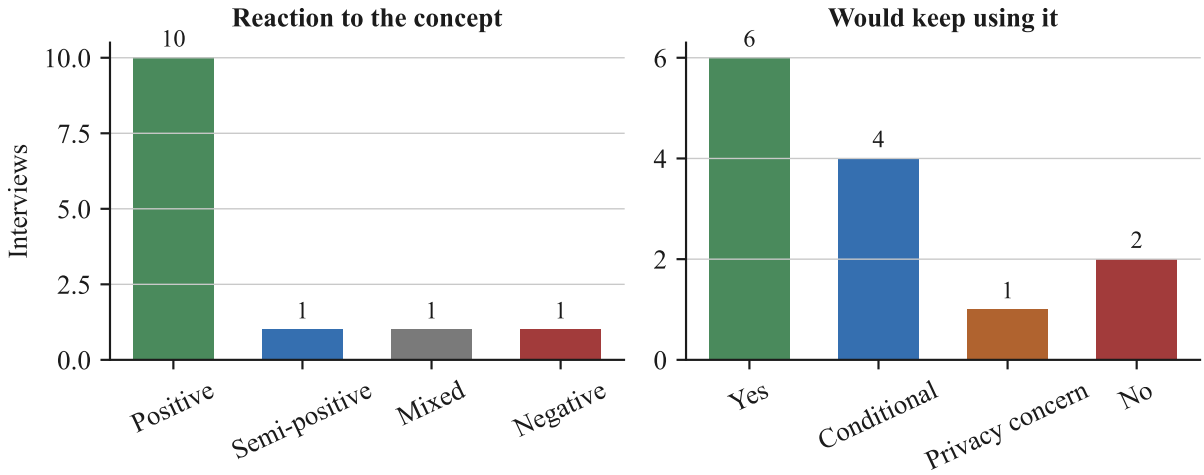


Figure A.1: Coded interview responses (13 interviews).

The workarounds participants already employ were consistent: informal notes written by hand or with an assistant to record where they left off, repeated manual restructuring because every project has a different structure, and simply tolerating the friction because automating it appears to cost more than it saves.

A.6 Additional Response Distributions

Figures A.2, A.3, A.4, A.5, A.6, A.7 and A.8 present the remaining distributions underlying the findings cited in Chapters 2 and 4.

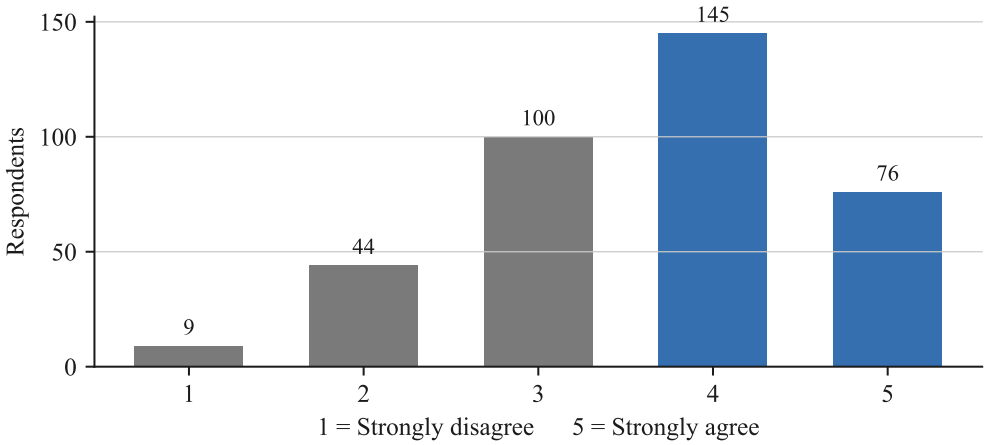


Figure A.2: Agreement that files become disorganised.

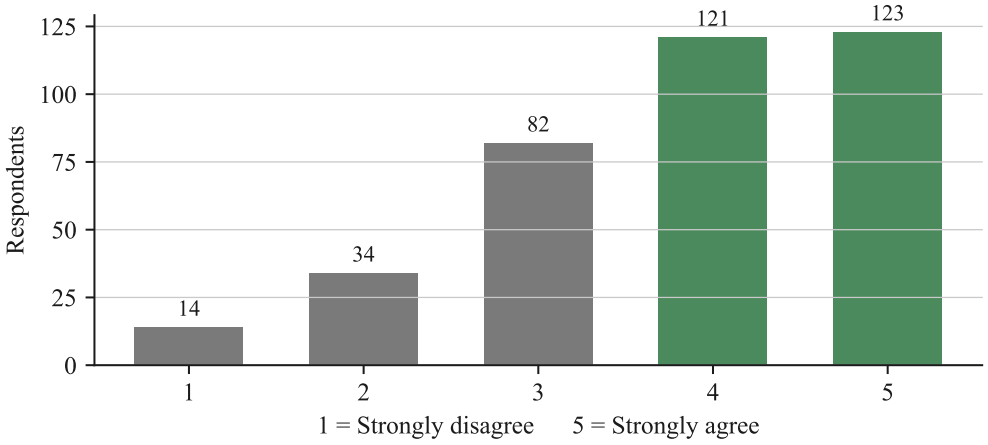


Figure A.3: Agreement on workspace-only trust.

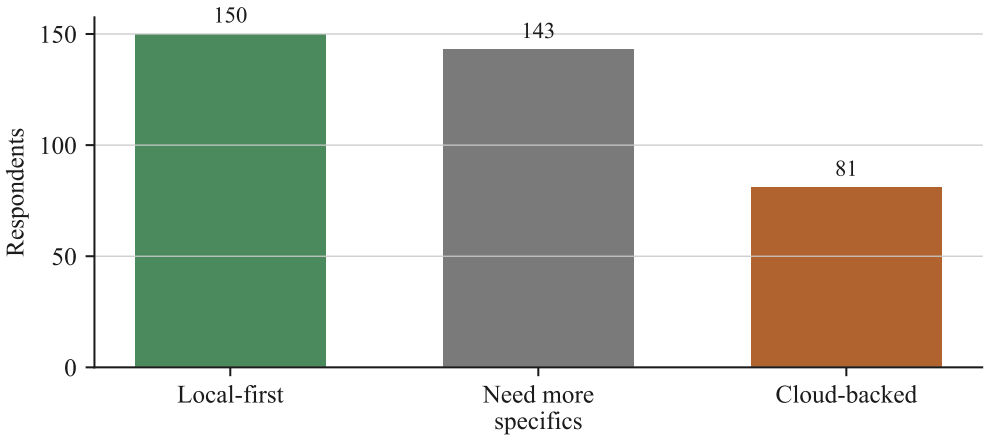


Figure A.4: Local versus cloud preference.

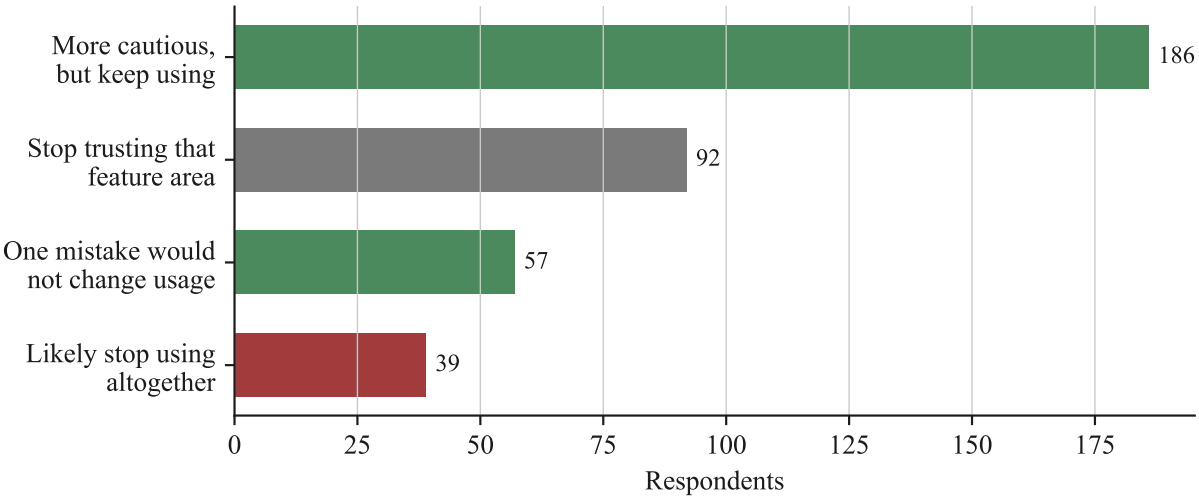


Figure A.5: Response to a single wrong or unwanted suggestion.

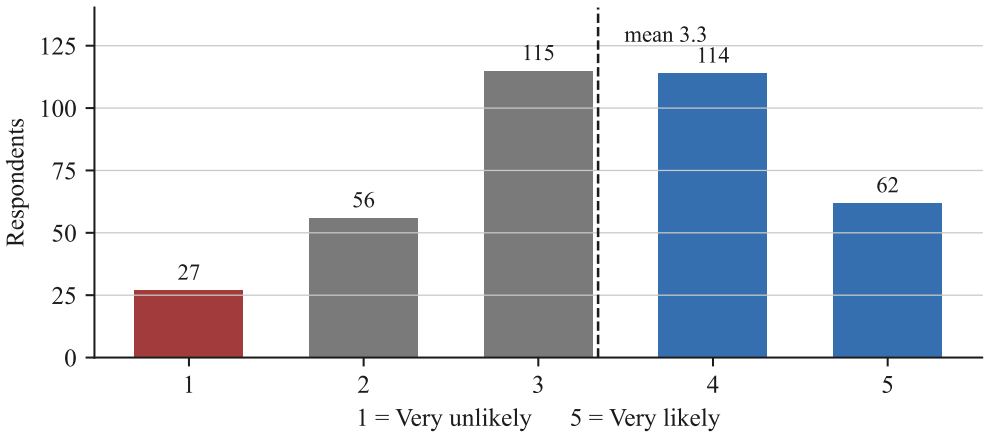


Figure A.6: Likelihood of continued use after the novelty.

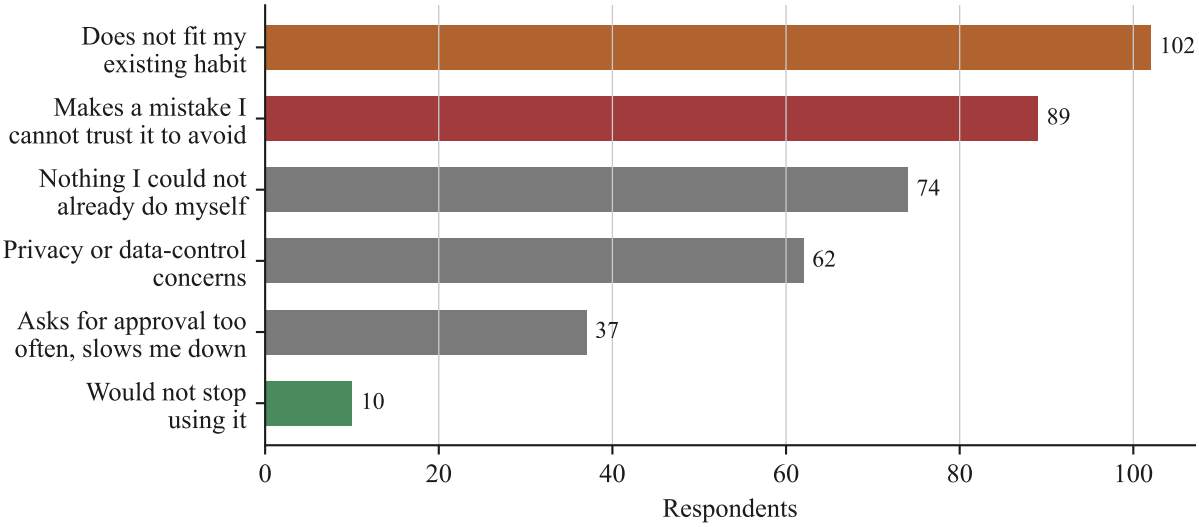


Figure A.7: Most likely reason for stopping after a week.

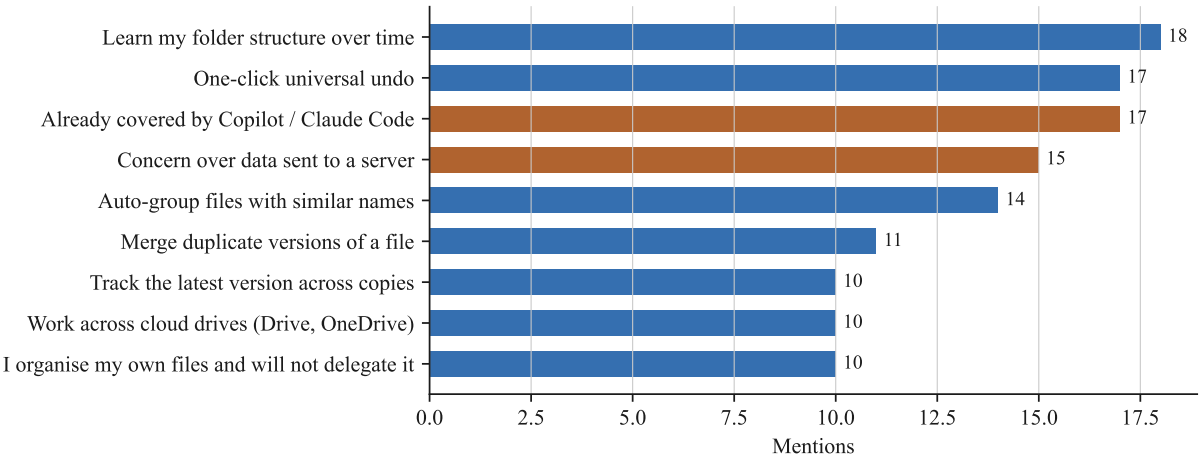


Figure A.8: Recurring themes in free-text responses.